Hochschule Karlsruhe
University of
Applied Sciences

HKA

Hochschule Karlsruhe

– Fakultät für Maschinenbau und Mechatronik–

F & E-Projekt Wintersemester 25/26

Modellbasierte Keypoint-Filterung für robustes Image-Based Visual Servoing

Projektdokumentation zum

F & E-Projekt Wintersemester 25/26

vorgelegt am 31. März 2026 von

Paul Glaser, B. Eng (glpa1013 / Matrikelnr. 100215)

Tim Schäfer, B. Eng (scti1057 / Matrikelnr. 71606)

Felix Wietschel, B. Eng (wife1013 / Matrikelnr. 74940)

Erstprüfer	:	Prof. Dr.-Ing. Christian Friedrich
Zweitprüfer	:	Dominik Werman, M. Sc.
Projektkoordinator	:	Prof. Dr.-Ing. Frank Pöhler

Inhaltsverzeichnis

1 Einleitung **1**
- 1.1 Motivation . . . 1
- 1.2 Problemstellung . . . 1
- 1.3 Herausforderung bei reflektierenden Objekten . . . 2
- 1.4 Ziel des Projekts . . . 3
- 1.5 Beitrag der Arbeit . . . 3

2 Grundlagen **4**
- 2.1 Bildbasierte Roborerregelung (IBVS) . . . 4
- 2.2 Interaction Matrix und Image Jacobian . . . 5
- 2.3 Keypoint Detection und Feature Matching . . . 6
- 2.4 Zustandsmodelle für visuelle Features . . . 7

3 Zustandsschätzung und Filtermethoden **8**
- 3.1 Der lineare Kalman-Filter . . . 8
- 3.2 Ausreißererkennung mittels Mahalanobis-Distanz . . . 10
- 3.3 Erweiterte Kalman-Filter für nichtlineare Systeme . . . 11
- 3.4 Der 1€-Filter . . . 14

4 Verwandte Arbeiten **15**
- 4.1 Robuste und stabilisierende Ansätze im Image-Based Visual Servoing . 15
- 4.2 Filter- und schätzbasierte Ansätze in Visual Servoing . . . 16
- 4.3 Moderne lernbasierte Ansätze . . . 17
- 4.4 Forschungslücke und Abgrenzung der eigenen Arbeit . . . 18

5 Systemarchitektur und Methode **19**
- 5.1 Systemübersicht . . . 19
- 5.2 Hardwareaufbau . . . 21
- 5.3 Softwarearchitektur . . . 22
- 5.4 Feature Detection . . . 23
- 5.5 Keypoint-Datenbank . . . 25

6 Modellbasierte Keypoint-Vorhersage **28**
- 6.1 Motivation der modellbasierten Vorhersage . . . 28
- 6.2 Nutzung der Roboterbewegung und Kamerakinematik . . . 29
- 6.3 Mathematisches Bewegungsmodell der Bildmerkmale . . . 29
- 6.4 Zeitdiskrete Vorhersage der Keypoint-Positionen . . . 30
- 6.5 Tiefenmodell und Tiefenberechnung . . . 30
- 6.6 Aktive Keypoint-Menge und geometrische Konsistenz . . . 31
- 6.7 Zusammenspiel von Vorhersage und Messupdate . . . 31

6.8 Einordnung in das Gesamtsystem . . . 31
6.9 Zusammenfassung . . . 32

7 Filter- und Entscheidungslogik 33
7.1 Ziel des Filters . . . 33
7.2 Dynamisches Keypoint-Tracking und Active-Set-Architektur . . . 33
7.3 Architekturen und kinematische Vorsteuerung . . . 34
7.4 Systemtuning und Parametrisierung . . . 34
7.5 Fehlerbehandlung und Relokalisation . . . 35
7.6 Stabilität und Vermeidung von Singularitäten . . . 35

8 Feature Confidence und Switching Strategie 36
8.1 Definitionen und theoretische Grundlagen . . . 36
8.2 Prädiktionsbasiertes Local-Rescue-Matching . . . 37

9 Regelung des Roboterarms 39
9.1 Regelungsprinzip . . . 39
9.2 Einbindung realer und gefilterter Keypoints . . . 40
9.3 Berechnung der kartesischen Stellgröße . . . 40
9.4 Tiefenverarbeitung . . . 41
9.5 Transformation in das Roboterframe . . . 41
9.6 Stabilisierung des Regelkreises . . . 42
9.7 Verhalten bei Featureverlust und unsicheren Beobachtungen . . . 42
9.8 Einordnung im Gesamtsystem . . . 43
9.9 Zusammenfassung . . . 43

10 Softwareimplementierung 44
10.1 Systemumgebung und Softwarebasis . . . 44
10.2 Paketstruktur des ROS2-Workspaces . . . 44
10.3 Implementierung der zentralen ROS2-Nodes . . . 45
10.4 Nachrichtenformate und Datenfluss . . . 47
10.5 Start- und Laufzeitsteuerung über den Session Manager . . . 48
10.6 Besonderheiten der Implementierung . . . 48
10.7 Zusammenfassung . . . 49

11 Benchmarking-Methodik und Evaluierungsmetriken 50
11.1 Definition des Versuchsraums . . . 50
11.2 Methodische Abgrenzung und Reduktion . . . 52
11.3 Design of Experiments (DoE) . . . 53
11.4 Versuchsplan und zeitliche Abschätzung . . . 54
11.5 Evaluierungsmetriken . . . 55

12 Ergebnisse 57
12.1 Datenbasierte Reduktion des Versuchsraums . . . 57
12.2 Zusatzanalyse des Local Rescue unter translatorischer Bewegung . . . 60
12.3 Rotationserweiterung auf Level b . . . 63
12.4 Zusammenfassung der Ergebnisse . . . 66

13 Fazit und Ausblick 68
13.1 Zusammenfassung . . . 68

13.2 Ausblick 69

A Anhang **71**

A.1 Ablaufdiagramme Filter 71

Abbildungsverzeichnis

5.1 Schematische Darstellung der IBVS-Pipeline. Die einzelnen Schritte sind modular aufgebaut, um eine flexible Erweiterung und Anpassung zu ermöglichen. 20

11.1 Obere Reihe: Visualisierung der Testszenarien II und III. Untere Reihe: Beispiele der verwendeten Stichproben/Bauteile für die Experimente.[1] . 51
11.2 Gesamter experimenteller Testaufbau (inkl. Brücke für Szenario II und Leuchtmittel für Szenario III; beispielhaft das Bauteil Typ 2).[2] 52

12.1 Level: a | Szen.: 2 | Bauteil: Typ 2 | Vergleich der Filterkonfigurationen unter Okklusion 58
12.2 Level: a | Szen.: 1 | Bauteil: Typ 1 | Mediane Pfadlänge über der Zeit . 58
12.3 Level: a | Szen.: 1 | Mediane translatorische Beschleunigung zur Bauteilauswahl 59
12.4 Level: a | Szen.: 3 | Success Rate zur Bauteilauswahl unter Lichtwechsel 60
12.5 Level: a | Szen.: 3 | Mediane translatorische Beschleunigung unter Lichtwechsel 60
12.6 Ergänzende Detailanalyse | Translatorische Bewegung | Lichtwechsel | Bauteil: Typ 2 | Einzelrun-Vergleich der translatorischen Geschwindigkeit und Beschleunigung 62
12.7 Ergänzende Detailanalyse | Translatorische Bewegung | Lichtwechsel | Bauteil: Typ 2 | Einzelrun-Vergleich des Image Based Visual Servoing (IBVS)-RMS-Fehlers 63
12.8 Level: b | Szen.: 1 | Bauteil: Typ 2 | Referenzverhalten unter Standardbedingungen 64
12.9 Level: b | Szen.: 2 | Bauteil: Typ 2 | Medianer Verlauf der Geschwindigkeits- und Beschleunigungsgrößen unter Okklusion 65
12.10 Level: b | Szen.: 2 | Bauteil: Typ 2 | Fehlerverlauf und Pfadlänge unter Okklusion 66

A.1 Ablaufdiagramm des KF. 71
A.2 Ablaufdiagramm des EKF. 72
A.3 Ablaufdiagramm des UKF. 72
A.4 Ablaufdiagramm des ESKF. 73
A.5 Ablaufdiagramm des 1€-Filters. 74

Tabellenverzeichnis

3.1 Auszug aus der χ^2-Verteilungstabelle für das Measurement-Gating. . . 10

11.1 Definierte System-Konfigurationen für das Benchmarking. 53
11.2 Matrix-Übersicht der Test-Kombinationen für Level a nach Konfiguration. 53
11.3 Matrix-Übersicht der Test-Kombinationen für Level b nach Konfiguration. 54
11.4 Zeitliche Abschätzung für 51 Kombinationen in Level a. 54
11.5 Zeitliche Abschätzung für 6 Kombinationen in Level b. 55

Abkürzungsverzeichnis

IBVS Image Based Visual Servoing
PBVS Position Based Visual Servoing
ROS2 Robotic Operating System 2
KF Kalman-Filter
UKF Unscented Kalman-Filter
EKF Extended Kalman-Filter
ESKF Error-State Kalman-Filter
CV Constant Velocity
DoF Degrees of Freedom
LR Local Rescue
SIFT Scale-Invariant Feature Transform
SURF Speeded-Up Robust Features
ORB Oriented FAST and Rotated BRIEF
TCP Tool Center Point
DLS Damped Least Squares
EMA Exponential Moving Average
ROI Region of Interest
TF Transform

1 Einleitung

1.1 Motivation

In der industriellen Robotik spielt die präzise und flexible Positionierung von Werkzeugen oder Greifern eine zentrale Rolle. Klassische Automatisierungssysteme basieren häufig auf festen Programmen oder vorab definierten Positionen im Raum. Diese Ansätze stoßen jedoch an ihre Grenzen, sobald sich Umgebungsbedingungen ändern oder Objekte nicht exakt an der erwarteten Position liegen. Eine vielversprechende Lösung stellt die visuelle Roboterführung dar. Dabei wird ein Kamerasystem genutzt, um visuelle Informationen aus der Umgebung zu erfassen und diese direkt in den Regelkreis des Roboters einzubeziehen. Ein besonders verbreiteter Ansatz ist IBVS. Hierbei wird die Bewegung des Roboters so geregelt, dass bestimmte Bildmerkmale – sogenannte Features oder Keypoints – im Kamerabild eine gewünschte Zielposition erreichen. Der Vorteil dieses Ansatzes liegt darin, dass keine vollständige dreidimensionale Rekonstruktion der Szene erforderlich ist. Stattdessen wird die Roboterbewegung direkt aus der Veränderung der Bildmerkmale berechnet. Dadurch können viele Aufgaben mit vergleichsweise geringer Modellierung der Umgebung gelöst werden. In realen industriellen Anwendungen treten jedoch häufig Probleme auf, die die Zuverlässigkeit der Feature-Erkennung beeinträchtigen. Besonders bei metallischen oder glänzenden Bauteilen kommt es durch starke Reflexionen, Überbelichtung oder ungünstige Beleuchtung zu instabilen Bildmerkmalen. Keypoints können temporär verschwinden, falsch erkannt werden oder ihre Position stark verändern. Dadurch wird der Regelkreis instabil oder verliert vollständig seine Referenzpunkte. Diese Problematik stellt eine zentrale Herausforderung für den praktischen Einsatz von IBVS in industriellen Umgebungen dar.

1.2 Problemstellung

Das grundlegende Problem bei der Verwendung von bildbasierten Merkmalen zur Robotersteuerung besteht darin, dass die Stabilität des Regelkreises direkt von der Zuverlässigkeit der Feature-Erkennung abhängt. Sobald relevante Keypoints nicht mehr korrekt erkannt werden können, entstehen Fehler in der Berechnung der Bildfehler und damit in den Steuerbefehlen des Roboters. In industriellen Szenarien treten mehrere Störeinflüsse auf:

- starke Reflexionen auf metallischen Oberflächen
- Überbelichtung einzelner Bildbereiche
- temporäre Okklusion durch Greifer oder Werkzeuge
- Veränderungen der Beleuchtung
- perspektivische Änderungen während der Roboterbewegung

Diese Effekte können dazu führen, dass zuvor zuverlässig erkannte Bildmerkmale plötzlich verschwinden oder falsch zugeordnet werden. Klassische Feature-Tracking-Methoden sind häufig nicht in der Lage, solche Situationen robust zu behandeln. Ein weiteres Problem besteht darin, dass viele IBVS-Systeme ausschließlich auf den aktuell erkannten Bildmerkmalen basieren. Sobald diese Merkmale nicht mehr verfügbar sind, kann der Regelkreis keine stabilen Steuergrößen mehr berechnen. Daher besteht die zentrale Herausforderung darin, ein System zu entwickeln, das auch bei temporär fehlenden oder unzuverlässigen Keypoints weiterhin stabile Positionsinformationen bereitstellen kann.

1.3 Herausforderung bei reflektierenden Objekten

Reflektierende Oberflächen stellen in der industriellen Bildverarbeitung eine besondere Schwierigkeit dar. Metallische Bauteile oder polierte Oberflächen reflektieren Licht häufig sehr stark und können dadurch lokale Überbelichtungen im Kamerabild verursachen. Diese Effekte führen zu sogenannten Specular Highlights, bei denen einzelne Bildbereiche deutlich heller erscheinen als ihre Umgebung. Für viele Feature-Detektionsalgorithmen stellt dies ein Problem dar, da sich die lokalen Bildstrukturen stark verändern können. Ein Keypoint, der unter normalen Beleuchtungsbedingungen zuverlässig erkannt wird, kann unter Reflexion plötzlich verschwinden oder stark verschoben erscheinen. Zusätzlich verändert sich das Erscheinungsbild reflektierender Oberflächen stark in Abhängigkeit vom Blickwinkel der Kamera. Bereits kleine Bewegungen des Roboters können dazu führen, dass sich Reflexionen im Bild verschieben oder neue Reflexionspunkte entstehen. Diese Effekte erschweren sowohl die stabile Feature-Erkennung als auch das anschließende Feature-Tracking. In der Folge kann es zu folgenden Problemen kommen:

- Verlust von Keypoints während der Bewegung
- falsche Zuordnung von Features
- instabile Bildfehler im Regelkreis
- Veränderungen der Beleuchtung
- unruhige oder falsche Roboterbewegungen

In vielen industriellen Anwendungen ist es jedoch nicht möglich, die Beleuchtungsbedingungen vollständig zu kontrollieren oder die Oberflächen der Bauteile zu verändern. Daher ist es notwendig, Methoden zu entwickeln, die auch unter solchen Bedingungen zuverlässig arbeiten.

1.4 Ziel des Projekts

Ziel dieses Projekts ist die Entwicklung eines robusten Systems zur Stabilisierung von IBVS unter schwierigen Bildbedingungen. Insbesondere soll ein Ansatz entwickelt werden, der reale Bildfeatures mit modellbasierten Vorhersagen kombiniert. Die zentrale Idee besteht darin, zusätzlich zu den tatsächlich im Kamerabild erkannten Keypoints eine modellbasierte Vorhersage der erwarteten Featurepositionen zu berechnen. Diese Vorhersage kann auf Grundlage der bekannten Roboterkinematik sowie eines mathematischen Bewegungsmodells erfolgen. Das System nutzt somit zwei Informationsquellen:

- reale Keypoints aus dem Kamerabild
- modellierte Keypoints aus der Bewegung des Roboters

Durch die Kombination beider Informationsquellen kann das System auch dann stabile Positionsinformationen liefern, wenn reale Features temporär nicht erkannt werden. Zusätzlich wird eine Filter- und Entscheidungslogik entwickelt, die die Qualität der verfügbaren Keypoints bewertet und dynamisch entscheidet, welche Features für die Regelung verwendet werden sollen.

1.5 Beitrag der Arbeit

Der Beitrag dieser Arbeit besteht in der Entwicklung eines hybriden Ansatzes zur Stabilisierung von Feature-basierten Roboterregelungen in visuellen Servoing-Systemen. Die wesentlichen Beiträge lassen sich wie folgt zusammenfassen:

- Entwicklung einer Architektur zur Kombination von realen und modellierten Keypoints
- Aufbau einer lokalen Keypoint-Datenbank zur Speicherung von Featureinformationen
- Featurewiedererkennung durch Positionsschätzung basierend auf der Roboterkinematik
- Integration eines Filters zur Bewertung der Featurequalität
- Entwicklung einer Entscheidungsstrategie zur dynamischen Auswahl stabiler Features
- Implementierung eines modularen Systems zur Integration in eine Robotic Operating System 2 (ROS2)-basierte Robotiksoftware

2 Grundlagen

In diesem Kapitel werden die theoretischen Grundlagen vorgestellt, die für das Verständnis der in dieser Arbeit entwickelten Methode erforderlich sind. Dazu gehören die Prinzipien der bildbasierten Roboterregelung, Verfahren zur Erkennung und Verfolgung visueller Merkmale sowie grundlegende Filtermethoden zur Stabilisierung von Messdaten.

Diese Konzepte bilden die Basis für die im weiteren Verlauf der Arbeit vorgestellte Architektur zur robusten Feature-basierten Robotersteuerung.

2.1 Bildbasierte Roborerregelung (IBVS)

Visual Servoing bezeichnet eine Klasse von Regelungsverfahren, bei denen visuelle Informationen aus Kamerabildern zur Steuerung eines Roboters verwendet werden. Ziel ist es, die Bewegung des Roboters so zu regeln, dass bestimmte visuelle Merkmale eine gewünschte Position im Kamerabild erreichen. [1]

Grundsätzlich lassen sich zwei Hauptansätze des Visual Servoing unterscheiden:

- Position Based Visual Servoing
- Image Based Visual Servoing

Beim Position Based Visual Servoing (PBVS) wird zunächst die vollständige Pose des Objekts im dreidimensionalen Raum geschätzt. Anschließend wird aus dieser Pose eine Roboterbewegung berechnet [1].

Beim IBVS hingegen erfolgt die Regelung direkt im Bildraum. Dabei werden Bildmerkmale wie Punkte, Linien oder Konturen als Referenzgrößen verwendet. Der Roboter wird so gesteuert, dass sich die Position dieser Merkmale im Kamerabild einer gewünschten Zielposition annähert [1, 2].

Der Vorteil von IBVS besteht darin, dass keine vollständige dreidimensionale Rekonstruktion der Szene erforderlich ist. Dadurch kann das Verfahren oft robuster gegenüber Modellierungsfehlern sein [1].

Der grundlegende Regelkreis von IBVS besteht aus folgenden Schritten:

- Aufnahme eines Kamerabildes
- Erkennung visueller Merkmale (Features)
- Berechnung des Bildfehlers zwischen aktueller und gewünschter Featureposition
- Berechnung einer Roboterbewegung zur Minimierung dieses Fehlers

Der Bildfehler wird dabei typischerweise als Differenz zwischen der aktuellen Position eines Features im Bild und seiner gewünschten Zielposition definiert [1].

2.2 Interaction Matrix und Image Jacobian

Ein zentrales Element von IBVS ist die sogenannte Interaction Matrix, auch als Image Jacobian bezeichnet. Sie beschreibt den Zusammenhang zwischen der Bewegung der Kamera und der zeitlichen Änderung der Bildmerkmale im Bildraum [1, 2].

Allgemein lässt sich dieser Zusammenhang durch

$$\dot{\mathbf{s}} = \mathbf{L}_s \, \mathbf{v}_c \tag{2.1}$$

beschreiben. Dabei gilt:

- $\dot{\mathbf{s}}$: zeitliche Änderung der Bildmerkmale
- $\mathbf{L}_s$: Interaction Matrix
- $\mathbf{v}_c$: Geschwindigkeitsvektor der Kamera

Der kamerabezogene Geschwindigkeitsvektor setzt sich aus translatorischen und rotatorischen Anteilen zusammen und kann geschrieben werden als

$$\mathbf{v}_c = \begin{bmatrix} v_x & v_y & v_z & \omega_x & \omega_y & \omega_z \end{bmatrix}^\top . \tag{2.2}$$

Für ein einzelnes Feature mit normierten Bildkoordinaten (x, y) und Tiefe Z ergibt sich die zugehörige Interaction Matrix zu [1]

$$\mathbf{L}_p = \begin{bmatrix} -\frac{1}{Z} & 0 & \frac{x}{Z} & xy & -(1+x^2) & y \\ 0 & -\frac{1}{Z} & \frac{y}{Z} & 1+y^2 & -xy & -x \end{bmatrix}. \tag{2.3}$$

Diese Matrix beschreibt, wie sich die Bildkoordinaten eines einzelnen Features in Abhängigkeit von der Kamerabewegung verändern. Dabei hängen die translatorischen Anteile direkt von der Tiefe Z ab, während die rotatorischen Anteile von der aktuellen Bildlage des Punktes beeinflusst werden [1].

Werden mehrere Features gleichzeitig betrachtet, so wird für jedes Merkmal eine eigene Interaction Matrix aufgestellt. Die gesamte Interaction Matrix ergibt sich dann aus den Einzelmatrizen:

$$\mathbf{L}_s = \begin{bmatrix} \mathbf{L}_{p,1} \\ \mathbf{L}_{p,2} \\ \vdots \\ \mathbf{L}_{p,n} \end{bmatrix}. \tag{2.4}$$

Für n Features besitzt die gesamte Interaction Matrix damit die Dimension $2n \times 6$. Da diese Matrix im Allgemeinen nicht quadratisch ist, kann sie in der Regel nicht direkt invertiert werden. Stattdessen wird ihre Pseudoinverse verwendet. Diese ermöglicht es, auch für überbestimmte oder unterbestimmte Gleichungssysteme eine Lösung für die Kamerageschwindigkeit zu berechnen. Im Kontext von IBVS wird dabei typischerweise diejenige Geschwindigkeit bestimmt, welche den Bildfehler im Sinne einer Least-Squares-Lösung bestmöglich reduziert [1, 2].

Zur Regelung wird der Bildfehler zwischen aktueller und gewünschter Merkmalslage definiert als

$$\mathbf{e} = \mathbf{s} - \mathbf{s}^*, \tag{2.5}$$

wobei $\mathbf{s}$ die aktuellen und $\mathbf{s}^*$ die gewünschten Bildmerkmale bezeichnet. Ein typischer IBVS-Regler ergibt sich dann zu [1]

$$\mathbf{v}_c = -\lambda \, \mathbf{L}_s^+ \, (\mathbf{s} - \mathbf{s}^*), \tag{2.6}$$

wobei $\lambda > 0$ die Regelverstärkung und $\mathbf{L}_s^+$ die Pseudoinverse der Interaction Matrix bezeichnet.

Das Ziel des Reglers besteht darin, den Bildfehler schrittweise zu reduzieren, sodass die aktuellen Bildmerkmale gegen ihre gewünschte Referenzlage konvergieren. Die Interaction Matrix bildet dabei die mathematische Grundlage, um aus einem Fehler im Bildraum eine geeignete Kamerabewegung abzuleiten [1].

2.3 Keypoint Detection und Feature Matching

Damit ein IBVS-System arbeiten kann, müssen zunächst stabile Bildmerkmale im Kamerabild erkannt werden. Diese Merkmale werden häufig als Keypoints oder Features bezeichnet.

Keypoints sind charakteristische Bildpunkte, die sich zuverlässig wiedererkennen lassen. Beispiele hierfür sind [3]:

- Ecken
- Kantenübergänge
- markante Texturpunkte

Moderne Feature-Detektionsverfahren berechnen zusätzlich sogenannte Deskriptoren, die die lokale Bildstruktur um einen Keypoint beschreiben. Diese Deskriptoren ermöglichen es, Features zwischen verschiedenen Bildern wiederzuerkennen [3].

Bekannte klassische Feature-Verfahren sind beispielsweise:

- Scale-Invariant Feature Transform (SIFT)
- Speeded-Up Robust Features (SURF)
- Oriented FAST and Rotated BRIEF (ORB)

Neben klassischen Verfahren existieren auch neuere Ansätze auf Basis neuronaler Netze, die häufig robustere Features liefern können. Ein Beispiel hierfür ist der Feature-Detektor xFeat, der speziell für robuste Feature-Erkennung entwickelt wurde [4].

Beim Feature Matching werden Deskriptoren miteinander verglichen, um zu bestimmen, welche Features in zwei Bildern zueinander gehören. Eine häufig verwendete Ähnlichkeitsmetrik ist die Cosinus-Ähnlichkeit oder die euklidische Distanz zwischen den Deskriptorvektoren [3].

Eine zuverlässige Feature-Erkennung ist eine zentrale Voraussetzung für stabile Visual-Servoing-Systeme.

2.4 Zustandsmodelle für visuelle Features

Um temporäre Ausfälle von Bildmerkmalen zu kompensieren, können Zustandsmodelle verwendet werden. Ein Zustandsmodell beschreibt die erwartete Entwicklung eines Systems über die Zeit [5].

Im Kontext von Visual Servoing kann ein solches Modell verwendet werden, um die zukünftige Position eines Features im Bild vorherzusagen. Grundlage dafür ist häufig die bekannte Bewegung des Roboters oder der Kamera [1, 6].

Ein einfaches Beispiel ist ein Bewegungsmodell, das davon ausgeht, dass sich ein Feature mit konstanter Geschwindigkeit bewegt. In diesem Fall kann die zukünftige Featureposition aus der aktuellen Position und der geschätzten Geschwindigkeit berechnet werden [7].

Komplexere Modelle können zusätzlich Informationen über die Roboterkinematik oder geometrische Transformationen zwischen verschiedenen Kamerapositionen berücksichtigen [2].

Solche Modelle ermöglichen es, auch dann eine Schätzung der Featureposition zu erhalten, wenn ein Feature im aktuellen Bild nicht direkt erkannt werden kann [5].

3 Zustandsschätzung und Filtermethoden

Wie in Kapitel 2 beschrieben, enthalten Messdaten aus Kamerasystemen häufig Rauschen, Ausreißer oder fallen durch externe Störeinflüsse wie Reflexionen temporär ganz aus. Um in solchen Situationen dennoch eine stabile Roboterregelung zu gewährleisten, sind robuste Filtermethoden zur Zustandsschätzung unerlässlich. In diesem Kapitel werden die theoretischen und mathematischen Grundlagen der in dieser Arbeit verwendeten Filter vorgestellt.

Im Anhang A.1 sind zu den im folgenden vorgestellten Filtern Ablaufdiagramme dargestellt.

3.1 Der lineare Kalman-Filter

Der Standard Kalman-Filter (KF) ist ein optimaler rekursiver Schätzer für lineare Systeme unter Einfluss von Gaußschem Rauschen [8, 9]. Für das Tracking von Keypoints im Bildraum (u, v) mittels eines Constant-Velocity-Modells (Constant Velocity (CV)) wird der Zustandsvektor $\mathbf{x}_k$ zum Zeitpunkt k als:

$$\mathbf{x}_k = \begin{bmatrix} u_k \\ v_k \\ \dot{u}_k \\ \dot{v}_k \end{bmatrix} \tag{3.1}$$

definiert.

Der Filter arbeitet in zwei rekursiven Schritten: Prädiktion (*a priori* Schätzung) und Korrektur (*a posteriori* Schätzung). Die Notation $\mathbf{x}_{k|k-1}$ beschreibt den geschätzten Zustand zum Zeitpunkt k, basierend auf dem Wissen bis zum Zeitpunkt $k-1$.

3.1.1 Prädiktion (Predict)

In diesem Schritt wird der Zustand basierend auf dem Systemmodell in die Zukunft projiziert. Die Matrix F beschreibt die lineare Kinematik. Q ist die Prozessrauschkovarianz und modelliert die Modellunsicherheit.

1. Zustand $\mathbf{x}_{k|k-1}$ prädizieren:

$$\mathbf{x}_{k|k-1} = F\mathbf{x}_{k-1|k-1} \tag{3.2}$$

2. Fehlerkovarianz $P_{k|k-1}$ prädizieren (Unsicherheit der Schätzung):

$$P_{k|k-1} = FP_{k-1|k-1}F^T + Q \tag{3.3}$$

Für das 2D-CV-Modell mit der Zeitschrittweite Δt gilt für die Systemmatrix F:

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3.4}$$

3.1.2 Korrektur (Update)

Sobald eine neue Messung der Pixelkoordinaten $\mathbf{z}_k = [u_{\text{gemessen}}, v_{\text{gemessen}}]^T$ vorliegt, wird die Prädiktion korrigiert. Die Messmatrix H projiziert den Zustandsraum in den Messraum, R ist die Messrauschkovarianz (Unsicherheit des Sensors).

1. Innovation (Residuum) berechnen:

$$\tilde{\mathbf{y}}_k = \mathbf{z}_k - H\mathbf{x}_{k|k-1} \tag{3.5}$$

   - H (Messmatrix): Übersetzt den theoretischen Zustand in den Messraum (blendet hier die Geschwindigkeiten aus, da die Kamera diese nicht direkt misst).

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \tag{3.6}$$

   - $\tilde{\mathbf{y}}_k$: Ist die Differenz zwischen Messung und prädiziertem Zustand.

2. Innovationskovarianz berechnen:

$$S_k = HP_{k|k-1}H^T + R \tag{3.7}$$

3. Kalman-Verstärkung (Kalman Gain) berechnen:

$$K_k = P_{k|k-1}H^T S_k^{-1} \tag{3.8}$$

   Der Kalman-Gain bestimmt, wie stark das Residuum $\tilde{\mathbf{y}}_k$ gewichtet wird.

4. Zustand aktualisieren:

$$\mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + K_k\tilde{\mathbf{y}}_k \tag{3.9}$$

5. Fehlerkovarianz aktualisieren:

$$P_{k|k} = (I - K_kH)P_{k|k-1} \tag{3.10}$$

3.2 Ausreißererkennung mittels Mahalanobis-Distanz

Feature-Matcher sind anfällig für falsche Assoziationen (*False Positives*) durch Clutter oder Reflexionen. Die Integration solcher Ausreißer führt zur Divergenz der Zustandsschätzung. Um dies zu verhindern, wird ein Gating-Verfahren angewendet, das Messungen vor dem Update-Schritt auf statistische Plausibilität prüft [5, 10].

Anstelle der euklidischen Distanz wird die Mahalanobis-Distanz verwendet. Sie gewichtet das Residuum $\tilde{\mathbf{y}}_k$ mit der Innovationskovarianz S_k:

$$D^2 = \tilde{\mathbf{y}}_k^T S_k^{-1} \tilde{\mathbf{y}}_k \tag{3.11}$$

Dies hat den Vorteil, dass die Distanz mit Berücksichtigung der Unsicherheit des Systems und der Messung skaliert wird [5, 11].

Unter der Annahme von normalverteiltem Rauschen folgt D^2 einer χ^2-Verteilung mit m Freiheitsgraden (Degrees of Freedom (DoF)). Um eine Messung zu validieren, wird ein Konfidenzintervall gewählt und der entsprechende Schwellenwert γ aus der χ^2-Tabelle entnommen (siehe Tabelle 3.1) [12].

Freiheitsgrade (DoF)	**γ für 95 % Konfidenz**	**γ für 99 % Konfidenz**
2 (z. B. 1 Pixel)	5.99	9.21
4 (z. B. 2 Pixel)	9.49	13.28
6 (z. B. 3D-Pose)	12.59	16.81
10 (z. B. 5 Keypoints)	18.48	23.21
20 (z. B. 10 Keypoints)	31.41	37.57
30 (z. B. 15 Keypoints)	43.77	50.89
40 (z. B. 20 Keypoints)	55.76	63.69
⋮	⋮	⋮
60 (z. B. 30 Keypoints)	79.082	88.379
⋮	⋮	⋮

Tabelle 3.1: Auszug aus der χ^2-Verteilungstabelle für das Measurement-Gating.

Überschreitet D^2 den Wert γ, wird die Messung als Ausreißer abgelehnt und das Update übersprungen ($x_{k|k} = x_{k|k-1}$).

Da die Dimension des Messvektors (m) bei dynamischem Keypoint-Tracking stetig fluktuiert, wird die quadrierte Mahalanobis-Distanz in der praktischen Implementierung häufig durch die Messdimension m normiert (reduziertes Chi-Quadrat). Dies ermöglicht die Verwendung eines konstanten Schwellenwertes γ, unabhängig von der aktuellen Anzahl der getrackten Features.

3.3 Erweiterte Kalman-Filter für nichtlineare Systeme

Da reale Systeme, wie auch das in dieser Arbeit betrachtete Bewegungsmodell, nichtlineare Eigenschaften aufweisen, reicht der klassische Kalman-Filter nicht aus. Im Folgenden werden daher Erweiterungen für nichtlineare Systeme betrachtet.

3.3.1 Extended Kalman-Filter (EKF)

Sobald das System- oder Messmodell nichtlinear wird – was in der visuellen Robotik aufgrund von Kameramodellen der Regelfall ist – wird der Extended Kalman-Filter (EKF) eingesetzt. Er nutzt eine lokale Linearisierung mittels einer Taylorreihenentwicklung erster Ordnung [5].

Die konstanten Matrizen F und H werden durch nichtlineare Funktionen $f(x)$ und $h(x)$ ersetzt. Der EKF berechnet in jedem Zeitschritt die zugehörigen Jacobi-Matrizen F_k und H_k, evaluiert am aktuellen Schätzpunkt.

$$x_{k|k-1} = f(x_{k-1|k-1}, u_k) \tag{3.12}$$
$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q \tag{3.13}$$

Die Korrektur erfolgt analog unter Verwendung der Mess-Jacobi-Matrix H_k:

$$\tilde{y}_k = z_k - h(x_{k|k-1}) \tag{3.14}$$
$$S_k = H_k P_{k|k-1} H_k^T + R \tag{3.15}$$
$$K_k = P_{k|k-1} H_k^T S_k^{-1} \tag{3.16}$$
$$x_{k|k} = x_{k|k-1} + K_k \tilde{y}_k \tag{3.17}$$
$$P_{k|k} = (I - K_k H_k) P_{k|k-1} \tag{3.18}$$

3.3.2 Unscented Kalman-Filter (UKF)

Der EKF verliert an Genauigkeit, wenn Modelle stark nichtlinear sind. Der Unscented Kalman-Filter (UKF), basierend auf der Arbeit von Julier und Uhlmann, umgeht dieses Problem durch die *Unscented Transform* (UT) [13]. Anstatt analytische Ableitungen zu berechnen, wählt der UKF deterministisch Sigma-Punkte $\mathcal{X}^{(i)}$ aus, die die Wahrscheinlichkeitsverteilung repräsentieren. Diese werden direkt durch die nichtlinearen Funktionen propagiert und zu Mittelwert und Kovarianz rekonstruiert.

Die Unscented Transform (UT) und Sigma-Punkte

Für einen n-dimensionalen Zustandsvektor x mit Mittelwert μ und Kovarianz P wählt der UKF deterministisch $2n + 1$ sogenannte Sigma-Punkte $\mathcal{X}^{(i)}$ aus. Diese Punkte repräsentieren die Verteilung exakt bis zur zweiten Ordnung (Mittelwert und Kovarianz) [13].

Die Berechnung der Sigma-Punkte erfolgt typischerweise über die Matrixwurzel der Kovarianzmatrix (z. B. mittels Cholesky-Zerlegung):

$$\mathcal{X}^{(0)} = \mu \tag{3.19}$$

$$\mathcal{X}^{(i)} = \mu + \left(\sqrt{(n+\lambda)P}\right)_i \quad \text{für } i = 1, \ldots, n \tag{3.20}$$

$$\mathcal{X}^{(i)} = \mu - \left(\sqrt{(n+\lambda)P}\right)_{i-n} \quad \text{für } i = n+1, \ldots, 2n \tag{3.21}$$

Dabei ist λ ein Skalierungsparameter, der die Streuung der Sigma-Punkte um den Mittelwert steuert, und der Index i bezeichnet die i-te Spalte der Matrixwurzel [13].

Jedem Sigma-Punkt $\mathcal{X}^{(i)}$ werden Gewichte für den Mittelwert $W_m^{(i)}$ und für die Kovarianz $W_c^{(i)}$ zugeordnet [13].

Prädiktion und Update ohne Linearisierung

Anstatt Jacobi-Matrizen zu berechnen, werden die Sigma-Punkte direkt durch die nichtlinearen Funktionen f (Systemmodell) oder h (Messmodell) propagiert. Für das Messmodell (wie unser Kameramodell im IBVS) bedeutet das [13]:

$$\mathcal{Z}^{(i)} = h(\mathcal{X}^{(i)}) \tag{3.22}$$

Der prädizierte Messmittelwert $\hat{z}$ und die zugehörige Kovarianz S ergeben sich dann einfach als gewichtete Summe der transformierten Punkte [13]:

$$\hat{z} = \sum_{i=0}^{2n} W_m^{(i)} \mathcal{Z}^{(i)} \tag{3.23}$$

$$S = \sum_{i=0}^{2n} W_c^{(i)} (\mathcal{Z}^{(i)} - \hat{z})(\mathcal{Z}^{(i)} - \hat{z})^T + R \tag{3.24}$$

3.3.3 Error-State Kalman-Filter (ESKF)

Für Systeme, die 3D-Rotationen beinhalten, führt der Standard-EKF aufgrund der Überparametrisierung von Quaternionen häufig zu singulären Kovarianzmatrizen. Der Error-State Kalman-Filter (ESKF) löst dies, indem er den wahren Zustand x_t in einen Nominalzustand x und einen linearisierten Fehlerzustand δx aufteilt [14, 15]:

$$x_t = x \oplus \delta x \tag{3.25}$$

Der Filter schätzt ausschließlich den Fehlerzustand δx. Nach dem Update wird der geschätzte Fehler in den Nominalzustand injiziert und für den nächsten Rekursionsschritt zurückgesetzt ($\delta x \leftarrow 0$).

ESKF Prädiktion (Predict)

Der Nominalzustand x wird analog zum EKF deterministisch und rauschfrei über das nichtlineare Systemmodell f propagiert:

$$x_{k|k-1} = f(x_{k-1|k-1}, u_k) \tag{3.26}$$

Die Prädiktion des Fehlerzustands ist im Erwartungswert Null ($\delta x_{k|k-1} = 0$). Die Fehler-Kovarianz P wächst jedoch basierend auf der Fehler-Kinematik:

$$P_{k|k-1} = F_{\delta x} P_{k-1|k-1} F_{\delta x}^T + F_i Q F_i^T \tag{3.27}$$

Dabei ist $F_{\delta x}$ die Jacobi-Matrix der zeitdiskreten Fehlerkinematik ($F_{\delta x} = \frac{\partial f_{\delta x}}{\partial \delta x}$) und F_i die Rausch-Eingangsmatrix, die das Prozessrauschen Q (z. B. Internal Measurement Unit) korrekt in den Zustandsraum mappt [15].

ESKF Korrektur (Update) und die Kettenregel

Liegt eine Messung z_k vor, wird das Residuum $\tilde{y}_k$ bezüglich des unkorrigierten Nominalzustands mittels der Standard-Messfunktion $h(x)$ gebildet:

$$\tilde{y}_k = z_k - h(x) \tag{3.28}$$

Da der Kalman-Filter den Fehlerzustand schätzt, muss die Mess-Jacobi-Matrix $H_{\delta x}$ nach dem Fehlerzustand abgeleitet werden. Dies erfolgt über die Kettenregel:

$$H_{\delta x} = \frac{\partial h(x_t)}{\partial \delta x} = \underbrace{\frac{\partial h}{\partial x_t}}_{H_x} \cdot \underbrace{\frac{\partial x_t}{\partial \delta x}}_{X_{\delta x}} \tag{3.29}$$

- H_x: Die Standard-Mess-Jacobi-Matrix (wie im EKF, z. B. das Lochkameramodell).
- $X_{\delta x}$: Die Zusammensetzungs-Jacobi-Matrix (Jacobian of Composition). Sie bildet den minimalen Fehlerraum (z. B. 3D-Vektor) auf den vollen Zustandsraum (z. B. 4D-Quaternion) ab.

Mit $H_{\delta x}$ erfolgt das Standard-Update:

$$S_k = H_{\delta x} P_{k|k-1} H_{\delta x}^T + R \tag{3.30}$$

$$K_k = P_{k|k-1} H_{\delta x}^T S_k^{-1} \tag{3.31}$$

Der Filter schätzt nun den aktuellen Fehlerzustand und aktualisiert die Kovarianz:

$$\delta x_{k|k} = K_k \tilde{y}_k \tag{3.32}$$

$$P_{k|k} = (I - K_k H_{\delta x}) P_{k|k-1} \tag{3.33}$$

[15].

Injektion und Reset

Abschließend wird der geschätzte Fehler in den Nominalzustand injiziert, um diesen zu korrigieren. Da der Fehler nun Teil des Nominalzustands ist, wird der Fehlerzustand für den nächsten rekursiven Schritt zurückgesetzt (Reset) [15]:

$$x \leftarrow x \oplus \delta x_{k|k} \tag{3.34}$$

$$\delta x_{k|k} \leftarrow 0 \tag{3.35}$$

Dieser Reset-Mechanismus sorgt dafür, dass der Fehlerzustand stets nahe Null bleibt, wodurch die Linearisierungsfehler des EKF minimiert werden.

3.4 Der 1€-Filter

Für das reine Glätten von 2D-Pixelkoordinaten in IBVS-Anwendungen, bei denen keine komplexe physikalische Modellierung erforderlich ist, stellt der 1€-Filter eine hocheffiziente Alternative dar [16]. Er adressiert direkt den Trade-off zwischen Jitter (Zittern im Stillstand) und Lag (Latenz bei Bewegung).

Der Filter ist ein adaptiver Tiefpassfilter erster Ordnung. Die Grenzfrequenz f_c wird dynamisch in Abhängigkeit der aktuellen Signaländerungsrate $|\hat{\dot{x}}_i|$ moduliert:

$$f_c = f_{min} + \beta|\hat{\dot{x}}_i| \tag{3.36}$$

Wobei f_{min} bei Stillstand dominiert und β die Grenzfrequenz bei schnellen Bewegungen anhebt, um Signalverzögerungen zu minimieren.

4 Verwandte Arbeiten

In diesem Kapitel werden relevante Arbeiten zur robusten visuellen Regelung vorgestellt und im Hinblick auf die in dieser Arbeit behandelte Problemstellung eingeordnet. Der Schwerpunkt liegt dabei auf Ansätzen, die die Stabilität von IBVS unter unsicheren oder wechselnden Bildbedingungen verbessern sollen. Dazu zählen sowohl regelungsseitige Erweiterungen klassischer IBVS-Verfahren als auch filterbasierte Schätzmethoden und moderne lernbasierte Ansätze. Abschließend werden die Grenzen bestehender Arbeiten aufgezeigt, um die Motivation und Abgrenzung des in dieser Arbeit verfolgten Ansatzes zu verdeutlichen.

4.1 Robuste und stabilisierende Ansätze im Image-Based Visual Servoing

Klassisches IBVS verwendet Bildmerkmale direkt im Bildraum, um die Bewegung eines Roboters zu steuern. Die zugrunde liegende Idee besteht darin, den Fehler zwischen aktuellen und gewünschten Bildmerkmalen zu minimieren [1, 6, 17]. Die Stabilität eines solchen Regelkreises hängt jedoch wesentlich davon ab, dass die verwendeten Bildmerkmale zuverlässig detektiert und über die Bewegung hinweg konsistent genutzt werden können. In realen Anwendungen ist dies häufig nicht vollständig gegeben, etwa bei wechselnder Sichtbarkeit, Sichtfeldgrenzen, Okklusionen oder instabilen Bildstrukturen.

Vor diesem Hintergrund wurden verschiedene Erweiterungen klassischer IBVS-Ansätze vorgeschlagen, die die Robustheit der Regelung erhöhen sollen, ohne den bildbasierten Charakter des Verfahrens vollständig aufzugeben.

4.1.1 Wechselnde Sichtbarkeit und Sichtfeldbedingungen

Eine erste Gruppe von Arbeiten behandelt die Problematik, dass sich die Sichtbarkeit einzelner Bildmerkmale während der Regelung ändern kann. García-Aracil et al. untersuchen hierzu ein kontinuierliches Aufgabenfunktionskonzept mit gewichteten Merkmalen, sodass der Regelprozess auch dann fortgeführt werden kann, wenn einzelne Features zeitweise nicht verfügbar sind [18]. Der Ansatz zeigt, dass der Verlust einzelner sichtbarer Merkmale nicht zwangsläufig zum Abbruch der visuellen Regelung führen muss, sofern deren Einfluss geeignet in die Aufgabenformulierung integriert wird.

Einen stärker regelungstheoretischen Ansatz verfolgen Bechlioulis et al., die ein robustes IBVS-Verfahren unter expliziten *Field-of-View*-Randbedingungen formulieren [19]. Ziel ist es, die Bildmerkmale während der Bewegung innerhalb des sichtbaren Kamerabereichs zu halten und gleichzeitig ein gewünschtes Konvergenzverhalten des Bildfehlers sicherzustellen. Solche Arbeiten verdeutlichen, dass die Stabilität visueller Regelung

nicht nur von der Merkmalsextraktion, sondern auch von der Struktur des Reglers und den berücksichtigten Nebenbedingungen abhängt.

Neuere Arbeiten betrachten zusätzlich die aktive Vermeidung von Okklusionen. So kombinieren Zhang et al. IBVS mit probabilistischen *Control Barrier Certificates*, um Bildmerkmale im Sichtfeld zu halten und okklusionsfreie Bewegungen zu erzeugen [20]. Damit wird die klassische bildbasierte Regelung um constraint-basierte Strategien erweitert, die nicht nur den aktuellen Bildfehler, sondern auch die zukünftige Sichtbarkeit der Merkmale berücksichtigen.

4.1.2 Hybride und direkte Visual-Servoing-Formulierungen

Neben Arbeiten zur expliziten Sicherstellung der Sichtbarkeit wurden auch alternative Formulierungen des visuellen Regelproblems vorgeschlagen. Ein bekanntes Beispiel ist das *2-1/2-D Visual Servoing*, das Eigenschaften bildbasierter und posebasierter Verfahren kombiniert, um typische Nachteile rein zweidimensionaler oder rein dreidimensionaler Ansätze abzumildern [21]. Hybride Formulierungen dieser Art verfolgen das Ziel, günstigere Konvergenz- und Stabilitätseigenschaften zu erreichen, ohne vollständig auf Bildinformation zu verzichten.

Eine weitere Richtung bilden Verfahren des *Direct Visual Servoing*. Silveira und Malis formulieren die Regelung dabei nicht primär über explizit detektierte Keypoints, sondern direkt über Bildinformation zwischen aktuellem und Referenzbild [22]. Solche Ansätze zeigen, dass visuelle Regelung auch ohne klassische Merkmalskorrespondenzen möglich ist. Für die vorliegende Arbeit ist dies insbesondere als Gegenposition relevant: Während Direct Visual Servoing die explizite Feature-Ebene weitgehend vermeidet, verfolgt die vorliegende Arbeit das Ziel, featurebasierte Regelung auch unter schwierigen Bildbedingungen gezielt zu stabilisieren.

Die genannten Arbeiten zeigen, dass bereits verschiedene Strategien zur Stabilisierung von IBVS existieren. Ihr Schwerpunkt liegt jedoch vor allem auf Sichtfeldgrenzen, wechselnder Sichtbarkeit, okklusionsfreier Trajektorienführung oder alternativen Regelungsformulierungen. Die spezifische Problematik temporär instabiler Keypoints infolge von Reflexionen, Überbelichtungen oder lokal stark veränderten Bildstrukturen wird dabei nur begrenzt adressiert.

4.2 Filter- und schätzbasierte Ansätze in Visual Servoing

Neben regelungsseitigen Erweiterungen existieren Arbeiten, die die Robustheit visueller Regelung durch die Schätzung verborgener oder verrauschter Zustandsgrößen erhöhen. In diesen Ansätzen dienen Bildmerkmale primär als Messgrößen, aus denen regelungsrelevante Größen wie Pose, Tiefe oder Geschwindigkeit robuster rekonstruiert werden.

Janabi-Sharifi und Marey schlagen hierzu ein Verfahren zur Poseschätzung in Visual-Servoing-Anwendungen vor, das auf einem iterativen adaptiven erweiterten Kalman-Filter basiert [23]. Ziel der Arbeit ist es, die Echtzeitschätzung der relativen Pose zwischen Kamera und Objekt gegenüber ungenauen Rauschannahmen, schlechter Initialisierung und starker Nichtlinearität robuster zu machen. Die Bildmerkmale dienen

dabei als Messgrößen, während der Filterzustand die Pose und ihre zeitliche Änderung umfasst. Der Schwerpunkt liegt somit auf einer robusteren zustandsbasierten Poseschätzung, nicht auf der persistenten Verwaltung oder modellgestützten Wiederverwendung einzelner Bildmerkmale.

Einen verwandten, aber dynamisch stärker ausgerichteten Ansatz präsentieren Chang et al. [24]. Die Autoren erweitern klassische bildbasierte visuelle Regelung um einen Kalman-filter-basierten Schätzer für Tiefe und Geschwindigkeit. Dabei wird die Tiefe der Bildmerkmale sowie die Geschwindigkeit des Endeffektors im Kamerakoordinatensystem aus Bildinformationen geschätzt und in eine modifizierte dynamische Visual-Servoing-Architektur integriert. Zusätzlich wird eine *computed-torque*-basierte Regelungsstruktur verwendet, um Systemdynamiken explizit zu berücksichtigen. Der Ansatz verbessert damit die Schätzung regelungsrelevanter Größen innerhalb des Servoing-Regelkreises, adressiert jedoch nicht die modellgestützte Vorhersage, Wiedererkennung oder Ersetzung einzelner Keypoints bei temporärem Merkmalsausfall.

Beide Arbeiten zeigen, dass Filtermethoden in Visual-Servoing-Systemen einen wichtigen Beitrag zur Robustheit leisten können. Gleichzeitig wird deutlich, dass der Fokus dieser Ansätze überwiegend auf der Schätzung globaler oder regelungsrelevanter Zustandsgrößen liegt. Die Frage, wie einzelne lokale Bildmerkmale bei reflektionsbedingten Ausfällen, Überbelichtungen oder kurzfristig instabiler Detektion robust weiterverwendet werden können, bleibt dabei weitgehend offen.

4.3 Moderne lernbasierte Ansätze

Auch moderne lernbasierte Merkmalsmethoden wie SuperPoint, SuperGlue, R2D2 oder XFeat verbessern die Robustheit von Detektion und Matching gegenüber klassischen handentwickelten Verfahren [4, 25, 26, 27]. Sie lösen jedoch nicht grundsätzlich das Problem, dass lokale Bildmerkmale bei starken Reflexionen, Überbelichtungen oder temporären Sichtausfällen instabil oder unzuverlässig werden, weshalb der Bedarf an einer modellgestützten Stabilisierung auf Keypoint-Ebene bestehen bleibt.

Eine zweite Richtung versucht, die explizite Modellierung oder Korrespondenz einzelner Merkmale ganz oder teilweise durch lernbasierte Verfahren zu ersetzen. Bateux et al. präsentieren mit *Visual Servoing from Deep Neural Networks* einen Ansatz, bei dem ein Convolutional Neural Network die relative Pose zwischen aktuellem und gewünschtem Bild direkt aus den Bilddaten schätzt [28]. Das Netzwerk wird auf synthetisch erweiterten Trainingsdaten feinjustiert, die aus einem realen Szenenbild unter Variation von Beleuchtung und Okklusion erzeugt werden. Die geschätzte Relativpose wird anschließend in eine Visual-Servoing-Regelung eingespeist. Der Ansatz zeigt, dass robuste visuelle Regelung auch ohne explizites Matching einzelner Keypoints möglich ist, verlagert die Robustheit jedoch von der Merkmalsebene auf eine lernbasierte Pose-Schätzung aus dem Gesamtbild.

Moderne lernbasierte Ansätze erweitern damit die Möglichkeiten visueller Regelung erheblich. Gleichzeitig unterscheiden sie sich konzeptionell deutlich von featurebasierten Verfahren, die explizit mit lokalen Keypoints arbeiten. Für die vorliegende Arbeit sind sie daher vor allem als alternative Strategien zur Robustheitserhöhung relevant, nicht jedoch als direkte Vorläufer eines modellgestützten Keypoint-Managements.

4.4 Forschungslücke und Abgrenzung der eigenen Arbeit

Die betrachteten Arbeiten zeigen, dass die Robustheit visueller Regelung bereits aus verschiedenen Perspektiven adressiert wurde. Erweiterte IBVS-Ansätze berücksichtigen wechselnde Sichtbarkeit, Sichtfeldgrenzen oder okklusionsfreie Trajektorienplanung [18, 19, 20]. Filter- und Schätzansätze verbessern die Robustheit durch die Rekonstruktion regelungsrelevanter Größen wie Pose, Tiefe oder Geschwindigkeit [23, 24]. Lernbasierte Verfahren erhöhen entweder die Qualität der Merkmalsextraktion oder umgehen die explizite Feature-Ebene teilweise ganz [4, 25, 26, 27, 28].

Nur begrenzt behandelt wird jedoch die Fragestellung, wie lokale Bildmerkmale in einem featurebasierten IBVS-System auch dann robust weiterverwendet werden können, wenn sie durch Reflexionen, Überbelichtungen, lokale Strukturänderungen oder temporäre Sichtausfälle kurzfristig instabil werden. Insbesondere fehlt es an Ansätzen, die reale Bildmerkmale gezielt mit modellbasierten Vorhersagen koppeln, um temporär unzuverlässige Keypoints auf Merkmalsebene zu stabilisieren.

Die in dieser Arbeit vorgestellte Methode setzt genau an dieser Stelle an. Im Gegensatz zu Arbeiten, die vor allem Sichtbarkeit sichern, globale Zustandsgrößen schätzen oder auf lernbasierte Gesamtbildrepräsentationen setzen, liegt der Fokus hier auf der robusten Weiterverwendung lokaler Keypoints unter schwierigen Bildbedingungen. Dazu werden reale Bildmerkmale mit modellierten Keypoints kombiniert, die auf der bekannten Roboterbewegung sowie einem mathematischen Zustandsmodell basieren. Ergänzend kommt eine Filter- und Entscheidungslogik zum Einsatz, welche die Qualität der verfügbaren Merkmale bewertet und dynamisch entscheidet, welche Informationen für die Regelung verwendet werden. Ziel ist es, die Stabilität featurebasierter visueller Regelung zu erhöhen, ohne den Keypoint-basierten Charakter des Ansatzes aufzugeben.

5 Systemarchitektur und Methode

Dieses Kapitel beschreibt die Architektur des entwickelten Systems zur bildbasierten Roboterregelung. Ziel des Systems ist es, einen modularen IBVS-Stack zu implementieren, der es ermöglicht, einen UR5e-Roboter mithilfe visueller Merkmale aus einer Kamerabeobachtung zu steuern. Das System basiert auf einer modularen Architektur, bei der einzelne Komponenten wie Feature-Erkennung, Referenzmodellbildung, Matching, Filterung und Regelung unabhängig voneinander entwickelt und ausgetauscht werden können. Diese Struktur ermöglicht eine flexible Erweiterung des Systems sowie die einfache Integration alternativer Verfahren. Die gesamte Pipeline ist in mehrere ROS2-Pakete aufgeteilt und wird in einem Docker-Container betrieben, wodurch eine reproduzierbare und klar strukturierte Systemarchitektur entsteht [29, 30].

5.1 Systemübersicht

Das entwickelte System implementiert eine vollständige Pipeline für IBVS. Die Pipeline beginnt mit der Aufnahme eines Kamerabildes, aus dem visuelle Merkmale extrahiert werden. Anschließend werden diese Merkmale mit einem zuvor erzeugten Referenzdatensatz verglichen, um korrespondierende Bildpunkte zu bestimmen. Auf Grundlage dieser Feature-Korrespondenzen wird ein Bildfehler berechnet, der schließlich in eine Roboterbewegung umgewandelt wird.

Ein zentrales Ziel der Architektur besteht darin, ein stabiles Referenzmodell aus einer längeren Beobachtungung aufzubauen, anstatt sich auf eine einzelne Bildaufnahme zu verlassen. Dadurch kann die Robustheit der Feature-Erkennung erhöht werden. Die gesamte IBVS-Pipeline besteht aus den folgenden Hauptschritten und ist in Abbildung 5.1 schematisch dargestellt:

1. Aufnahme eines RGB-Kamerabildes durch die RealSense-Kamera
2. Erkennung von Keypoints und Deskriptoren mit einem Feature-Detektor
3. Aufbau eines stabilen Referenzdatensatzes aus mehreren Frames
4. Matching der aktuellen Bildfeatures mit den Referenzfeatures
5. Filterung und Validierung der Feature-Korrespondenzen
6. Berechnung des Bildfehlers
7. Berechnung einer kartesischen Geschwindigkeitssteuerung für den Roboter

Die resultierende Steuergröße wird anschließend als kartesischer *Twist*-Befehl an den Robotercontroller gesendet. Die Systemarchitektur ist bewusst modular gestaltet. Dadurch können einzelne Komponenten, wie der Feature-Detektor oder das Matching-Verfahren, unabhängig voneinander ausgetauscht werden, ohne dass die übrigen Komponenten angepasst werden müssen.

Auf die Modellbasierte Keypointvorhersage, die in den Kapiteln 6 sowie 8 beschrieben wird, folgt eine Rückkopplungsschleife zwischen dem Filter und dem Matching. Der Filter veröffentlicht kontinuierlich prädizierte Positionen der Keypoints im Bild. Geht ein Feature verloren, nutzt das Matching diese Vorhersagen, um das Feature in einer zielgerichteten lokalen Suche wiederzufinden. Dieser Top-Down-Ansatz erhöht die Robustheit des Systems erheblich, insbesondere unter schwierigen Bildbedingungen.

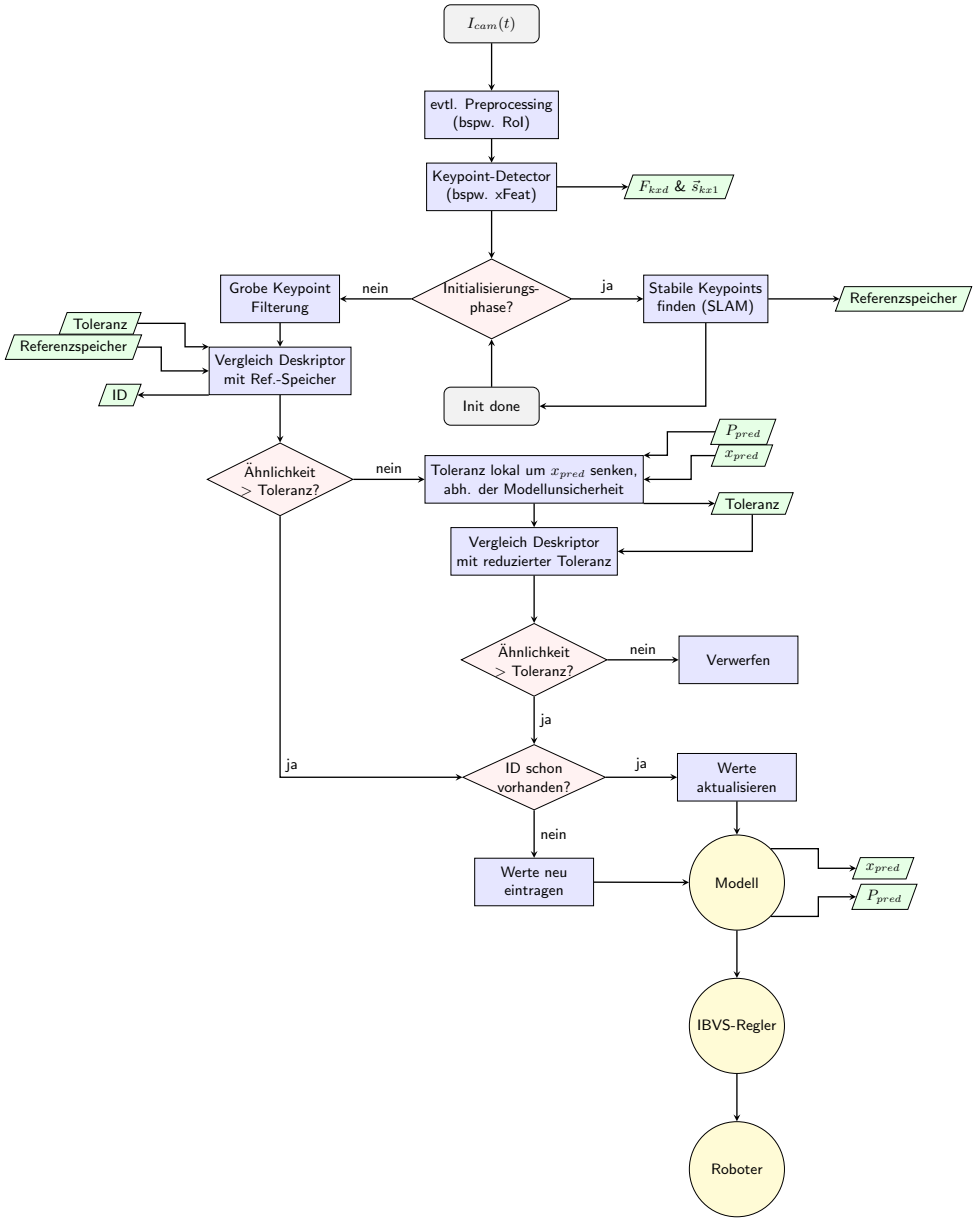


Abbildung 5.1: Schematische Darstellung der IBVS-Pipeline. Die einzelnen Schritte sind modular aufgebaut, um eine flexible Erweiterung und Anpassung zu ermöglichen.

5.2 Hardwareaufbau

Das System besteht aus zwei zentralen Hardwarekomponenten: einer RGB-D-Kamera zur visuellen Wahrnehmung der Umgebung sowie einem industriellen Robotersystem zur Ausführung der Bewegungen.

5.2.1 Kamerasystem: Intel RealSense D435

Als Kamerasystem wird eine Intel RealSense D435 verwendet. Diese Kamera stellt sowohl RGB- als auch Tiefeninformationen bereit und ist für Robotik- und Wahrnehmungsanwendungen weit verbreitet. Die D435 basiert auf einem stereoskopischen Tiefenprinzip und ist für kompakte, echtzeitfähige Anwendungen ausgelegt [31, 32].

Im Rahmen dieser Arbeit wird primär das RGB-Bild für die Feature-Erkennung verwendet. Die Tiefeninformation wird genutzt um eine Region of Interest (ROI)-Filterungen für die Featureauswahl durchzuführen. Jedes Feature bekommt zudem seine gemessene Tiefeninformation für die Visual-Servoing-Berechnung, wenn diese noch nie verfügbar war, wird ein Fallbackwert verwendet. Die Kamera wird in der vorliegenden Implementierung mit folgenden Einstellungen betrieben:

- Auflösung: 640 $\times$ 480 Pixel
- Bildrate: $60\,\text{Bilder pro Sekunde}$

In der Praxis kann es aufgrund von Systemlast oder Kommunikationsverzögerungen zu gelegentlichen Einbrüchen der Bildrate kommen.

5.2.2 Robotersystem: UR5e

Als Robotersystem wird ein kollaborativer Industrieroboter des Typs UR5e verwendet. Der UR5e besitzt sechs Freiheitsgrade und einen Arbeitsradius von $850\,\text{mm}$; er wird in dieser Arbeit über den offiziellen ROS2-Treiber für Universal Robots angesteuert [33, 34]. Die Steuerung erfolgt über einen kartesischen Geschwindigkeitscontroller. Dabei werden kartesische Geschwindigkeitsbefehle direkt an den Roboter gesendet, die anschließend vom Controller in Gelenkbewegungen umgesetzt werden.

Der verwendete Controller ist `cartesian_twist_passthrough_controller` und akzeptiert kartesische Geschwindigkeiten in Form eines *Twist*-Vektors. Das System kann alle sechs Freiheitsgrade regeln, diese können jedoch je nach Anwendungsfall eingeschränkt werden.

5.3 Softwarearchitektur

Die Softwarearchitektur des Systems basiert auf ROS2. ROS2 unterstützt die Aufteilung komplexer robotischer Systeme in modulare Nodes, die über Topics, Services und weitere Kommunikationsmechanismen miteinander interagieren [29, 30]. Die Funktionalität ist in mehrere unabhängige ROS2-Pakete aufgeteilt, die jeweils spezifische Aufgaben innerhalb der IBVS-Pipeline übernehmen. Die Verwendung einer modularen Architektur ermöglicht eine klare Trennung zwischen Wahrnehmung, Datenverarbeitung und Robotersteuerung.

5.3.1 ROS2-Systemstruktur

Die Implementierung ist in mehrere ROS2-Pakete unterteilt:

- `ibvs_perception`: Erkennung von Keypoints und Deskriptoren aus Kamerabildern
- `ibvs_reference`: Aufbau und Verwaltung eines stabilen Referenzdatensatzes
- `ibvs_matching`: Zuordnung von aktuellen Bildfeatures zu Referenzfeatures
- `ibvs_filter_cpp`: Bewertung der Qualität der Feature-Korrespondenzen und Stabilisierung der Featurepositionen
- `ibvs_control`: Berechnung der Roboterbewegung auf Basis des IBVS-Reglers
- `ibvs_msgs`: Definition der verwendeten ROS-Nachrichten

Diese Struktur erlaubt eine klare Trennung der einzelnen Verarbeitungsschritte und erleichtert die Erweiterung des Systems.

5.3.2 Docker-basierte Systemarchitektur

Zur Vereinfachung der Systemverwaltung wird das gesamte System in mehreren Docker-Containern betrieben. Jeder Container enthält einen klar definierten Teil des Systems. Die Container sind:

- `camera_driver`: RealSense-Treiber und Kamerapipeline
- `ros2_ur_driver`: ROS2-Treiber für den UR5e-Roboter
- `ros_ws`: Workspace mit den IBVS-Paketen und der eigentlichen Visual-Servoing-Implementierung

Alle Container verwenden das Netzwerkmodell `network_mode: host`. Dadurch können ROS2-Nodes in verschiedenen Containern direkt miteinander kommunizieren. Zusätzlich teilen sich alle Container dieselbe ROS-Domain-ID, wodurch die automatische Erkennung von ROS2-Nodes und Topics ermöglicht wird.

5.3.3 Datenfluss im System

Der Datenfluss innerhalb des Systems folgt einer klar definierten Pipeline. Zunächst wird ein RGB-Bild von der Kamera aufgenommen und an die Feature-Erkennungsnode weitergeleitet. Diese Node extrahiert Keypoints und Deskriptoren aus dem Bild und veröffentlicht sie als ROS-Nachricht. Die Referenzmanager-Node nutzt diese Informationen, um über mehrere Frames hinweg ein stabiles Referenz-Feature-Set aufzubauen. Anschließend werden aktuelle Bildfeatures mit den Referenzfeatures gematcht. Die daraus entstehenden Feature-Korrespondenzen werden an den IBVS-Controller übergeben. Der Controller berechnet aus dem Bildfehler eine kartesische Geschwindigkeit, die schließlich als Steuerbefehl an den UR-Roboter gesendet wird.

Der gesamte Datenfluss kann vereinfacht wie folgt dargestellt werden:

Camera Image
↓
Keypoint Detection
↓
Reference Feature Set
↓
Descriptor Matching
↓
Feature Filtering
↓
IBVS Controller
↓
Robot Motion

Diese Pipeline bildet die Grundlage für die visuelle Roboterregelung im entwickelten System.

Zur Erhöhung der Robustheit existiert zusätzlich eine Rückkopplung (Feedback-Loop) zwischen dem IBVS-Controller (bzw. dem integrierten Filter) und dem Feature-Matching. Der Zustandsfilter veröffentlicht mit einer festen Frequenz kontinuierlich die prädizierten (vorhergesagten) Positionen der Keypoints im Bild. Geht ein Feature in der regulären Detektions-Pipeline temporär verloren, nutzt das Matching-Modul diese Vorhersagen, um das Feature in einer zielgerichteten lokalen Suche (*Local Rescue*) wiederzufinden. Dieser Top-Down-Ansatz wird in den folgenden Kapiteln im Detail beschrieben.

5.4 Feature Detection

Die zuverlässige Erkennung stabiler Bildmerkmale stellt eine zentrale Voraussetzung für die erfolgreiche Anwendung von *Image-Based Visual Servoing* dar. Da die Regelung direkt auf der Veränderung von Bildfeatures basiert, hängt die Stabilität des Systems maßgeblich von der Qualität der Feature-Erkennung und des Feature-Matchings ab. In dieser Arbeit wird zur Extraktion visueller Merkmale ein lernbasierter Feature-Detektor verwendet. Dieser Ansatz ermöglicht eine robustere Feature-Erkennung unter schwierigen Bildbedingungen als viele klassische Verfahren.

5.4.1 Keypoint Detection mit xFeat

Zur Erkennung von Keypoints und zugehörigen Deskriptoren wird der Feature-Detektor xFeat verwendet. xFeat ist ein moderner, lernbasierter Feature-Detektor, der speziell für eine effiziente und robuste lokale Feature-Erkennung in ressourcenbeschränkten Echtzeitsystemen entwickelt wurde [4].

Lernbasierte Feature-Methoden haben sich in den letzten Jahren als leistungsfähige Alternative zu klassischen Verfahren etabliert. Während traditionelle Algorithmen wie SIFT oder ORB auf handentwickelten Bildoperatoren basieren [35, 36], lernen moderne Verfahren die Feature-Repräsentationen direkt aus Trainingsdaten. Dadurch können sie häufig robuster gegenüber Beleuchtungsänderungen, Bildrauschen und geometrischen Transformationen sein [4, 25, 26].

Der verwendete xFeat-Detektor erzeugt für jedes Bild eine Menge von Keypoints sowie zugehörige Deskriptorvektoren. Jeder Keypoint beschreibt eine charakteristische Position im Bild, während der zugehörige Deskriptor die lokale Bildstruktur um diesen Punkt kodiert. Die Detektion erfolgt in mehreren Schritten:

1. Extraktion lokaler Bildfeatures durch ein neuronales Netzwerk
2. Auswahl der relevantesten Keypoints
3. Berechnung zugehöriger Deskriptorvektoren

Die Anzahl der extrahierten Keypoints wird durch den Parameter `top_k` begrenzt, wodurch die Anzahl der zu verarbeitenden Features kontrolliert werden kann. In der aktuellen Konfiguration werden bis zu 1024 Keypoints pro Bild verwendet.

5.4.2 Feature-Deskriptoren

Neben der Position eines Keypoints wird für jedes Feature ein Deskriptorvektor berechnet. Dieser Vektor beschreibt die lokale Bildstruktur um den Keypoint und ermöglicht die spätere Wiedererkennung des Features in anderen Bildern. Die Deskriptoren werden als hochdimensionale Vektoren dargestellt und in der ROS-Nachricht `ibvs_msgs/Keypoints` übertragen.

Die Struktur dieser Nachricht enthält insbesondere:

- die Bildkoordinaten der Keypoints,
- die Dimension der Deskriptorvektoren,
- die Deskriptorwerte selbst,
- optionale Score-Werte zur Bewertung der Featurequalität.

Die Deskriptoren werden im weiteren Verlauf der Pipeline verwendet, um aktuelle Bildfeatures mit einem zuvor gespeicherten Referenzdatensatz zu vergleichen. Der Vergleich der Deskriptoren erfolgt mithilfe einer Ähnlichkeitsmetrik. In dieser Arbeit wird hierfür die Cosinus-Ähnlichkeit verwendet. Zwei Features werden als korrespondierend betrachtet, wenn ihre Deskriptoren eine Ähnlichkeit oberhalb eines definierten Schwellenwerts aufweisen.

5.4.3 Depth-basierte Region-of-Interest-Filterung

Zusätzlich zur RGB-Bildinformation stellt die RealSense-Kamera Tiefendaten bereit. Diese Tiefeninformationen können genutzt werden, um bestimmte Bildbereiche gezielt zu filtern. In der vorliegenden Implementierung kann optional eine depth-basierte ROI verwendet werden. Dabei werden nur Features berücksichtigt, deren Tiefenwert innerhalb eines definierten Bereichs liegt.

Der Vorteil dieses Ansatzes besteht darin, dass Bildbereiche außerhalb des relevanten Arbeitsraums des Roboters ausgeblendet werden können. Dadurch lässt sich die Anzahl irrelevanter Features reduzieren und die Stabilität des Feature-Matchings erhöhen. Die Tiefendaten werden über das Topic `/camera/camera/aligned_depth_to_color/image_raw` bereitgestellt. Falls die Depth-ROI aktiviert ist, wird für jeden Keypoint der zugehörige Tiefenwert überprüft. Features mit ungültigen oder außerhalb des definierten Bereichs liegenden Tiefenwerten werden verworfen.

5.5 Keypoint-Datenbank

Für die robuste Zuordnung von Bildmerkmalen im IBVS-System wird eine zweistufige Keypoint-Verwaltung eingesetzt. Diese besteht aus einer während der Initialisierung aufgebauten Referenzdatenbank sowie einem laufzeitseitigen Abgleichskontext. Während die Referenzdatenbank den festen Sollzustand der relevanten Bildmerkmale beschreibt, unterstützt der Laufzeitkontext die Wiederzuordnung dieser Merkmale im Regelbetrieb.

5.5.1 Aufbau der Referenzdatenbank

Die Referenzdatenbank wird in einer separaten Initialisierungsphase erzeugt. Hierzu werden die vom Keypoint-Detektor gelieferten Merkmale nicht unmittelbar übernommen, sondern zunächst in einer temporären Kandidatenstruktur gesammelt. Ziel ist es, nur solche Keypoints in die spätere Referenz zu überführen, die über mehrere Frames hinweg stabil beobachtet werden.

Die Zuordnung neuer Beobachtungen zu bestehenden Kandidaten erfolgt anhand ihrer Bildposition und, sofern verfügbar, zusätzlich anhand der Ähnlichkeit ihrer Deskriptoren. Wird ein Keypoint einem vorhandenen Kandidaten zugeordnet, so werden dessen Position, Beobachtungszähler und Deskriptor aktualisiert. Letzterer wird dabei über eine exponentielle Glättung fortgeschrieben, um Einzelrauschen zu reduzieren.

Nach Abschluss der Initialisierungsphase werden die Kandidaten anhand ihrer Beobachtungshäufigkeit bewertet und die stabilsten Einträge ausgewählt. Diese bilden die Referenzdatenbank. Sie enthält die Bildkoordinaten der Referenz-Keypoints, ihre Deskriptoren sowie statistische Informationen zur Stabilität der Beobachtung. Die Referenzdatenbank wird anschließend veröffentlicht und im weiteren Verlauf des Systems als feste Sollstruktur verwendet.

5.5.2 Verwendung im Matching

Im laufenden Betrieb dient die Referenzdatenbank als Grundlage für den Abgleich zwischen aktuellem Kamerabild und Referenzzustand. Die gespeicherten Referenzdeskriptoren werden in der Matching-Komponente eingelesen, normiert und mit den normierten Deskriptoren der aktuell detektierten Keypoints verglichen. Aus diesem Vergleich ergibt sich für jeden aktuellen Keypoint die ähnlichste Referenzbeschreibung.

Die Ähnlichkeit zwischen aktuellem und referenzseitigem Deskriptor kann durch

$$S_{ij} = \mathbf{d}_i^{\mathrm{cur}} \cdot \mathbf{d}_j^{\mathrm{ref}} \tag{5.1}$$

beschrieben werden. Dabei bezeichnen $\mathbf{d}_i^{\mathrm{cur}}$ den normierten Deskriptor des aktuellen Keypoints i und $\mathbf{d}_j^{\mathrm{ref}}$ den normierten Deskriptor des Referenz-Keypoints j. Aufgrund der Normierung entspricht dieser Ausdruck einer Cosinus-Ähnlichkeit.

Eine Korrespondenz wird nur dann übernommen, wenn die Ähnlichkeit einen vorgegebenen Schwellwert überschreitet. Zusätzlich kann eine wechselseitige Konsistenzprüfung verwendet werden, um mehrdeutige Zuordnungen zu reduzieren. Auf diese Weise liefert die Referenzdatenbank eine stabile Grundlage für die Bestimmung robuster Punktkorrespondenzen zwischen aktuellem Bild und Zielzustand.

5.5.3 Laufzeitseitiger Abgleichskontext

Neben der festen Referenzdatenbank wird im laufenden Betrieb ein dynamischer Abgleichskontext genutzt. Dieser stellt keine zweite Referenzdatenbank im engeren Sinn dar, sondern eine Hilfsstruktur, die aus bereits gefilterten Zuordnungen früherer Zeitschritte gewonnen wird. Gespeichert werden dabei insbesondere Referenz-IDs, geschätzte Bildpositionen und Unsicherheitsmaße.

Dieser Kontext wird dann relevant, wenn das globale Deskriptormatching allein nicht ausreicht, beispielsweise bei Reflexionen, kurzzeitigen Verdeckungen oder instabiler Merkmalsdetektion. In solchen Situationen kann für bekannte Referenzpunkte eine lokale Nachsuche in der Umgebung ihrer erwarteten Bildposition durchgeführt werden. Dabei werden sowohl die räumliche Nähe zur vorhergesagten Position als auch die Deskriptorähnlichkeit zur zugehörigen Referenz berücksichtigt.

Der laufzeitseitige Abgleichskontext ergänzt somit die feste Referenzdatenbank. Während die Referenzdatenbank die langfristig gültige Sollstruktur bereitstellt, verbessert der Laufzeitkontext die Robustheit der Wiederzuordnung im Regelbetrieb.

5.5.4 Zusammenfassung

Die Keypoint-Verwaltung des Systems setzt sich somit aus zwei funktional unterschiedlichen Ebenen zusammen. Die Referenzdatenbank wird einmalig aus stabilen Beobachtungen der Initialisierungsphase aufgebaut und dient anschließend als feste Vergleichsbasis. Der laufzeitseitige Abgleichskontext wird fortlaufend aus aktuellen und bereits gefilterten Zuordnungen abgeleitet und unterstützt das Wiederauffinden von Merkmalen unter erschwerten Sichtbedingungen. Durch diese Aufteilung wird eine robuste und konsistente Merkmalzuordnung für die visuelle Regelung ermöglicht.

6 Modellbasierte Keypoint-Vorhersage

Dieses Kapitel beschreibt die modellbasierte Vorhersage von Keypoints als zentrale Ergänzung zur rein bildbasierten Merkmalsverarbeitung. Ziel ist es, die erwarteten Bildpositionen relevanter Merkmale aus der bekannten Roboterbewegung und einem mathematischen Zustandsmodell abzuleiten. Dadurch kann der Regelkreis auch dann mit plausiblen Merkmalslagen arbeiten, wenn reale Keypoints aufgrund von Reflexionen, Überbelichtung oder kurzzeitiger Verdeckung instabil oder vorübergehend nicht sichtbar sind.

Im Gegensatz zu einer klassischen 3D-Simulation basiert der in dieser Arbeit verwendete Ansatz nicht auf einem vollständigen geometrischen Objektmodell, sondern auf einer kompakten Zustandsbeschreibung im Bildraum. Die Vorhersage erfolgt durch zeitdiskrete Fortschreibung der Merkmalspositionen unter Verwendung der Kamerabewegung und der IBVS-Zusammenhänge. Die modellbasierten Merkmale bilden damit eine zweite Informationsquelle neben den direkt gemessenen Bildpunkten.

6.1 Motivation der modellbasierten Vorhersage

In industriellen Bildverarbeitungsanwendungen können reale Merkmale durch Reflexionen, Blendung oder Okklusionen vorübergehend unzuverlässig werden. Insbesondere bei metallischen oder stark reflektierenden Oberflächen entstehen Situationen, in denen Keypoints an erwarteten Bildpositionen nicht oder nur instabil detektiert werden. Für einen IBVS-Regelkreis ist dies problematisch, da die Stellgröße direkt von der Verfügbarkeit und Qualität der Bildmerkmale abhängt.

Aus diesem Grund wird in der vorliegenden Arbeit eine modellbasierte Vorhersage eingesetzt, die erwartete Keypoint-Positionen aus der bekannten Roboterbewegung ableitet. Die Grundidee besteht darin, dass das System neben den realen Bildpunkten auch vorhergesagte Merkmalslagen bereitstellt. Diese können entweder direkt zur Stabilisierung der Filterung verwendet oder später in die Entscheidungs- und Switching-Logik eingebunden werden. Die modellbasierte Vorhersage dient somit nicht als Ersatz der Wahrnehmung, sondern als robuste Ergänzung bei unsicheren Beobachtungen.

6.2 Nutzung der Roboterbewegung und Kamerakinematik

Grundlage der Vorhersage ist die Annahme, dass die aktuelle Bewegung des Roboters bekannt ist. Im implementierten System wird hierzu ein kartesischer Geschwindigkeitsvektor aus dem Robotercontroller eingelesen. Dieser liegt zunächst im Basisframe des Roboters vor und wird anschließend unter Verwendung der bekannten Transform (TF)-Beziehungen in das Kameraframe transformiert. Dabei werden sowohl die Rotation zwischen Basis-, Tool Center Point (TCP)- und Kameraframe als auch der Hebelarm zwischen TCP und Kamera berücksichtigt.

Der resultierende kamerabezogene Geschwindigkeitsvektor wird als

$$\mathbf{v}_{\text{cam}} = \begin{bmatrix} v_x & v_y & v_z & \omega_x & \omega_y & \omega_z \end{bmatrix}^\top \tag{6.1}$$

beschrieben und bildet die Eingangsgröße des Vorhersagemodells. Falls die Geschwindigkeitsinformation nicht verfügbar, veraltet oder die notwendige TF-Transformation nicht auflösbar ist, wird in der Implementierung auf einen Nullvektor zurückgegriffen. Dies verhindert unkontrollierte Vorhersagesprünge und stellt sicher, dass die Modellvorhersage in solchen Fällen konservativ bleibt.

6.3 Mathematisches Bewegungsmodell der Bildmerkmale

Die modellbasierte Vorhersage wird direkt im Bildraum formuliert. Der Zustand des Vorhersagemodells besteht aus den aktuellen Pixelkoordinaten einer aktiven Teilmenge von Keypoints. Für n aktive Merkmale kann der Zustandsvektor als

$$\mathbf{x} = \begin{bmatrix} u_1 & v_1 & u_2 & v_2 & \dots & u_n & v_n \end{bmatrix}^\top \tag{6.2}$$

geschrieben werden.

Für die Berechnung der Bewegung im Bildraum werden die Pixelkoordinaten zunächst mit den Kameraintrinsiken in normierte Bildkoordinaten überführt. Dies erfolgt in der Implementierung mithilfe der inversen Kameramatrix $\mathbf{K}^{-1}$. Für ein Merkmal mit Pixelkoordinaten (u_i, v_i) ergibt sich

$$\begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = \mathbf{K}^{-1} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}. \tag{6.3}$$

Auf dieser Grundlage wird für jedes Merkmal eine punktweise Interaction Matrix gebildet. Für ein einzelnes Merkmal mit normierten Koordinaten (x_i, y_i) und Tiefe Z_i gilt

$$\mathbf{L}_i = \begin{bmatrix} -\frac{1}{Z_i} & 0 & \frac{x_i}{Z_i} & x_i y_i & -(1+x_i^2) & y_i \\ 0 & -\frac{1}{Z_i} & \frac{y_i}{Z_i} & 1+y_i^2 & -x_i y_i & -x_i \end{bmatrix}. \tag{6.4}$$

Damit ergibt sich die normierte Bildgeschwindigkeit des i-ten Merkmals zu

$$\dot{\mathbf{s}}_i = \mathbf{L}_i \, \mathbf{v}_{\text{cam}}. \tag{6.5}$$

Anschließend wird diese wieder auf Pixelkoordinaten skaliert, sodass für alle aktiven Merkmale ein Gesamtvektor $\dot{\mathbf{x}}$ der vorhergesagten Pixelgeschwindigkeiten entsteht. In der implementierten C++-Filterarchitektur erfolgt diese Berechnung für alle unterstützten Filtervarianten über dieselbe Grundfunktion zur Bestimmung der Pixelgeschwindigkeiten.

6.4 Zeitdiskrete Vorhersage der Keypoint-Positionen

Die Vorhersage der Keypoint-Positionen erfolgt zeitdiskret. Ausgehend vom aktuellen Zustandsvektor $\mathbf{x}_k$ und dem zugehörigen Geschwindigkeitsvektor $\dot{\mathbf{x}}_k$ wird der nächste Zustand mit

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \dot{\mathbf{x}}_k \, \Delta t \tag{6.6}$$

fortgeschrieben. Die Zeitdifferenz Δt wird aus der tatsächlich vergangenen Zeit zwischen zwei Vorhersageschritten bestimmt. Falls diese Zeitdifferenz ungültig oder ungewöhnlich groß ist, wird in der Implementierung auf die inverse Vorhersagerate als Ersatzwert zurückgegriffen. Dadurch bleibt der Prädiktionsschritt auch bei unregelmäßigem Timing numerisch stabil.

Je nach verwendetem Filtertyp wird dieser Grundschritt unterschiedlich in die jeweilige Schätzgleichung eingebettet. Beim EKF wird beispielsweise ein linearisiertes Zustandsübergangsmodell verwendet, während der UKF die Prädiktion über Sigma-Punkte realisiert. Unabhängig von der konkreten Filtervariante bleibt jedoch die zugrunde liegende Bildpunktdynamik identisch: Die Veränderung der Merkmalspositionen wird aus der Interaction Matrix und der kamerabezogenen Bewegung berechnet.

6.5 Tiefenmodell und Tiefenberechnung

Da die Interaction Matrix explizit von der Tiefe der Merkmale abhängt, ist die Vorhersage auf Tiefenwerte pro Keypoint angewiesen. Liegen gültige Tiefenmessungen vor, werden diese direkt verwendet. Für Merkmale ohne verlässliche Tiefenmessung greift die Implementierung auf einen Fallback-Wert Z_{fallback} zurück. Auf diese Weise bleibt die Vorhersage auch dann definiert, wenn einzelne Tiefenmessungen fehlen oder außerhalb gültiger Grenzen liegen.

Zusätzlich enthält die C++-Node eine modellbasierte Tiefenberechnung. Für ein Merkmal mit normierten Koordinaten (x_i, y_i), Tiefe Z_i und kamerabezogener Geschwindigkeit wird die Tiefenänderung gemäß

$$\dot{Z}_i = -v_z + Z_i \left(\omega_y x_i - \omega_x y_i\right) \tag{6.7}$$

berechnet. Daraus ergibt sich für den nächsten Zeitschritt

$$Z_{i,k+1} = Z_{i,k} + \dot{Z}_i \, \Delta t. \tag{6.8}$$

Der vorhergesagte Tiefenwert wird anschließend auf einen zulässigen Bereich begrenzt. Diese Tiefenberechnung verbessert die Konsistenz des Modells, da nicht nur die Bildlage, sondern auch die zugrunde liegende Merkmaltiefe zeitlich mitgeführt wird.

6.6 Aktive Keypoint-Menge und geometrische Konsistenz

Die modellbasierte Vorhersage arbeitet nicht notwendigerweise auf allen verfügbaren Referenzmerkmalen, sondern auf einer aktiven Teilmenge. Diese aktive Menge wird innerhalb der Basisklasse des Filters verwaltet und über Parameter wie die maximale Anzahl aktiver Keypoints sowie Mindestanzahlen für Initialisierung und Update begrenzt. Dadurch wird zum einen die Zustandsdimension kontrolliert, zum anderen wird verhindert, dass der Filter bei zu wenigen oder ungeeigneten Beobachtungen in einen instabilen Zustand übergeht.

Ein weiterer wichtiger Aspekt ist die geometrische Konsistenz der vorhergesagten Merkmale. In der Implementierung wird hierzu geprüft, ob aus den aktuellen aktiven Punkten eine sinnvolle Homographie zwischen gewünschter und gefilterter Merkmalslage berechnet werden kann. Erst wenn diese Geometrieprüfung erfolgreich ist, wird die aktualisierte Merkmalsschätzung übernommen. Auf diese Weise wird vermieden, dass numerisch mögliche, aber geometrisch inkonsistente Zustände in den weiteren Regelbetrieb eingehen.

6.7 Zusammenspiel von Vorhersage und Messupdate

Die modellbasierte Keypoint-Vorhersage bildet nur einen Teil des gesamten Schätzzyklus. Nach dem Prädiktionsschritt werden aktuelle Matches aus der Wahrnehmung als Messgröße eingebracht. Das bedeutet, dass die modellierte Bewegung der Merkmale nicht isoliert verwendet wird, sondern kontinuierlich mit realen Beobachtungen abgeglichen wird. Die Vorhersage liefert dabei die erwartete Lage der Merkmale, während das Messupdate Abweichungen korrigiert und auf diese Weise Drift begrenzt.

Das Messupdate erfolgt in der implementierten Architektur nur für diejenigen Merkmale, die zur aktiven Menge gehören und die vorgesehenen Gating- und Konsistenzkriterien erfüllen. Auf diese Weise wird verhindert, dass starke Ausreißer oder geometrisch unplausible Beobachtungen die Vorhersage destabilisieren. Gerade bei reflektierenden Objekten ist dieses Zusammenspiel aus Vorhersage und selektivem Update von zentraler Bedeutung, da reale Bildinformationen zwar wertvoll, aber nicht jederzeit zuverlässig sind.

6.8 Einordnung in das Gesamtsystem

Im Gesamtsystem übernimmt die modellbasierte Keypoint-Vorhersage die Rolle einer stabilisierenden Zwischenschicht zwischen Merkmalsabgleich und Regelung. Die C++-Filter-Node empfängt Referenzmerkmale, aktuelle Matches, Kamerainformationen sowie die Roboterbewegung und publiziert daraus gefilterte beziehungsweise vorhergesagte Merkmalslagen. Diese stehen anschließend dem Regler als alternative oder bevorzugte Merkmalsquelle zur Verfügung. Parallel dazu können sie auch für weitere Matching-Mechanismen, etwa lokale Wiederfindungsstrategien, genutzt werden.

Damit ist die modellbasierte Vorhersage kein isoliertes Hilfsmodul, sondern ein zentrales Bindeglied zwischen Wahrnehmung und Regelung. Sie ermöglicht es, die bekannten Bewegungsinformationen des Roboters in den Bildraum zu überführen und dort als robuste Zusatzinformation für den visuellen Regelkreis nutzbar zu machen.

6.9 Zusammenfassung

Zusammenfassend basiert die modellbasierte Keypoint-Vorhersage in dieser Arbeit auf einem zeitdiskreten Zustandsmodell im Bildraum. Ausgangspunkt sind die aktiven Pixelkoordinaten der Merkmale, deren Veränderung über die punktweise Interaction Matrix und den in den Kamerarahmen transformierten Geschwindigkeitsvektor des Roboters berechnet wird. Durch die zusätzliche Berücksichtigung von Tiefeninformationen, einer aktiven Keypoint-Auswahl, geometrischer Konsistenzprüfungen sowie der anschließenden Korrektur durch reale Messungen entsteht ein robuster Vorhersagemechanismus für Merkmalspositionen. Dieser bildet die Grundlage dafür, den visuellen Regelkreis auch unter instabilen Beobachtungsbedingungen aufrechtzuerhalten.

7 Filter- und Entscheidungslogik

7.1 Ziel des Filters

Das primäre Ziel der in Kapitel 3 vorgestellten Filtermethoden ist es, eine konstante, rauschfreie und durchgehend verfügbare Zustandsschätzung für den Bild-Regelkreis (IBVS) bereitzustellen. Wie zuvor beschrieben, schwankt die Anzahl der von Detektoren erkannten Keypoints N_k stetig, und Ausreißer sowie Verdeckungen treten regelmäßig auf. Dieses Kapitel beschreibt die Architekturentscheidungen und Implementierungsstrategien, mit denen die mathematischen Modelle adaptiert werden, um die Steuerung unter solch dynamischen Umgebungsbedingungen robust auszuführen.

7.2 Dynamisches Keypoint-Tracking und Active-Set-Architektur

In initialen Architektur-Entwürfen wurde untersucht, die Bildpunkte über eine Homographie in vier künstliche *Proxy-Punkte* (eine Bounding-Box) zu transformieren, um eine konstante Dimension des Zustandsvektors zu erzwingen. Dieser Ansatz degradiert das System jedoch zu einem reinen 2D-Objekt-Tracker, wodurch der lokale, strukturelle Bezug individueller Keypoints – die Kernvoraussetzung für ein präzises featurebasiertes Visual Servoing – verloren geht.

Daher wurde eine dynamische *Active-Set-Architektur* entwickelt, die echte, individuelle Keypoints in einem einzigen kombinierten Filter trackt. Das System wählt aus den verfügbaren Roh-Matches deterministisch eine Teilmenge (*Active Set*) von N Keypoints aus (z. B. bis zu $N_{max} = 10$). Die Auswahl (*Spatial Spreading*) maximiert die räumliche Verteilung der Keypoints im Bild, um degenerierte Geometrien (kolineare Punkte) zu vermeiden und eine gut konditionierte Interaktionsmatrix zu garantieren.

Der Zustandsvektor x wird dynamisch auf die Dimension $2N$ (bzw. $4N$ für den Standrad-KF) initialisiert. Im Falle des EKF beinhaltet der Vektor die 2D-Pixelkoordinaten aller aktiven Punkte:

$$x = [u_1, v_1, u_2, v_2, \ldots, u_N, v_N]^T \tag{7.1}$$

Das Data-Association-Problem (die Zuordnung von Messungen zu Zustandsvariablen über mehrere Frames) wird gelöst, indem die festen Referenz-IDs des Feature-Matchers (z. B. *xfeat*) in einer Lookup-Tabelle gespeichert werden.

Tritt nun temporäre Okklusion auf und ein Keypoint des Active Sets wird in der aktuellen Messung nicht gefunden, passt sich die Messmatrix H dynamisch an. Die nicht beobachteten Zustände werden aus H ausgeblendet, wodurch der Filter für diese Punkte rein prädiktiv (über die Roboterkinematik) weiterläuft, während die noch sichtbaren Punkte den Update-Schritt durchlaufen.

7.3 Architekturen und kinematische Vorsteuerung

Bei rein kinematischen Annahmen wie dem CV-Modell divergiert die Prädiktion bei abrupter Roboterbewegung (Filter-Lag). Um dies zu lösen, wird die Steuerungskinematik in das Prädiktionsmodell integriert.

7.3.1 Image-Space EKF

Anstatt die fehleranfällige 3D-Kamera-Pose zu filtern, bleibt der Zustandsvektor im 2D-Bildraum (*Image-Space EKF*) [37]. Die bekannte 3D-Kamerageschwindigkeit v_{cam} wird als Eingangsgröße u_k genutzt. Die resultierende Pixelgeschwindigkeit $\dot{s}$ wird über die Bild-Jacobi-Matrix (Interaktionsmatrix) L_s prädiziert [1]:

$$\dot{s} = L_s(x, Z) \cdot v_{cam} \tag{7.2}$$

Da die Messung z_k direkt aus projizierten Koordinaten besteht, vereinfacht sich das Messmodell zu einer linearen Einheitsmatrix $H = I$. Dies führt zu exzellenter Performance und eliminiert Latenzen durch Vorsteuerung [38].

7.3.2 Semi-Nichtlinearer UKF und ESKF

Als alternative Architekturen werden der UKF sowie der ESKF im System konzeptioniert:

- **Semi-Nichtlinearer UKF:** Da nur das Prädiktionsmodell L_s nichtlinear ist, wird die aufwendige Unscented Transform (UT) nur auf den Prädiktionsschritt angewandt [39]. Der Update-Schritt verbleibt streng linear, was Rechenzeit spart.
- **ESKF:** Obwohl der Bildraum flach ($\mathbb{R}^8$) ist, ermöglicht die Aufteilung in Nominal- und Fehlerzustand eine saubere Entkopplung der asynchronen, hochfrequenten Kinematik (Prädiktion) von der niederfrequenten Sensorkorrektur (Kamera-Update).

7.4 Systemtuning und Parametrisierung

Das Verhalten der Filter hängt maßgeblich von der Parametrisierung ab.

7.4.1 Bestimmung von Q und R im Kalman-Filter

- **Messrauschen R:** Repräsentiert den Pixel-Jitter des Detektors. Bei empirisch ermittelten Schwankungen von ca. ± 10 Pixeln ergibt sich eine Varianz von $R = 100.0 \cdot I$. Ein zu großes R dämpft extrem, erhöht jedoch die Latenz.
- **Prozessrauschen Q:** Modelliert die Unsicherheit der Systemkinematik. Im CV-Modell muss Q hoch gewählt werden ($Q \approx 500.0$), um unmodellierte Beschleunigungen abzufangen. Bei Verwendung der exakten Vorsteuerung über L_s im EKF kann Q drastisch reduziert werden.

7.4.2 Systematisches Tuning des 1€-Filters

Das Tuning des 1€-Filters erfolgt entkoppelt am realen System:

1. **Minimierung des Jitters (Statisch):** Der Geschwindigkeitskoeffizient β wird auf 0 gesetzt. Die Kamera bleibt stationär. Die minimale Grenzfrequenz f_{min} wird verringert (z. B. auf $0.05\,\text{Hz}$), bis das Rauschen ausreichend unterdrückt ist.
2. **Minimierung der Latenz (Dynamisch):** Die Kamera wird rasch bewegt. β wird schrittweise erhöht, bis die dynamische Grenzfrequenz f_c schnell genug ansteigt, sodass der sichtbare Lag (*Phasenverschiebung*) eliminiert wird.

7.5 Fehlerbehandlung und Relokalisation

Wird die Messung über mehrere Frames hinweg verworfen (z. B. durch das Mahalanobis-Gate bei Okklusion), greift das System ausschließlich auf den Prädiktionsschritt zurück. Dies führt zu einem Drift der Features im Bildraum.

Um ein *Deadlocking* des Filters bei andauernder Fehlmessung zu verhindern, ist eine Relokalisations-Logik (Tracking Recovery) implementiert [40]. Fällt die Anzahl der erfolgreich gematchten Keypoints des Active Sets unter einen kritischen Schwellenwert (N_{min_update}, z. B. 4 Punkte), wird ein harter Reset forciert:

- Das bisherige Active Set wird verworfen und ein komplett neues Set aus der aktuellen Rohmessung initialisiert.
- Die Fehlerkovarianz P wird auf einen extrem hohen Initialwert zurückgesetzt, was maximaler Unsicherheit entspricht und das Filter-Vertrauen in die nächste Messung reaktiviert.

7.6 Stabilität und Vermeidung von Singularitäten

Beim geschlossenen Regelkreis müssen mathematische Singularitäten abgefangen werden:

- **Kolinearität und Feature-Verteilung:** Eine Konzentration aller Features in einem engen Bildbereich würde zu einer schlecht konditionierten Interaktionsmatrix L_s führen. Die deterministische *Spread-Selektion* bei der Initialisierung des Active Sets stellt sicher, dass Features räumlich über das Bild gestreut werden, was die numerische Stabilität der Pseudoinversen maximiert.
- **Aktor-Singularitäten (DLS):** Nähert sich der Roboterarm einer Streckung, verliert die Jacobi-Matrix den vollen Rang. Zur Unterdrückung extremer Gelenkgeschwindigkeiten wird die *Damped Least Squares* (DLS) Methode nach Wampler [41] für die Pseudoinverse herangezogen.

8 Feature Confidence und Switching Strategie

8.1 Definitionen und theoretische Grundlagen

Um die Mechanismen des prädiktionsbasierten Matchings formal zu beschreiben, müssen die Begriffe der *Feature Confidence* und der *Switching Strategie* im Kontext der Computer Vision und Zustandsschätzung definiert werden.

8.1.1 Feature Confidence

Die **Feature Confidence** (dt. Merkmalssicherheit) beschreibt ein probabilistisches Maß für die Zuverlässigkeit, mit der ein Bildmerkmal im aktuellen Frame detektiert und korrekt assoziiert werden kann. Sie setzt sich in der Regel aus zwei Komponenten zusammen:

1. **Deskriptor-Konfidenz:** Die Unterscheidbarkeit eines Merkmals gegenüber anderen Kandidaten im Bildraum. Ein klassisches Maß hierfür ist die *Ratio Test Distance* nach [35], welche das Verhältnis zwischen dem besten und dem zweitbesten Match bewertet.
2. **Geometrische Konfidenz:** Die Übereinstimmung der Messung mit der prädizierten Position im Zustandsraum. Diese wird häufig über den Mahalanobis-Abstand zwischen Messung und Prädiktion unter Berücksichtigung der Kovarianzmatrix des Filters bestimmt [12].

Wissenschaftlich lässt sich die Konfidenz als eine A-posteriori-Wahrscheinlichkeit $P(z_t|Z_{1:t-1})$ interpretieren, wobei z_t die aktuelle Messung und $Z_{1:t-1}$ die Historie der Merkmalsbewegungen darstellt.

8.1.2 Switching Strategie

Unter einer **Switching Strategie** versteht man die Entscheidungslogik, die zwischen verschiedenen Matching-Modi (Global Search vs. Local Guided Matching) wechselt. Ziel ist es, die Robustheit des Tracking-Systems gegenüber transienten Störungen (Okklusionen, Bewegungsunschärfe) zu erhöhen, ohne die Echtzeitfähigkeit zu gefährden.

Nach [42] und [43] (Active Matching) ist ein permanentes globales Matching (*Bottom-up*) rechenintensiv und fehleranfällig für Ausreißer. Die Switching Strategie agiert als Kontrollinstanz:

- **Normalbetrieb:** Globales Matching zur Detektion neuer Merkmale.
- **Rescue-Mode:** Bei Unterschreiten einer Konfidenzschwelle wird auf ein prädiktionsbasiertes Fenster-Matching (*Top-down*) umgeschaltet, um den „Track“ des Keypoints aufrechtzuerhalten.

8.2 Prädiktionsbasiertes Local-Rescue-Matching

Klassische Feature-Matching-Verfahren arbeiten in der Regel nach dem reinen Bottom-up-Prinzip: Bildmerkmale werden extrahiert und global mit einer Referenz verglichen. Schlägt dieser Prozess fehl, beispielsweise durch partielle Okklusion oder starke Reflexionen, geht der Keypoint für den Regelkreis verloren. Um dieses Problem zu adressieren, wurde eine Top-down-Rückkopplung (*Guided Matching* oder *Local Rescue*) in das System implementiert.

Der in den vorherigen Kapiteln beschriebene Zustandsfilter berechnet seine Prädiktion mit einer festen, definierten Frequenz, die unabhängig vom asynchronen Mess-Update (Kamerabild) abläuft. Diese prädizierten Pixelkoordinaten sowie die zugehörige Schätzunsicherheit werden kontinuierlich an das Matching-Modul publiziert. Kann ein zuvor aktiver Keypoint im globalen Matching nicht mehr zugeordnet werden, initiiert das System einen lokalen Rettungsversuch rund um die vom Modell vorhergesagte Position.

8.2.1 Adaptive Suchradien und Konfidenzschwellen

Der Local-Rescue-Ansatz nutzt die vom Filter berechnete Unsicherheit (Standardabweichung σ_{px}), um die Parameter der lokalen Suche dynamisch an den Systemzustand anzupassen. Die Unsicherheit wird zunächst basierend auf empirischen Grenzwerten (σ_{low} und σ_{high}) auf ein normiertes Konfidenzmaß $u \in [0, 1]$ skaliert:

$$u = \text{clip}\left(\frac{\sigma_{px} - \sigma_{low}}{\sigma_{high} - \sigma_{low}},\, 0,\, 1\right) \tag{8.1}$$

Dieser Unsicherheitswert u moduliert sowohl den effektiven Suchradius r_{eff} (in Pixeln) als auch den Mindest-Ähnlichkeitswert t_{eff} (*Similarity Threshold*) für den Vergleich der Deskriptoren linear:

$$r_{eff} = r_{min} + u \cdot (r_{max} - r_{min}) \tag{8.2}$$

$$t_{eff} = t_{min} + u \cdot (t_{max} - t_{min}) \tag{8.3}$$

Ist sich der Filter seiner Vorhersage sehr sicher ($u \approx 0$), wird nur in einem sehr engen Radius mit einer moderaten Ähnlichkeitshürde gesucht. Ist der Filter hingegen unsicher, weil die Prädiktion über längere Zeit nicht korrigiert wurde ($u \approx 1$), wird der Suchradius stark vergrößert. Gleichzeitig wird ein sehr exakter Deskriptor-Match (hohes t_{eff}) gefordert, um Fehlassoziationen (*False Positives*) in dem nun vergrößerten Suchbereich zu vermeiden.

8.2.2 Dynamische Score-Gewichtung

Alle Kandidaten, die innerhalb des adaptiven Suchradius r_{eff} gefunden werden und die harte Ähnlichkeitsgrenze überschreiten, werden anhand eines kombinierten Scores bewertet. Dieser setzt sich aus der räumlichen Nähe (*Position Score* S_{pos}) und der optischen Übereinstimmung (*Similarity Score* S_{sim}) zusammen.

Der Position-Score nimmt exponentiell mit der euklidischen Distanz d zur vorhergesagten Position ab. Dies wird als nicht-normalisierte Gauß-Verteilung mit der Breite $\sigma_{pos} = \max(1.0, 0.5 \cdot r_{eff})$ modelliert:

$$S_{pos} = \exp\left(-\frac{1}{2}\left(\frac{d}{\sigma_{pos}}\right)^2\right) \tag{8.4}$$

Der Similarity-Score wird basierend auf der berechneten Cosinus-Ähnlichkeit sim zwischen den Deskriptoren und einem absoluten Minimum sim_{floor} auf das Intervall $[0, 1]$ normiert:

$$S_{sim} = \text{clip}\left(\frac{sim - sim_{floor}}{1.0 - sim_{floor}},\ 0,\ 1\right) \tag{8.5}$$

Um eine situative Priorisierung zu erreichen, wird der finale Score durch das Unsicherheitsmaß u exponentiell gewichtet:

$$Score = (S_{pos})^{1-u} \cdot (S_{sim})^{u} \tag{8.6}$$

Dieses Design garantiert, dass bei hoher Prädiktionsgewissheit (kleines u) die örtliche Nähe bei der Auswahl des Keypoints dominiert. Bei hoher Unsicherheit (großes u) verschiebt sich die Gewichtung, sodass die optische Deskriptor-Ähnlichkeit das ausschlaggebende Kriterium für einen erfolgreichen Match wird.

8.2.3 Ambiguity Check (Mehrdeutigkeitsprüfung)

In industriellen Umgebungen existieren häufig wiederkehrende, identische Strukturen (z. B. Lochraster, Schrauben oder Gitter), die den *Local Rescue* täuschen könnten. Um Fehlzuordnungen in solchen stark repetitiven Szenen zu unterdrücken, wird abschließend ein Ambiguity Check durchgeführt.

Hierfür wird der beste gefundene Kandidat mit dem zweitbesten Kandidaten verglichen. Die Zuordnung wird nur dann als gültig akzeptiert und als erfolgreicher „Rescue“ an den Filter zurückgemeldet, wenn der beste Score sowohl einen absoluten Mindestabstand (*min_gap*) als auch ein definiertes Verhältnis (*min_ratio*) zum zweitbesten Score aufweist. Schlägt diese Eindeutigkeitsprüfung fehl, wird der Keypoint in diesem Frame verworfen, um eine Destabilisierung des Regelkreises zu verhindern.

9 Regelung des Roboterarms

Dieses Kapitel beschreibt die Regelung des Roboterarms im entwickelten IBVS-System. Ziel der Regelung ist es, die Differenz zwischen aktuell beobachteten und gewünschten Bildmerkmalen zu minimieren und daraus kartesische Bewegungsbefehle für den Roboter abzuleiten. Im Gegensatz zu einer rein bildbasierten Standardregelung wird der Regler in dieser Arbeit so erweitert, dass neben direkt beobachteten Merkmalen auch gefilterte beziehungsweise modellgestützte Merkmalspositionen verarbeitet werden können. Dadurch soll insbesondere in Szenarien mit Reflexionen, kurzzeitigem Featureverlust und instabiler Tiefeninformation ein robusteres Regelverhalten erreicht werden.

9.1 Regelungsprinzip

Die entwickelte Regelung folgt dem Grundprinzip des IBVS. Ausgangspunkt sind Bildmerkmale, die sowohl in einer Referenzansicht als auch im aktuellen Kamerabild vorliegen. Für jedes Merkmal wird die Differenz zwischen aktueller und gewünschter Bildposition berechnet. Aus diesem Bildfehler wird anschließend ein kartesischer Geschwindigkeitsvektor bestimmt, der den TCP des Roboters so bewegt, dass sich die Merkmale der Referenzlage annähern.

Seien

$$\mathbf{s}_i = \begin{bmatrix} u_i \\ v_i \end{bmatrix} \qquad \text{und} \qquad \mathbf{s}_i^* = \begin{bmatrix} u_i^* \\ v_i^* \end{bmatrix} \tag{9.1}$$

die aktuelle beziehungsweise gewünschte Bildposition des i-ten Merkmals. Dann ergibt sich der zu minimierende Bildfehler zu

$$\mathbf{e}_i = \mathbf{s}_i - \mathbf{s}_i^*. \tag{9.2}$$

Für n Merkmale wird daraus der Gesamtfehlervektor

$$\mathbf{e} = \begin{bmatrix} u_1 - u_1^* & v_1 - v_1^* & \cdots & u_n - u_n^* & v_n - v_n^* \end{bmatrix}^\top. \tag{9.3}$$

Da die Regelung nicht direkt in Pixelkoordinaten, sondern in normierten Bildkoordinaten formuliert wird, erfolgt zunächst eine Umrechnung mit den Kameraintrinsiken:

$$x_i = \frac{u_i - c_x}{f_x}, \qquad y_i = \frac{v_i - c_y}{f_y}. \tag{9.4}$$

Die normierten Koordinaten bilden anschließend die Grundlage für die Interaktionsmatrix und damit für die Berechnung des Geschwindigkeitsbefehls.

9.2 Einbindung realer und gefilterter Keypoints

Der Regler verarbeitet nicht ausschließlich rohe, direkt aus dem Kamerabild stammende Merkmale. Stattdessen ist die Architektur so ausgelegt, dass zwischen unterschiedlichen Merkmalsquellen unterschieden werden kann. Grundsätzlich stehen zwei Arten von Eingangsgrößen zur Verfügung:

1. direkt gematchte Merkmale aus dem aktuellen Bild und
2. gefilterte beziehungsweise modellgestützte Merkmale aus der vorgelagerten Filterstufe.

Im Normalfall wird die gefilterte Merkmalsquelle bevorzugt verwendet, da diese zeitlich geglättete und im Idealfall robustere Bildpunktlagen liefert. Dies ist insbesondere dann vorteilhaft, wenn einzelne Keypoints aufgrund von Reflexionen, Belichtungsänderungen oder kurzzeitiger Verdeckung instabil werden. Für den Fall, dass keine aktuellen gefilterten Merkmale verfügbar sind, kann der Regler auf rohe Matches zurückfallen. Dadurch bleibt der Regelkreis auch dann arbeitsfähig, wenn die Filterstufe temporär keine verlässlichen Schätzungen bereitstellt.

Diese Einbindung gefilterter Keypoints stellt die zentrale Verbindung zu den vorherigen Kapiteln dar: Während dort die Bewertung, Auswahl und Stabilisierung der Merkmale erfolgt, nutzt die Regelung diese Informationen als robuste Eingangsgröße zur Berechnung der Roboterbewegung.

9.3 Berechnung der kartesischen Stellgröße

Die Beziehung zwischen Bildfehler und Kamerabewegung wird über die Interaktionsmatrix beschrieben. Für ein einzelnes Punktmerkmal mit normierten Bildkoordinaten (x_i, y_i) und Tiefe z_i ergibt sich die bekannte Punkt-Interaktionsmatrix zu

$$\mathbf{L}_i = \begin{bmatrix} -\frac{1}{z_i} & 0 & \frac{x_i}{z_i} & x_i y_i & -(1+x_i^2) & y_i \\ 0 & -\frac{1}{z_i} & \frac{y_i}{z_i} & 1+y_i^2 & -x_i y_i & -x_i \end{bmatrix}. \tag{9.5}$$

Für n Merkmale ergibt sich daraus folgende Gesamtinteraktionsmatrix:

$$\mathbf{L} = \begin{bmatrix} \mathbf{L}_1 \\ \mathbf{L}_2 \\ \vdots \\ \mathbf{L}_n \end{bmatrix}. \tag{9.6}$$

Zur Berechnung des Geschwindigkeitsvektors wird in der vorliegenden Arbeit eine gedämpfte Least-Squares-Lösung verwendet. Diese ist numerisch robuster als eine direkte Inversion und reduziert die Empfindlichkeit gegenüber schlecht konditionierten Konfigurationen oder einer ungünstigen Merkmalsverteilung. Der kamerabezogene Geschwindigkeitsvektor $\mathbf{v}_{\text{cam}}$ ergibt sich zu

$$\mathbf{v}_{\text{cam}} = -\lambda \left(\mathbf{L}^\top \mathbf{L} + \mu^2 \mathbf{I} \right)^{-1} \mathbf{L}^\top \mathbf{e}, \tag{9.7}$$

wobei λ den Reglerverstärkungsfaktor und μ die Dämpfung der Damped Least Squares (DLS)-Lösung bezeichnet.

Ein wesentliches Merkmal der Implementierung besteht darin, dass die aktiven kartesischen DoFs gezielt freigeschaltet werden können. Dadurch ist der Regler nicht auf eine feste Bewegungsdimension beschränkt. Vielmehr kann für die Versuche untersucht werden, wie sich unterschiedliche Kombinationen translatorischer und rotatorischer Freiheitsgrade auf das Regelverhalten auswirken. Methodisch entspricht dies einer Reduktion der Interaktionsmatrix auf die jeweils aktiven Spalten. Die resultierende Lösung wird anschließend wieder in einen vollständigen sechsdimensionalen Twist eingebettet, wobei inaktive Komponenten auf Null gesetzt werden.

9.4 Tiefenverarbeitung

Für die Berechnung der Interaktionsmatrix wird pro Merkmal eine Tiefeninformation benötigt. Liegen gültige Tiefenwerte vor, werden diese direkt verwendet. In realen industriellen Szenarien ist jedoch davon auszugehen, dass Tiefenmessungen nicht jederzeit vollständig oder zuverlässig verfügbar sind. Insbesondere reflektierende Oberflächen können zu unvollständigen oder verrauschten Tiefenschätzungen führen.

Aus diesem Grund verwendet die Regelung zusätzlich eine Tiefen-Fallback-Strategie. Falls für einzelne Merkmale keine gültige Tiefe vorliegt, wird auf einen Ersatzwert zurückgegriffen. Dieser kann aus einer Grundschätzung stammen oder durch zeitliche Glättung vorheriger Tiefenbeobachtungen stabilisiert werden. Auf diese Weise bleibt die Interaktionsmatrix auch bei partiell fehlender Tiefeninformation definiert, und der Regler kann weiterarbeiten, ohne unmittelbar in einen instabilen Zustand zu geraten.

Die Tiefenverarbeitung ist damit kein separates Modul neben dem Regler, sondern ein integraler Bestandteil der eigentlichen Stellgrößenberechnung. Sie trägt wesentlich dazu bei, dass die Regelung auch unter praxisnahen Bedingungen mit unvollständigen Sensordaten robust bleibt.

9.5 Transformation in das Roboterframe

Der aus dem Bildfehler berechnete Geschwindigkeitsvektor liegt zunächst im Kamerakoordinatensystem vor. Für die Ansteuerung des Roboterarms muss dieser Vektor jedoch in das Koordinatenframe überführt werden, welches der kartesische Robotercontroller erwartet. Zu diesem Zweck wird der kamerabezogene Twist zunächst in das TCP-Frame und anschließend in das Basisframe des Roboters transformiert.

Für die Transformation vom Kameraframe in das TCP-Frame wird neben der Rotationsbeziehung auch der Hebelarm zwischen Kamera und TCP berücksichtigt. Damit ergibt sich für den translatorischen Anteil

$$\mathbf{v}_{\mathrm{tcp}} = \mathbf{R}_{\mathrm{tcp}\leftarrow\mathrm{cam}}\,\mathbf{v}_{\mathrm{cam}} - \boldsymbol{\omega}_{\mathrm{tcp}} \times \mathbf{p}_{\mathrm{tcp}\leftarrow\mathrm{cam}}, \tag{9.8}$$

wobei $\mathbf{R}_{\mathrm{tcp}\leftarrow\mathrm{cam}}$ die Rotationsmatrix zwischen Kamera- und TCP-Frame und $\mathbf{p}_{\mathrm{tcp}\leftarrow\mathrm{cam}}$ den Positionsvektor zwischen beiden Frames bezeichnet. Anschließend wird der Twist in das Basisframe des Roboters transformiert:

$$\mathbf{v}_{\mathrm{base}} = \mathbf{R}_{\mathrm{base}\leftarrow\mathrm{tcp}}\,\mathbf{v}_{\mathrm{tcp}}. \tag{9.9}$$

Diese Transformation ist für die praktische Einsetzbarkeit des Reglers entscheidend. Nur so kann sichergestellt werden, dass die aus dem Bildraum berechnete Korrekturbewegung konsistent in eine physikalisch sinnvolle Bewegung des Roboterarms überführt wird.

9.6 Stabilisierung des Regelkreises

Damit der Regler auch unter schwankenden Beobachtungsbedingungen ein ruhiges Bewegungsverhalten zeigt, werden mehrere Stabilisierungsmechanismen kombiniert. Zunächst wird die berechnete Stellgröße betragsmäßig begrenzt, sodass lineare und rotatorische Geschwindigkeiten definierte Maximalwerte nicht überschreiten. Dies verhindert abrupte Kommandos und erhöht die Betriebssicherheit.

Zusätzlich erfolgt eine zeitliche Glättung des Geschwindigkeitsvektors. Dazu werden sprunghafte Änderungen zunächst durch eine kurze Medianbildung unterdrückt und anschließend mit einer Exponential Moving Average (EMA)-artigen Glättung zeitlich abgefedert. Ergänzend kann die Änderungsgeschwindigkeit der Stellgröße begrenzt werden. Dadurch wird nicht nur der absolute Betrag, sondern auch die zeitliche Dynamik des Befehls beschränkt. Besonders bei verrauschten Bildmerkmalen oder kurzzeitigen Fehlzuordnungen reduziert dies das Risiko eines hektischen oder oszillierenden Roboterverhaltens.

Ein weiterer stabilisierender Aspekt ist die gedämpfte Lösung der IBVS-Gleichung. Durch die DLS-Formulierung reagiert der Regler weniger empfindlich auf ungünstige Merkmalskonfigurationen und bleibt auch dann numerisch beherrschbar, wenn die Interaktionsmatrix schlecht konditioniert ist.

9.7 Verhalten bei Featureverlust und unsicheren Beobachtungen

Ein zentrales Ziel der vorliegenden Arbeit ist die Stabilisierung des Regelkreises bei unsicheren oder zeitweise ausfallenden Bildmerkmalen. Entsprechend sieht die Regelung mehrere Schutzmechanismen vor.

Zunächst wird eine Bewegung nur dann freigegeben, wenn die Initialisierung abgeschlossen ist und überhaupt eine gültige Referenz vorliegt. Weiterhin wird geprüft, ob eine ausreichende Anzahl aktueller Merkmalszuordnungen verfügbar ist. Ist dies nicht der Fall, wird kein Bewegungsbefehl ausgegeben. Dasselbe gilt, wenn die zuletzt empfangenen Merkmale bereits veraltet sind oder keine konsistente Merkmalsquelle ausgewählt werden kann.

Darüber hinaus wird die aktuelle Abweichung zur Referenz kontinuierlich überwacht. Unterschreitet der Bildfehler einen vorgegebenen Schwellwert über eine gewisse Haltezeit, wird das Ziel als erreicht betrachtet und der Regler gibt ein Nullkommando aus. Dadurch wird vermieden, dass der Roboter in der Zielnähe unnötig weiterregelt oder auf geringfügiges Bildrauschen reagiert.

Gerade im Zusammenspiel mit den vorgelagerten Confidence- und Switching-Mechanismen zeigt sich hier der Mehrwert des Gesamtansatzes: Der Regler ist nicht darauf angewiesen, dass jederzeit alle realen Features stabil sichtbar sind. Stattdessen kann er mit gefilterten oder modellgestützten Merkmalslagen weiterarbeiten und nur dann auf einen sicheren Stillstand gehen, wenn die Beobachtungslage insgesamt nicht mehr ausreichend verlässlich ist.

9.8 Einordnung im Gesamtsystem

Die Regelung des Roboterarms bildet die letzte Verarbeitungsstufe des entwickelten Systems. Während die vorherigen Komponenten die Detektion, Zuordnung, Bewertung und gegebenenfalls Modellierung von Keypoints übernehmen, übersetzt der Regler diese Information in eine physische Roboterbewegung. Der Regler ist damit nicht isoliert zu betrachten, sondern als Teil einer Kette aus Merkmalsextraktion, Matching, Filterung und Entscheidungslogik.

Aus methodischer Sicht ist dabei besonders relevant, dass die Regelung gezielt auf die Herausforderungen reflektierender Objekte ausgelegt wurde. Die Einbindung gefilterter Keypoints, die robuste Tiefenbehandlung, die Freischaltung unterschiedlicher DoFs sowie die Glättung und Begrenzung der Stellgrößen schaffen gemeinsam die Grundlage für einen stabileren visuellen Regelkreis auch unter schwierigen Bildbedingungen.

9.9 Zusammenfassung

Zusammenfassend realisiert die entwickelte Reglerstruktur eine robuste IBVS-basierte Ansteuerung des Roboterarms. Die Regelung verarbeitet Referenz- und Ist-Merkmale, berechnet daraus über die Interaktionsmatrix einen kartesischen Geschwindigkeitsvektor und transformiert diesen in den Roboterbasisrahmen. Durch die Einbindung gefilterter Keypoints, eine stabile Tiefenbehandlung, die selektive Aktivierung von DoFs sowie mehrere Schutz- und Glättungsmechanismen wird der Regelkreis gezielt auf instabile Sichtbedingungen vorbereitet. Damit bildet das Kapitel die methodische Grundlage für die spätere experimentelle Untersuchung des Regelverhaltens unter unterschiedlichen Beobachtungs- und Störbedingungen.

10 Softwareimplementierung

Dieses Kapitel beschreibt die softwaretechnische Umsetzung des entwickelten IBVS-Systems. Ziel der Implementierung war eine modulare und erweiterbare Architektur, die Wahrnehmung, Referenzverwaltung, Merkmalsabgleich, Filterung und Regelung klar voneinander trennt. Gleichzeitig sollte die Software so aufgebaut sein, dass unterschiedliche Detektoren, Filtervarianten und Regelungsparameter mit möglichst geringem Anpassungsaufwand untersucht werden können.

Die Umsetzung basiert auf ROS2 und folgt einem paketorientierten Aufbau. Die Kommunikation zwischen den einzelnen Komponenten erfolgt über Topics, Services und Parameter. Dadurch lässt sich das System flexibel konfigurieren und schrittweise erweitern. Ein weiterer Schwerpunkt lag auf der Trennung zwischen experimenteller Entwicklungsumgebung in Python und einer performanteren C++-Variante für den Filterteil.

10.1 Systemumgebung und Softwarebasis

Die Software wurde für den Einsatz in einer ROS2-basierten Umgebung entwickelt. Der Gesamtaufbau orientiert sich an einem klassischen verteilten Robotiksystem, in dem Wahrnehmung, Schätzung und Regelung als voneinander getrennte Knoten ausgeführt werden. Für die Implementierung wurden sowohl Python-basierte als auch C++-basierte ROS2-Pakete verwendet. Dadurch konnte einerseits eine schnelle Entwicklung und Anpassung von Algorithmen realisiert werden, andererseits stehen für rechenintensive Komponenten performantere Implementierungen zur Verfügung.

Die modulare Umsetzung unterstützt darüber hinaus eine reproduzierbare Integration in eine containerisierte Entwicklungsumgebung. Dies ist insbesondere für die Nachvollziehbarkeit von Experimenten, die Verwaltung von Abhängigkeiten sowie den Austausch zwischen verschiedenen Rechnern von Vorteil.

10.2 Paketstruktur des ROS2-Workspaces

Der entwickelte Workspace ist in mehrere funktional getrennte Pakete unterteilt. Eine zentrale Rolle übernimmt dabei das Paket `ibvs_msgs`, das die benutzerdefinierten Nachrichtenstrukturen des Systems bereitstellt. Dieses Paket bildet die gemeinsame Kommunikationsgrundlage für alle weiteren Komponenten.

Das Paket `ibvs_perception` bildet den Einstiegspunkt der visuellen Verarbeitung. Es enthält die Keypoint-Erkennung aus Kamerabildern und ist als Python-Paket umgesetzt. Darauf aufbauend übernimmt `ibvs_matching` den Merkmalsabgleich zwischen aktuellen und Referenzmerkmalen. Für die modellbasierte Schätzung und Glättung

steht `ibvs_filter_cpp` als C++-basierte leistungsorientierte Ausführung zur Verfügung. Die Erzeugung der Roboterkommandos erfolgt im Paket `ibvs_control`, während `ibvs_reference` die Referenzmerkmale verwaltet und für den Regelkreis bereitstellt.

Durch diese Paketstruktur wird eine klare funktionale Trennung zwischen Wahrnehmung, Merkmalsabgleich, Zustandsschätzung, Referenzverwaltung und Regelung erreicht. Gleichzeitig lassen sich einzelne Komponenten unabhängig voneinander erweitern oder austauschen, ohne die Gesamtarchitektur grundlegend verändern zu müssen.

10.3 Implementierung der zentralen ROS2-Nodes

10.3.1 Keypoint-Erkennung

Die visuelle Wahrnehmung wird im Wesentlichen durch die Node `keypoint_node.py` realisiert. Diese empfängt die Kameradaten und extrahiert daraus die für das IBVS benötigten Bildmerkmale. Die Implementierung ist so ausgelegt, dass unterschiedliche Detektoren verwendet werden können. Hierzu zählen klassische Verfahren wie ORB und SIFT sowie lernbasierte Ansätze wie SuperPoint und XFeat. Dadurch ist die Wahrnehmungskomponente nicht auf einen einzelnen Merkmalsextraktor festgelegt, sondern kann abhängig vom Anwendungsfall angepasst werden.

Neben der Farbbildverarbeitung unterstützt die Node auch die Einbindung ausgerichteter Tiefendaten. Diese werden unter anderem genutzt, um eine tiefenbasierte ROI zu erzeugen und damit nahe beziehungsweise relevante Bildbereiche gezielt auszuwerten. Zusätzlich stellt der Knoten Debug-Topics zur Visualisierung detektierter Keypoints und verwendeter Masken bereit. Die Ausgabe der Merkmale erfolgt über eine benutzerdefinierte Nachricht des Typs `Keypoints`.

10.3.2 Referenzverwaltung

Die Verwaltung der Referenzmerkmale erfolgt über die Node `reference_manager_node.py`. Diese übernimmt die Aufgabe, während einer Initialisierungsphase mehrere Keypoint-Beobachtungen zu sammeln, zu konsolidieren und daraus eine stabile Referenzmenge zu erzeugen. Die Referenz wird anschließend auf dem Topic `/ibvs/reference/keypoints` bereitgestellt. Zusätzlich publiziert die Node über `/ibvs/init_done`, ob die Initialisierung erfolgreich abgeschlossen wurde. Beide Publisher sind dabei als latched-artige, transiente Verbindungen ausgelegt, sodass neu startende Nodes unmittelbar den aktuellen Zustand erhalten.

Zur Bildung einer stabilen Referenz werden wiederkehrende Kandidaten über ihre Position und ihre Deskriptorähnlichkeit zusammengeführt. Die Implementierung verwendet dabei unter anderem eine Cosinus-Ähnlichkeit für den Vergleich der Merkmalsvektoren sowie räumliche Gating-Kriterien im Bildraum. Auf diese Weise wird vermieden, dass instabile Einzelbeobachtungen direkt als Referenz übernommen werden. Die Referenzverwaltung bildet damit die Grundlage für einen konsistenten Merkmalsabgleich im späteren Trackingbetrieb.

10.3.3 Merkmalsabgleich

Der Merkmalsabgleich wird im Paket `ibvs_matching` umgesetzt. Dessen zentrale Aufgabe besteht darin, die aktuell detektierten Merkmale mit den zuvor gespeicherten Referenzmerkmalen in Beziehung zu setzen. Die resultierenden Zuordnungen werden anschließend als Eingangsgröße für Filterung und Regelung verwendet.

Die Implementierung unterstützt neben dem eigentlichen Deskriptor-Matching zusätzliche Mechanismen zur Robustheitssteigerung. Dazu zählen ein optionaler Prefilter zur Begrenzung auf besonders aussagekräftige Matches sowie ein konfigurierbarer Local-Rescue-Mechanismus. Letzterer ermöglicht es, ausgefallene globale Zuordnungen lokal wiederherzustellen, indem modellierte oder gefilterte Merkmalslagen als zusätzliche Hilfsinformation genutzt werden. Ergänzend stehen Visualisierungs- und Debug-Komponenten zur Verfügung, um die Qualität des Matchings im laufenden Betrieb zu analysieren.

10.3.4 Filterung und Zustandsschätzung

Die modellbasierte Filterung und Zustandsschätzung wird in der vorliegenden Implementierung durch die C++-Node `filter_node` aus dem Paket `ibvs_filter_cpp` realisiert. Diese Node ist als performante Laufzeitkomponente für die Verarbeitung der Merkmalszustände ausgelegt und bildet die zentrale Schätzinstanz zwischen Merkmalsabgleich und Regelung. Sie verarbeitet Referenzmerkmale, aktuelle Matches, Kamerainformationen sowie den vom Robotercontroller bereitgestellten Geschwindigkeitsinput und erzeugt daraus gefilterte Merkmalslagen für den weiteren Regelbetrieb.

Die Node unterstützt mehrere Filtertypen, die über den Parameter `filter_type` ausgewählt werden können. Implementiert sind ein EKF, ein UKF, ein ESKF sowie ein KF. Nach Empfang der Kameraintrinsiken aus der `CameraInfo`-Nachricht wird der jeweils gewählte Filter instanziiert und mit den aktuellen Rausch- und Gating-Parametern initialisiert. Zu den zentralen Laufzeitparametern gehören unter anderem `q_noise`, `r_noise`, `gate_threshold`, `z_depth`, `predict_rate`, `max_active_keypoints`, `min_init_keypoints` und `min_update_keypoints`.

Für die Prädiktion der Merkmalsbewegung nutzt die Node neben den Referenz- und Match-Daten auch den kamerabezogenen beziehungsweise in das Kameraframe transformierten Geschwindigkeitsvektor. Hierzu wird der empfangene kartesische Twist über die TF-Beziehungen zwischen Basis-, TCP- und Kameraframe in das Kameraframe umgerechnet. Auf dieser Grundlage kann die erwartete Bewegung der aktiven Merkmale modellbasiert vorhergesagt und anschließend mit den gemessenen Matches abgeglichen werden. Die gefilterten Ergebnisse werden auf `/ibvs/filtered_features` publiziert und stehen dem Regler damit als stabilisierte Merkmalsquelle zur Verfügung. Zusätzlich veröffentlicht die Node Status- und Diagnoseinformationen, darunter Filterstatus, Unsicherheit, Update-Status sowie die Anzahl aktiver und erfolgreich aktualisierter Merkmale.

Ein weiterer Vorteil der C++-Implementierung besteht in ihrer expliziten Unterstützung von Debug- und Testmodi. Über Laufzeitparameter beziehungsweise den Session Manager können unter anderem ein reiner Prädiktionsmodus ohne Match-Updates, eine manuelle Relokalisierung sowie ein kombinierter Reset-Freeze-Ablauf ausgelöst werden.

Dadurch lässt sich das Verhalten der Schätzung gezielt unter verschiedenen Störbedingungen untersuchen, ohne die Node neu implementieren oder den Gesamtablauf des Systems verändern zu müssen.

10.3.5 Reglerknoten

Die eigentliche Ansteuerung des Roboterarms erfolgt in der Node `ibvs_twist_controller_node.py`. Diese verarbeitet die aktuelle Referenz, rohe Matches und optional gefilterte Merkmale. Über den Parameter `feature_source` kann festgelegt werden, ob bevorzugt mit rohen oder mit gefilterten Merkmalen gearbeitet werden soll. Zusätzlich ist ein Fallback-Mechanismus implementiert, sodass bei fehlenden oder veralteten gefilterten Merkmalen automatisch auf rohe Matches zurückgegriffen werden kann.

Der Regler publiziert kartesische Twist-Kommandos für den Robotercontroller und stellt darüber hinaus ein separates Statussignal bereit, das das Erreichen des Regelziels kennzeichnet. Zusätzlich kann ein Debug-Topic für die Winkelgeschwindigkeit aktiviert werden. Für die tatsächliche Befehlsausgabe werden mehrere Verarbeitungsschritte kombiniert, darunter Geschwindigkeitsbegrenzung, Median- beziehungsweise EMA-basierte Glättung sowie eine TF-basierte Umrechnung des im Kameraframes berechneten Twists in den vom Roboter erwarteten Koordinatenframe.

10.4 Nachrichtenformate und Datenfluss

Die Kommunikation zwischen den einzelnen Nodes basiert auf benutzerdefinierten Nachrichten aus dem Paket `ibvs_msgs`. Diese bilden die Grundlage für den Austausch visueller Merkmale, Merkmalszuordnungen und Debug-Informationen. Im Zentrum stehen dabei insbesondere die Nachrichtentypen `Keypoints` und `Matches`. Während `Keypoints` die extrahierten Merkmale einschließlich ihrer Bildpositionen und Deskriptorinformationen repräsentiert, beschreibt `Matches` die Zuordnung zwischen aktuellen und gewünschten beziehungsweise referenzierten Merkmalen.

Der resultierende Datenfluss folgt der logischen Struktur des IBVS-Systems. Zunächst werden Kamerabilder in der Wahrnehmungskomponente verarbeitet und in Keypoints überführt. Diese werden anschließend mit den Referenzmerkmalen abgeglichen. Die daraus entstehenden Matches dienen wiederum als Eingangsgröße für die Filterung und für die Regelung. Der Regler erzeugt aus diesen Informationen kartesische Geschwindigkeitsbefehle für den Roboterarm. In vereinfachter Form ergibt sich somit der Ablauf:

$$\text{Kamerabild} \rightarrow \text{Wahrnehmung} \rightarrow \text{Matching} \rightarrow \text{Filterung} \rightarrow \text{Regelung} \rightarrow \text{Roboter}.$$

Diese Aufteilung erlaubt eine klare Trennung der Verantwortlichkeiten innerhalb des Systems. Gleichzeitig erleichtert sie das Debugging, da jede Verarbeitungsstufe über klar definierte Ein- und Ausgänge verfügt.

10.5 Start- und Laufzeitsteuerung über den Session Manager

Im Gegensatz zu einer rein launch-basierten Inbetriebnahme wird das System in der vorliegenden Arbeit über einen interaktiven Session Manager orchestriert. Dieser übernimmt das Starten, Stoppen und Parametrieren der zentralen Knoten zur Laufzeit. Die Implementierung unterscheidet dabei zwischen einer Core-Phase, einer Initialisierungsphase, einer Tracking-Phase und dem optionalen Filterbetrieb. Über das textbasierte Menü können unter anderem Core-Knoten gestartet, die Referenzinitialisierung ausgelöst, Tracking aktiviert oder verschiedene Filter- und Reglereinstellungen vorgenommen werden.

Beim Start des Core-Betriebs werden zunächst Keypoint-Erkennung und Referenzverwaltung aktiviert. Die Initialisierung ruft anschließend einen Service zur Referenzaufnahme auf und wartet auf das Signal `/ibvs/init_done`. Erst wenn dieser Zustand erreicht ist, wird der Trackingbetrieb mit Descriptor Matcher und Visualisierung freigegeben. Während des Übergangs zwischen Initialisierung und Tracking können außerdem Kamera- und Keypoint-Parameter angepasst werden, beispielsweise zur Nutzung tiefenbasierter Maskierung in der Initialisierungsphase und zu einer reduzierten Verarbeitung im späteren Trackingbetrieb.

Darüber hinaus erlaubt der Session Manager das Setzen von Local-Rescue-Modi, Prefilter-Parametern, Filtertypen und Glättungsprofilen des Controllers. Für den Regler sind mehrere Profile hinterlegt, die unterschiedliche Kompromisse zwischen Reaktionsgeschwindigkeit und Laufruhe abbilden. Auf diese Weise wird die Laufzeitsteuerung zu einem integralen Bestandteil der Softwarearchitektur und nicht lediglich zu einer externen Startlogik.

10.6 Besonderheiten der Implementierung

Eine wesentliche Eigenschaft der entwickelten Software ist ihre konsequente Modularität. Die einzelnen Verarbeitungsschritte sind nicht nur logisch, sondern auch implementierungstechnisch voneinander getrennt. Dadurch können alternative Detektoren, Matching-Strategien, Filtertypen oder Reglerkonfigurationen gezielt untersucht werden. Dies ist insbesondere für die experimentelle Arbeit von Vorteil, da Änderungen in einer Stufe nicht zwangsläufig Anpassungen in allen anderen Modulen erfordern.

Hinzu kommt die bewusste Unterscheidung zwischen experimenteller Flexibilität und späterer Leistungsoptimierung. Während große Teile des Systems in Python umgesetzt wurden, steht für die Filterung zusätzlich eine C++-Variante zur Verfügung. Diese Kombination ermöglicht sowohl eine schnelle Entwicklung als auch eine leistungsfähigere Ausführung für rechenkritische Komponenten.

Schließlich wurde die Implementierung so gestaltet, dass neben dem eigentlichen Regelbetrieb auch Debugging und Analyse gezielt unterstützt werden. Hierzu zählen latched-artige Referenzpublisher, Debug-Topics für Wahrnehmung und Matching, Laufzeitparameter für Filter und Regler sowie die menügesteuerte Steuerung der Gesamtpipeline. Die Software dient damit nicht nur als technische Umsetzung des Verfahrens, sondern gleichzeitig als experimentelle Forschungsplattform.

10.7 Zusammenfassung

Zusammenfassend wurde das entwickelte IBVS-System als modularer ROS2-Workspace mit klar voneinander getrennten Paketen für Nachrichten, Wahrnehmung, Referenzverwaltung, Matching, Filterung und Regelung umgesetzt. Die Kommunikation erfolgt über benutzerdefinierte Nachrichten und standardisierte ROS2-Mechanismen. Durch die Kombination aus Python- und C++-Komponenten, einer interaktiven Laufzeitsteuerung sowie umfangreichen Debug- und Parametrierungsmöglichkeiten entstand eine flexible Softwarebasis, die sowohl für die experimentelle Untersuchung als auch für eine robuste praktische Ausführung geeignet ist.

11 Benchmarking-Methodik und Evaluierungsmetriken

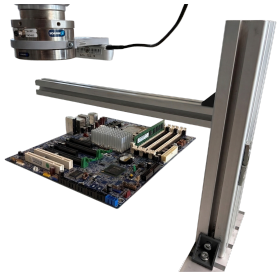
Um die Leistungsfähigkeit der entwickelten Filter- und Entscheidungslogik objektiv zu bewerten, wird ein systematisches Benchmarking durchgeführt. Die Herausforderung besteht hierbei in der hohen Anzahl an Freiheitsgraden innerhalb der Versuchskonfiguration, die zu einer kombinatorischen Explosion der Testfälle führt.

11.1 Definition des Versuchsraums

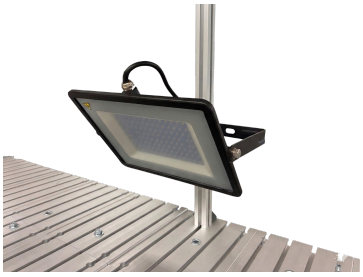
Der theoretische Versuchsraum spannt sich aus fünf Dimensionen auf:

- **Verarbeitungsstufen:**
  - Baseline (reine Keypoint-Detektion ohne Filterung)
  - Zustandsfilterung (Tracking der Keypoints)
  - Zustandsfilterung mit integriertem Local Rescue (LR)
- **Filter-Algorithmen:**
  - Kalman-Filter (KF)
  - Extended Kalman-Filter (EKF)
  - Unscented Kalman-Filter (UKF)
  - Error-State Kalman-Filter (ESKF)
  - 1€-Filter
- **Test-Szenarien:**
  - **Szenario I (Standard):** Ideale Beleuchtung, keine Verdeckungen.
  - **Szenario II (Okklusion):** Temporäre Verdeckung der Zielpunkte (z. B. durch eine 3D-Druck-Brücke).
  - **Szenario III (Lichtwechsel):** Starke Reflexionen und Überbelichtung (Specular Highlights).
- **Bewegungs-Levels (DoF):**
  - **Level a:** Translation in x, y, z sowie Rotation um γ (z-Achse). α, β fixiert.
  - **Level b:** Volle 6 Freiheitsgrade (Full DoF).

- **Stichproben/Bauteile:**
  - **Typ 1 (Texturiert, nicht spiegelnd):** Ein Bauteil mit ausgeprägter Textur und diffuser Oberfläche (z. B. ein beschriftetes Blatt Papier, siehe Abbildung 11.1c).
  - **Typ 2 (Texturiert, spiegelnd):** Ein Bauteil mit Textur, das jedoch spiegelnde Eigenschaften aufweist (z. B. eine Leiterplatte oder ein Mainboard, siehe Abbildung 11.1d).
  - **Typ 3 (Wenig Textur, stark spiegelnd):** Ein Bauteil mit geringer oder keiner Textur und stark spiegelnder Oberfläche (z. B. ein metallisches Bauteil oder eine polierte Fläche, siehe Abbildung 11.1e).

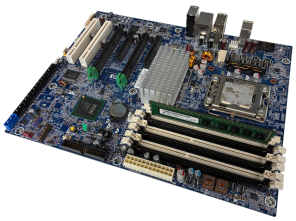


(a) Szenario II: Für Okklusion genutzte Brücke



(b) Szenario III: Für Lichtwechsel gen. Leuchtmittel

(c) Typ 1: Texturiert, nicht spiegelnd



(d) Typ 2: Texturiert, spiegelnd

(e) Typ 3: Wenig Textur, stark spiegelnd

Abbildung 11.1: Obere Reihe: Visualisierung der Testszenarien II und III.
Untere Reihe: Beispiele der verwendeten Stichproben/Bauteile für die Experimente.[1]

[1] Der Hintergrund des Bildes wurde zur besseren Visualisierung mit KI-Werkzeugen freigestellt und bearbeitet.

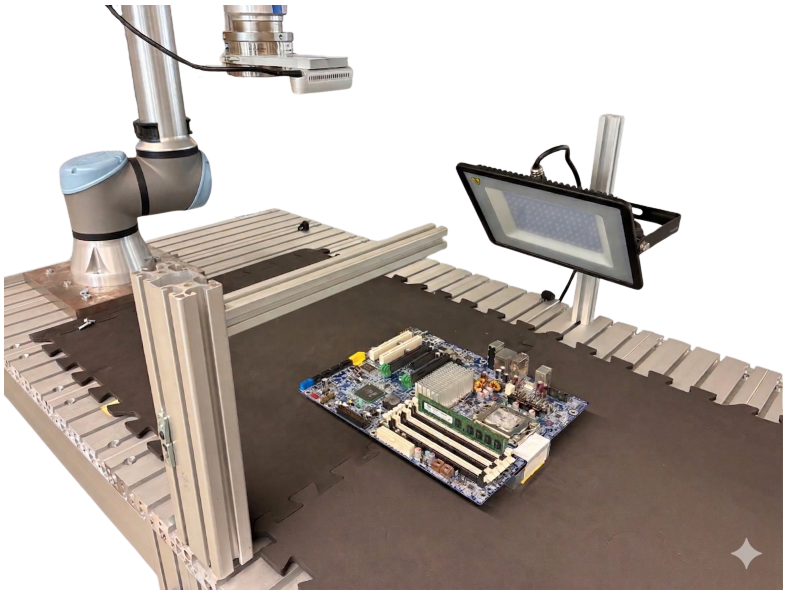


Abbildung 11.2: Gesamter experimenteller Testaufbau (inkl. Brücke für Szenario II und Leuchtmittel für Szenario III; beispielhaft das Bauteil Typ 2).[2]

11.2 Methodische Abgrenzung und Reduktion

Um den Umfang der Experimente auf ein realisierbares Maß zu reduzieren, werden folgende Komponenten begründet ausgeschlossen:

- **Ausschluss des SKF:** Der Standard KF setzt ein lineares System- und Messmodell voraus [8]. Da die perspektivische Projektion der Kamera sowie die Roboterkinematik inhärent nichtlinear sind, führt die notwendige Linearisierung bei starken Bewegungen zu signifikanten Fehlern in der Zustandsschätzung [5]. Während der SKF für lineare Probleme optimal ist, ist er für das präzise Feature-Tracking im Bildraum ungeeignet und wird daher durch seine nichtlinearen Erweiterungen (EKF/UKF/ESKF) ersetzt, welche die Nichtlinearitäten durch lokale Linearisierung, die Unscented Transform handhaben oder Fehlerzustands-Filterung ermöglichen [13].
- **Ausschluss des 1€-Filters:** Dieser Filter fungiert als adaptiver Tiefpass auf Signalebene [16]. Da er ausschließlich auf den zeitlichen Ableitungen des Eingangssignals operiert und über kein internes physikalisches Modell des Prozesses verfügt, ist er prinzipbedingt nicht in der Lage, während einer vollständigen Okklusion (Blindphasen) eine Prädiktion der Feature-Trajektorie zu liefern. Für die Aufrechterhaltung der Stabilität des IBVS-Regelkreises in dynamischen Szenarien ist eine modellbasierte Schätzung jedoch essenziell, weshalb dieser Filtertyp für die untersuchten Extrembedingungen nicht berücksichtigt wird [16].

[2] Der Hintergrund des Bildes wurde zur besseren Visualisierung mit KI-Werkzeugen freigestellt und bearbeitet.

11.3 Design of Experiments (DoE)

Basierend auf der Abgrenzung werden 7 System-Konfigurationen definiert. Die in Tabelle 11.2 dargestellte erste Versuchsmatrix bezieht sich zunächst ausschließlich auf Level a. Innerhalb dieses Levels beschränkt sich die Relativbewegung zwischen Start- und Zielpose auf Translationen in x, y und z sowie eine Rotation um die z-Achse. Um die Robustheit gegenüber unterschiedlichen Oberflächenbeschaffenheiten zu validieren, wird jede dieser Testreihen für die in Abschnitt 11.1 definierten Bauteil-Typen Typ 1 sowie Typ 2 durchgeführt.

Tabelle 11.1: Definierte System-Konfigurationen für das Benchmarking.

ID	Konfiguration	Beschreibung / Fokus
C_1	Baseline	Reine Keypoint-Detektion (Referenzwert)
C_2	EKF	Zustandsschätzung mittels lokaler Linearisierung
C_3	UKF	Filterung mittels Unscented Transform
C_4	ESKF	Fehlerzustands-Filterung für Orientierungen
C_5	EKF + LR	Kombination aus EKF und Local-Rescue-Logik (zu Vergleich)
C_6	UKF + LR	Kombination aus UKF und Local-Rescue-Logik (zu Vergleich)
C_7	ESKF + LR	Kombination aus ESKF und Local-Rescue-Logik (zu Vergleich)

Um die Anzahl der Testkombinationen zu reduzieren, wird bei Bauteil Typ 3 innerhalb von Level a ausschließlich der ESKF getestet (siehe Tabelle 11.2). Die Wahl des ESKF basiert auf den Ergebnissen der Messungen bei den Bauteil-Typen 1 und 2 und wird in Abschnitt 12 näher begründet.

Tabelle 11.2: Matrix-Übersicht der Test-Kombinationen für Level a nach Konfiguration.

Bauteil	Szenario	C_1	C_2	C_3	C_4	C_5	C_6	C_7	Summe
Typ 1	I	✓	✓	✓	✓	✓	✓	✓	7
	II	✓	✓	✓	✓	✓	✓	✓	7
	III	✓	✓	✓	✓	✓	✓	✓	7
Typ 2	I	✓	✓	✓	✓	✓	✓	✓	7
	II	✓	✓	✓	✓	✓	✓	✓	7
	III	✓	✓	✓	✓	✓	✓	✓	7
Typ 3	I	✓	-	-	✓	-	-	✓	3
	II	✓	-	-	✓	-	-	✓	3
	III	✓	-	-	✓	-	-	✓	3
Gesamt		9	6	6	9	6	6	9	**51**

Dies ergibt insgesamt 51 experimentelle Bedingungen für Level a, die eine systematische Isolierung der Einflussfaktoren Szenario, Materialeigenschaft und Konfiguration ermöglichen.

Die Level-b-Testreihe wird mit reduziertem Umfang ausschließlich für Bauteil Typ 2 durchgeführt, um die zusätzlichen Rotationsfreiheitsgrade α und β des IBVS-Controllers zu validieren. Typ 2 bietet durch seine Kombination aus ausgeprägter Texturierung und spiegelnden Oberflächeneigenschaften einen praxisnahen, anspruchsvollen Testfall. Die Evaluierung beschränkt sich auf Szenario I und II sowie die Konfigurationen C_1, C_4 und C_7. Diese Auswahl wird durch die Level-a-Ergebnisse begründet und in Abschnitt 12 detailliert dargelegt.

Tabelle 11.3: Matrix-Übersicht der Test-Kombinationen für Level b nach Konfiguration.

Bauteil	Szenario	C_1	C_2	C_3	C_4	C_5	C_6	C_7	Summe
Typ 2	**I**	✓	-	-	✓	-	-	✓	3
	II	✓	-	-	✓	-	-	✓	3
Gesamt		2	0	0	2	0	0	2	**6**

Damit ergeben sich für Level b weitere 6 experimentelle Bedingungen.

11.4 Versuchsplan und zeitliche Abschätzung

Um den experimentellen Gesamtaufwand in einem realisierbaren Rahmen zu halten, wird die Stichprobengröße pro Kombination auf $N = 5$ festgesetzt. Für die in Tabelle 11.2 dargestellten 51 Kombinationen in Level a ergibt sich daraus die folgende Zeitplanung. Da die Varianz der Messergebnisse hier über drei verschiedene physikalische Bauteile hinweg beobachtet wird, ist diese Reduktion der Wiederholungen pro Einzelkombination methodisch zulässig.

Tabelle 11.4 zeigt die resultierende Zeitplanung für Level a. Mit 255 Einzelläufen bietet dieser Plan eine breite Datenbasis und kann auf ein bis zwei Arbeitstage aufgeteilt werden.

Tabelle 11.4: Zeitliche Abschätzung für 51 Kombinationen in Level a.

Stichprobe (N)	Anzahl Läufe (Gesamt)	Netto-Zeit (1,5 min/Lauf)	Brutto-Zeit (Faktor 1,5)
3	153	229,5 min	5,735 h
5	**255**	**382,5 min**	**9,5625 h**
10	510	765 min	19,125 h

Für die ergänzende Level-b-Testreihe mit 6 Kombinationen ergibt sich die in Tabelle 11.5 dargestellte zusätzliche Zeitplanung.

Tabelle 11.5: Zeitliche Abschätzung für 6 Kombinationen in Level b.

Stichprobe (N)	**Anzahl Läufe (Gesamt)**	**Netto-Zeit (1,5 min/Lauf)**	**Brutto-Zeit (Faktor 1,5)**
3	18	27 min	0,675 h
5	**30**	**45 min**	**1,125 h**
10	60	90 min	2,25 h

Hinsichtlich des Versuchsablaufs wird sowohl für die 51 Kombinationen in Level a als auch für die 6 Kombinationen in Level b zu Beginn jeder Kombination eine neue Initialisierung durchgeführt, bei der ein dedizierter Referenzdatensatz aufgenommen wird. Diese Referenz bleibt für alle fünf zugehörigen Testläufe ($N = 5$) unverändert. Dieses Vorgehen wird aus zwei Gründen gewählt:

1. **Isolierung von Störeinflüssen:** Durch die konstante Referenz innerhalb einer Testreihe wird sichergestellt, dass die gemessene Varianz in den Evaluierungsmetriken primär auf die dynamischen Störungen der Szenarien (z. B. Okklusionen oder Lichtwechsel) zurückzuführen ist. Würde für jeden Lauf eine neue Initialisierung erfolgen, könnten geringfügige Abweichungen im Referenzmodell (z. B. durch Sensorrauschen während der Aufnahme) die Ergebnisse überlagern und die Vergleichbarkeit einschränken.
2. **Praxisnähe:** Das Vorgehen bildet einen typischen industriellen Prozess ab, bei dem ein System einmalig für eine Aufgabe initialisiert wird, um anschließend eine Serie von Arbeitszyklen unter schwankenden Umgebungsbedingungen stabil auszuführen.

Ergänzend zu den systematischen Benchmarkreihen in Level a und Level b wird eine zusätzliche Detailanalyse auf Basis von Einzelrun-Daten durchgeführt. Diese Analyse diente der isolierten Untersuchung des Einflusses der Local-Rescue-Logik unter gezielt eingebrachten Lichtwechseln. Zu diesem Zweck wird bewusst eine rein translatorische Bewegung ohne zusätzliche Rotationsanteile gewählt, um den Einflussfaktor zu isolieren.

11.5 Evaluierungsmetriken

Die Bewertung der Leistungsfähigkeit der verschiedenen Filter- und Entscheidungslogik-Konfigurationen erfolgt unter anderem anhand folgender Metriken, die sowohl die Regelungsgenauigkeit als auch die Steuerungsgüte des IBVS-Systems quantifizieren.

- **Regelungsgenauigkeit (IBVS-Error):** Quantifizierung des quadratischen Fehlers im Bildraum, um die Präzision der Zielverfolgung zu bewerten. Der RMS-Fehler wird berechnet als:

$$E_{rms} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(s_i^* - s_i)^2} \tag{11.1}$$

  Die Auswertung fokussiert auf den Fehlerverlauf über der Zeit sowie das Konvergenzverhalten, um die Stabilisierung der Regelung zu beurteilen.

- **Steuerungsgüte (Robot Effort):** Diese Metrik bewertet die Stabilität und „Glattheit“ der berechneten Stellgrößen. Um den mechanischen Bias des Roboters (wie interne Filterung oder Trägheit) zu eliminieren, wird direkt der Output des IBVS-Controllers — ein 6D-Twist-Vektor bestehend aus translatorischen $(\dot{x}, \dot{y}, \dot{z})$ und rotatorischen Geschwindigkeiten $(\dot{\alpha}, \dot{\beta}, \dot{\gamma})$ — als Messgröße herangezogen.

  Die translatorischen und rotatorischen Komponenten werden getrennt betrachtet und mittels der Euklidischen Norm (L_2-Norm) zu richtungsunabhängigen Beträgen zusammengefasst:

$$|v|_{trans} = \sqrt{\dot{x}^2 + \dot{y}^2 + \dot{z}^2} \quad [m/s] \quad \text{und} \tag{11.2}$$

$$|\omega|_{rot} = \sqrt{\dot{\alpha}^2 + \dot{\beta}^2 + \dot{\gamma}^2} \quad [rad/s] \tag{11.3}$$

  Die Analyse betrachtet die zeitlichen Verläufe dieser Geschwindigkeitsgrößen sowie die daraus abgeleiteten Beschleunigungskomponenten, um die Dynamik und Reproduzierbarkeit der Regelung zu untersuchen.

  Ergänzend wird die effektive Pfadlänge durch Integration der kommandierten Geschwindigkeiten über die Versuchszeit bestimmt. Sie dient als Maß für die Effizienz und Direktheit der Bewegung: Ein effizientes System zeichnet sich durch eine Pfadlänge aus, die nahe an der theoretischen Ideallinie liegt, während ein unruhiges System durch Schwingungen und Richtungswechsel eine deutlich erhöhte Pfadlänge aufweist.

12 Ergebnisse

12.1 Datenbasierte Reduktion des Versuchsraums

12.1.1 Filterauswahl auf Basis von Level a

Die Auswahl des für die Folgeexperimente verwendeten Filters erfolgt auf Basis der Level-a-Ergebnisse unter Okklusion. Abbildung 12.1 zeigt für Bauteil Typ 2 den zeitlichen Verlauf der medianen translatorischen Geschwindigkeit sowie des medianen IBVS-Fehlers im Bildraum. Ziel dieser Auswertung ist es, diejenigen Filterkonfigurationen zu identifizieren, welche den Regelkreis unter temporärem Featureverlust robust stabilisieren.

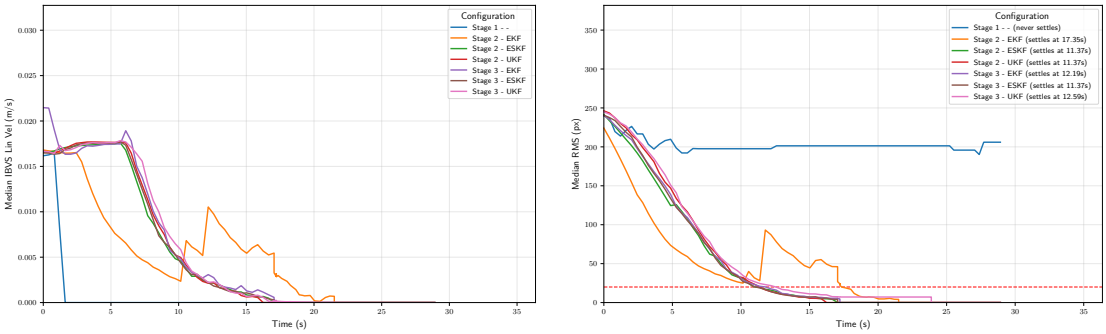
In Abbildung 12.1a ist die mediane translatorische Geschwindigkeit über der Zeit dargestellt. Die Baseline ohne Filter fällt bereits sehr früh auf eine Geschwindigkeit von nahezu null ab und verbleibt dort über den weiteren Verlauf. Dies zeigt, dass das System bei Okklusion ohne modellbasierte Zustandsfortschreibung keine stabile Zielannäherung mehr aufrechterhalten kann. Der EKF weist im Vergleich dazu einen deutlich unruhigeren Verlauf mit ausgeprägten Zwischenschwankungen auf. Demgegenüber zeigen ESKF und UKF einen wesentlich glatteren und nahezu deckungsgleichen Verlauf.

Dieses Bild wird durch Abbildung 12.1b bestätigt, in der der mediane IBVS-RMS-Fehler dargestellt ist. Die Baseline erreicht die Konvergenzschwelle nicht und verbleibt auf hohem Fehlerniveau. Der EKF konvergiert zwar grundsätzlich, zeigt jedoch erneut einen deutlich weniger monotonen Verlauf und erreicht die Konvergenz später als ESKF und UKF. Die beiden letztgenannten Filter verhalten sich dagegen sehr ähnlich und erreichen die Fehlergrenze deutlich früher. Aus diesen Ergebnissen ergibt sich zunächst, dass für die weiteren Untersuchungen ein nichtlinearer modellbasierter Filter erforderlich ist und der EKF den beiden Alternativen ESKF und UKF unter Okklusion unterlegen ist.

Da sich ESKF und UKF in den gezeigten Qualitätsmetriken sehr ähnlich verhalten, erfolgt die weitere Reduktion anhand des Implementierungs- und Laufzeitaufwands. Der UKF basiert auf der Propagation mehrerer Sigma-Punkte durch das nichtlineare Modell und ist damit in der praktischen Ausführung rechenaufwändiger als EKF- bzw. ESKF-basierte Ansätze [13, 44]. Vor dem Hintergrund der sehr ähnlichen Ergebnisqualität wird daher für die weiteren Experimente der ESKF als bevorzugter Filter ausgewählt.

Unter Idealbedingungen zeigt sich jedoch zugleich, dass der Einsatz eines Filters nicht in jeder Metrik automatisch zu einem Vorteil führt. Abbildung 12.2 zeigt für Bauteil Typ 1 die mediane Pfadlänge über der Zeit. Hier weist die Baseline ohne Filter den geringsten Wert auf. Dies deutet darauf hin, dass bei ungestörter Merkmalsextraktion die direkte Nutzung der ungefilterten Keypoints eine präzisere Feinannäherung erlaubt als eine auf eine reduzierte aktive Punktmenge gestützte Filterrepräsentati-

on. Für die weitere Interpretation ist daher zu berücksichtigen, dass der Vorteil der Filterung insbesondere unter Störeinflüssen sichtbar wird, während die Baseline unter Idealbedingungen weiterhin einen präzisen Referenzfall darstellt.



(a) Mediane translatorische Geschwindigkeit (b) Medianer IBVS-RMS-Fehler

Abbildung 12.1: Level: a | Szen.: 2 | Bauteil: Typ 2 | Vergleich der Filterkonfigurationen unter Okklusion

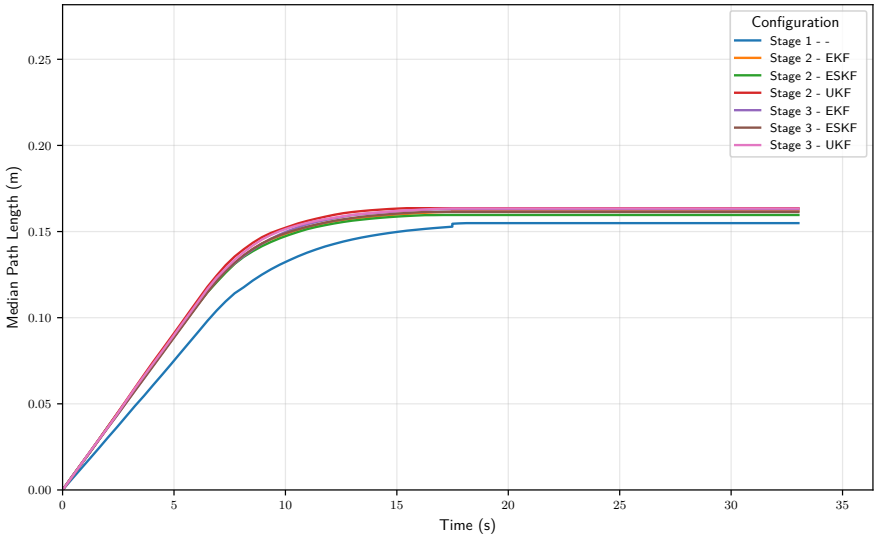


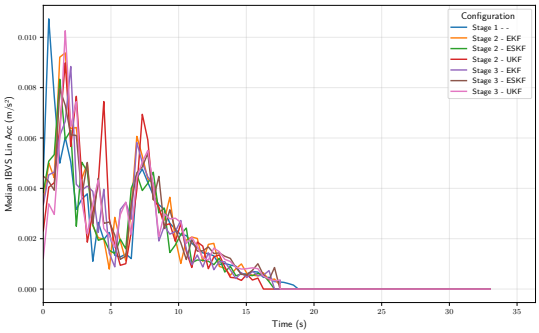
Abbildung 12.2: Level: a | Szen.: 1 | Bauteil: Typ 1 | Mediane Pfadlänge über der Zeit

12.1.2 Bauteilauswahl für die Folgeexperimente

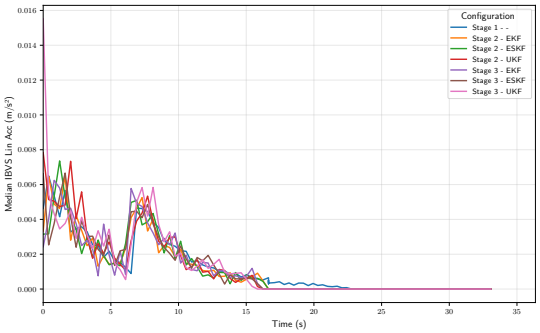
Die Auswahl des Bauteils für die vertiefenden Untersuchungen erfolgt in einem ersten Schritt unter Idealbedingungen. Hierfür wird geprüft, ob Baseline und Filter bereits ohne äußere Störung ein vergleichbares dynamisches Verhalten aufweisen. Nur dann lassen sich spätere Unterschiede unter Okklusion oder Lichtwechsel primär auf die eigentlichen Störeinflüsse und nicht bereits auf einen materialspezifischen Grundbias zurückführen.

Abbildung 12.3 zeigt für die drei Bauteiltypen unter Szenario I die mediane translatorische Beschleunigung über der Zeit. Für Typ 1 und Typ 2 liegen die Verläufe der Baseline und der Filterkonfigurationen vergleichsweise nahe beieinander. Beide Bauteile eignen sich daher grundsätzlich als Ausgangspunkt für einen belastbaren Vergleich zwischen

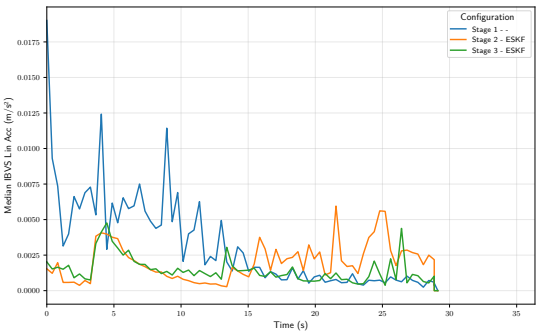
ungefilterter und gefilterter Regelung. Für Typ 3 zeigt sich dagegen bereits unter Idealbedingungen eine deutlich stärkere Abweichung zwischen Baseline und Filterverlauf. Insbesondere die Baseline weist hier zu Beginn deutlich höhere Beschleunigungsspitzen und insgesamt ein unruhigeres Verhalten auf. Dies deutet darauf hin, dass bei diesem Bauteil bereits die reine Merkmalsextraktion deutlich instabiler ist und damit ein starker materialspezifischer Bias vorliegt. Typ 3 wurde daher für die weiterführenden Untersuchungen nicht als primäres Vergleichsbauteil verwendet.



(a) Bauteil: Typ 1



(b) Bauteil: Typ 2

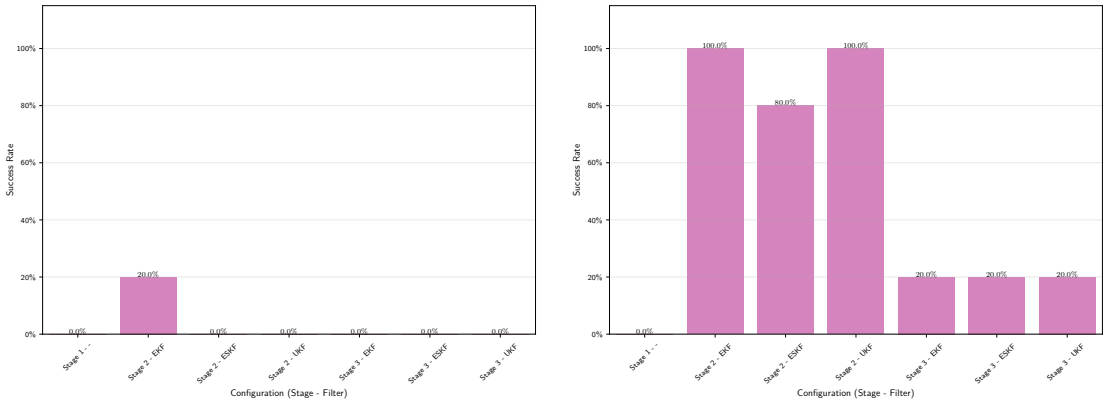


(c) Bauteil: Typ 3

Abbildung 12.3: Level: a | Szen.: 1 | Mediane translatorische Beschleunigung zur Bauteilauswahl

Zwischen Typ 1 und Typ 2 erfolgt die endgültige Auswahl anhand der Robustheit unter Lichtwechsel. Abbildung 12.4 zeigt die Success Rate beider Bauteile in Szenario III. Für Typ 1 wird unter Lichtwechsel nahezu keine erfolgreiche Konvergenz erreicht. Für Typ 2 zeigen die Stage-2-Konfigurationen dagegen deutlich höhere Erfolgsraten. Insbesondere Stage 2 mit EKF und UKF erreicht jeweils eine Success Rate von 1,0, während Stage 2 mit ESKF bei 0,8 liegt. Gleichzeitig fällt auf, dass die Stage-3-Konfigurationen mit LR in diesem Szenario jeweils nur eine Success Rate von 0,2 erreichen und damit keinen zusätzlichen Vorteil gegenüber Stage 2 liefern. Dies deutet bereits an, dass LR unter Lichtwechsel nicht automatisch zu einer Verbesserung führt.

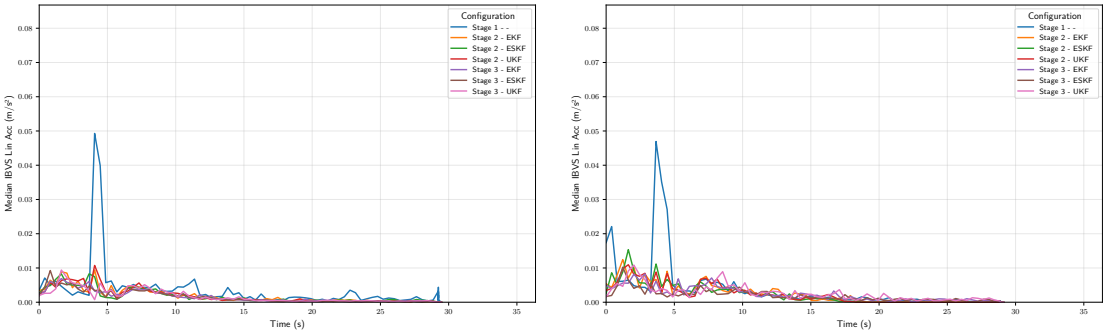
Eine zusätzliche Einordnung des Filterverhaltens unter Lichtwechsel liefert Abbildung 12.5. Dargestellt ist die mediane translatorische Beschleunigung für Typ 1 und Typ 2 in Szenario III. In beiden Fällen zeigt die Baseline ohne Filter nach dem Zuschalten der Beleuchtung einen ausgeprägten Peak, während die gefilterten Verläufe deutlich glatter bleiben. Dies unterstreicht den stabilisierenden Effekt der modellbasierten Schätzung auch unter photometrischen Störungen. Gleichzeitig ist in diesen aggregierten Verläufen kein klarer und konsistenter Vorteil der LR-Erweiterung gegenüber dem reinen Filterbetrieb erkennbar. Diese Beobachtung motiviert die im folgenden



(a) Bauteil: Typ 1 (b) Bauteil: Typ 2

Abbildung 12.4: Level: a | Szen.: 3 | Success Rate zur Bauteilauswahl unter Lichtwechsel

Abschnitt dargestellte Zusatzanalyse auf Einzelrun-Ebene.



(a) Bauteil: Typ 1 (b) Bauteil: Typ 2

Abbildung 12.5: Level: a | Szen.: 3 | Mediane translatorische Beschleunigung unter Lichtwechsel

Auf Basis dieser Ergebnisse wird Typ 2 als zentrales Bauteil für die weiteren Experimente ausgewählt. Das Bauteil besitzt einerseits ausreichend Textur für stabile Merkmalsextraktion, weist andererseits jedoch spiegelnde Eigenschaften auf und bildet damit einen praxisnahen und zugleich anspruchsvollen Versuchsfall. Die weiteren Untersuchungen in Level b sowie in der ergänzenden LR-Analyse werden daher auf Basis dieses Bauteils durchgeführt.

12.2 Zusatzanalyse des Local Rescue unter translatorischer Bewegung

Diese Detailanalyse der Local-Rescue-Mechanik wurde auf Grundlage von Einzelrundaten durchgeführt. Während die Level-a- und Level-b-Auswertungen primär aggregierte Metriken (Mittelwerte, Mediane) herangezogen haben, erfolgt hier eine laufspezifische Betrachtung. Dazu wurde eine rein translatorische Bewegung zwischen Start- und Zielpose gewählt. Während der Fahrt wurde nach 2 s für 1 s eine zusätzliche Beleuchtung eingeschaltet, anschließend für 1 s wieder ausgeschaltet und nach einer weiteren

Sekunde erneut für 1 s eingeschaltet. Auf diese Weise konnten gezielt kurzzeitige photometrische Störungen eingebracht werden, ohne zugleich weitere Freiheitsgrade als Störgröße zu überlagern.

Im Gegensatz zu den zuvor betrachteten Median- und Vergleichsdarstellungen werden in diesem Abschnitt bewusst Einzelruns ausgewertet. Dadurch bleiben run-spezifische Ausreißer und temporäre Störungen sichtbar, die in aggregierten Darstellungen nur abgeschwächt oder gar nicht mehr erkennbar wären.

12.2.1 Vergleich von Stage 1, Stage 2 und Stage 3 in der Stellgröße

Abbildung 12.6 zeigt für die drei Stufen die Verläufe der translatorischen Geschwindigkeit und der translatorischen Beschleunigung in allen fünf Einzelruns. In Stage 1 ohne Filter sind insbesondere zu Beginn der Bewegung sowie im Bereich der Beleuchtungswechsel deutliche run-spezifische Ausschläge sichtbar. Dies betrifft sowohl die Geschwindigkeit als auch die Beschleunigung und weist auf eine erhöhte Empfindlichkeit gegenüber kurzzeitigen Störungen in der Merkmalsextraktion hin.

In Stage 2 mit ESKF zeigt sich demgegenüber eine deutlich gleichmäßigere Dynamik. Die fünf Einzelruns verlaufen in der translatorischen Geschwindigkeit nahezu deckungsgleich und auch die Beschleunigung bleibt über den gesamten Verlauf auf einem wesentlich ruhigeren Niveau. Der Filter reduziert damit nicht nur die sichtbaren Ausreißer, sondern erhöht zugleich die Reproduzierbarkeit zwischen den einzelnen Durchläufen.

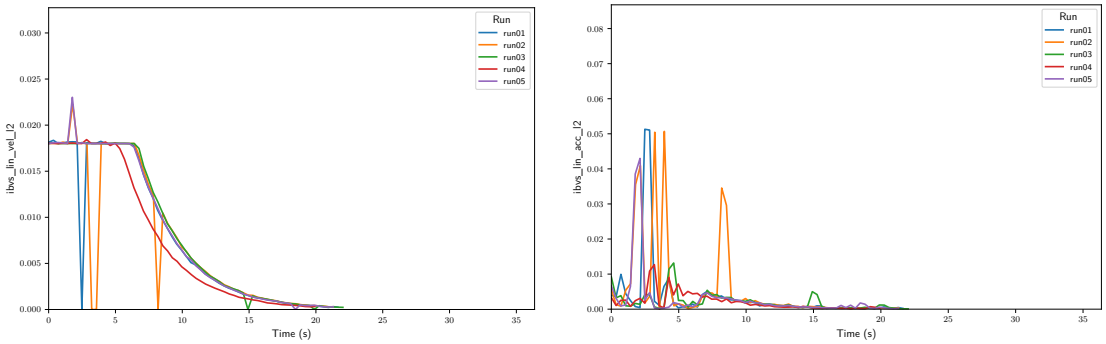
Für Stage 3 mit ESKF+LR ergibt sich kein zusätzlicher Stabilitätsgewinn. Zwar bleibt das System grundsätzlich regelbar, jedoch treten sowohl in der Geschwindigkeit als auch in der Beschleunigung erneut stärkere run-spezifische Ausschläge auf. Im Vergleich zu Stage 2 wird damit zusätzliches Rauschen in die Stellgröße eingetragen. Die LR-Erweiterung wirkt in diesem Szenario daher nicht stabilisierend, sondern erhöht vielmehr die Variabilität zwischen den Einzelruns.

12.2.2 Vergleich des Fehlerverlaufs auf Einzelrun-Ebene

Abbildung 12.7 zeigt ergänzend den IBVS-RMS-Fehler aller fünf Einzelruns für Stage 1 bis Stage 3. Auch hier wird der Nutzen des Filters gegenüber der ungefilterten Baseline sichtbar. In Stage 1 sind insbesondere in der frühen Phase der Bewegung stärkere run-spezifische Schwankungen des Fehlers erkennbar. Die fünf Verläufe konvergieren zwar grundsätzlich, zeigen jedoch ein sichtbares Zittern im Bereich der initialen Annäherung.

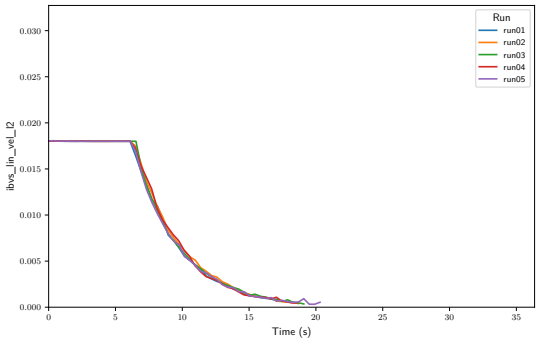
Stage 2 mit ESKF liefert den gleichmäßigsten Fehlerverlauf. Die fünf Kurven liegen nahezu übereinander und fallen über die Zeit sehr ähnlich ab. Dies bestätigt, dass der Filter nicht nur die Stellgröße glättet, sondern auch den eigentlichen Regelungsfehler reproduzierbar stabilisiert. Auffällig ist zudem, dass die Konvergenzzeiten in Stage 2 sehr eng beieinander liegen.

In Stage 3 mit ESKF+LR steigt die Streuung zwischen den fünf Einzelruns wieder an. Die Verläufe sind weniger deckungsgleich als in Stage 2 und zeigen zusätzliche lokale Abweichungen. Auch wenn einzelne Runs vergleichbare oder sogar etwas frühere Konvergenzzeiten erreichen, ist das Gesamtbild weniger konsistent. Für die vorliegende

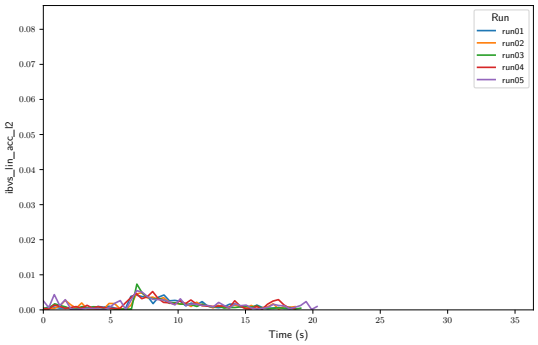


(a) Stage 1: LIN VEL L2

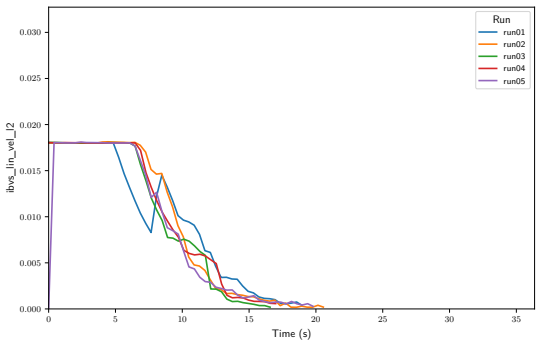
(b) Stage 1: LIN ACC L2



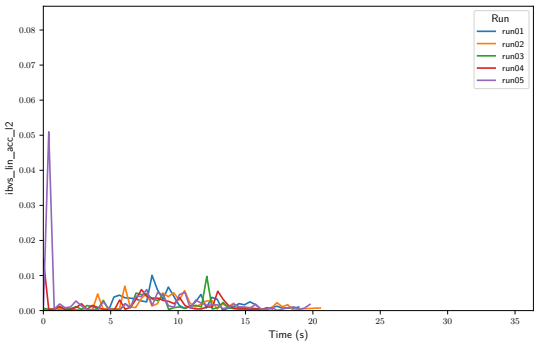
(c) Stage 2: LIN VEL L2



(d) Stage 2: LIN ACC L2



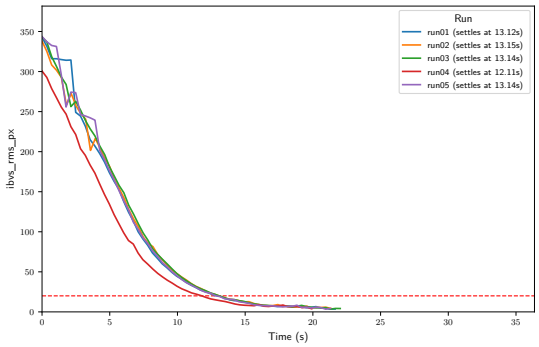
(e) Stage 3 (ESKF + LR): LIN VEL L2



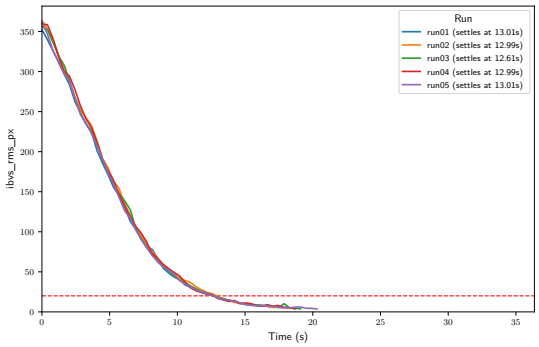
(f) Stage 3 (ESKF + LR): LIN ACC L2

Abbildung 12.6: Ergänzende Detailanalyse | Translatorische Bewegung | Lichtwechsel | Bauteil: Typ 2 | Einzelrun-Vergleich der translatorischen Geschwindigkeit und Beschleunigung

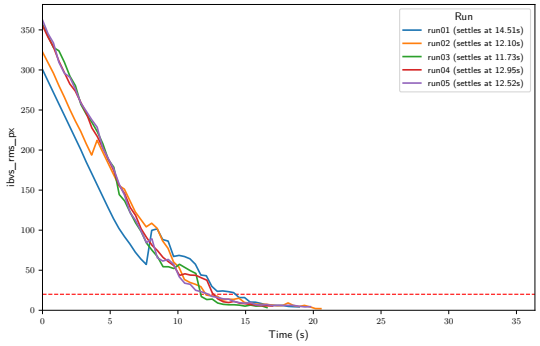
translatorische Lichtwechselanalyse ergibt sich damit kein belastbarer Zusatznutzen der LR-Erweiterung. Vielmehr zeigt sich, dass der ESKF ohne LR in diesem Szenario die stabilste und reproduzierbarste Lösung darstellt.



(a) Stage 1: RMS PX



(b) Stage 2: RMS PX



(c) Stage 3 (ESKF + LR): RMS PX

Abbildung 12.7: Ergänzende Detailanalyse | Translatorische Bewegung | Lichtwechsel | Bauteil: Typ 2 | Einzelrun-Vergleich des IBVS-RMS-Fehlers

Die Ergebnisse dieser Zusatzanalyse legen nahe, dass zusätzliche Updateschritte nicht automatisch zu einer besseren Schätzung führen. Vielmehr scheint LR im vorliegenden Setup zusätzliche, aber nicht hinreichend verlässliche Reassoziationen in den Filter einzubringen. Unter Lichtwechseln können diese lokalen Wiederzuordnungen photometrisch oder geometrisch inkonsistent sein und dadurch verrauschte Innovationsschritte erzeugen. In der Folge wird die ansonsten stabile modellbasierte Vorhersage nicht verbessert, sondern teilweise durch zusätzliche Korrektursprünge gestört.

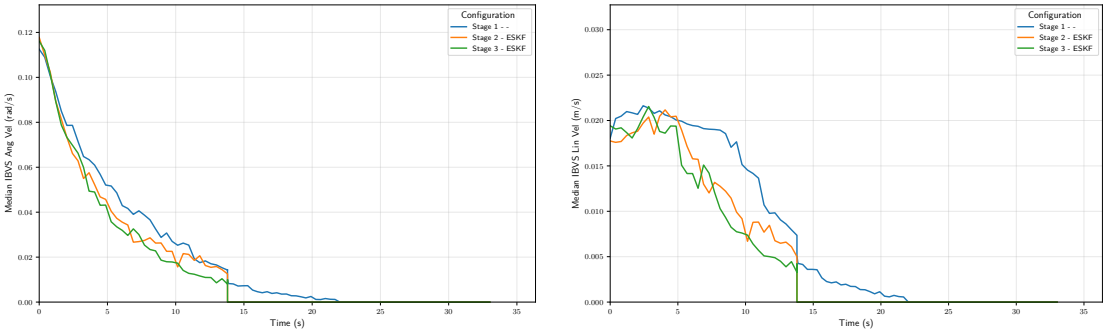
12.3 Rotationserweiterung auf Level b

Nach der datenbasierten Auswahl von Filter und Bauteil wird im nächsten Schritt die Erweiterung auf Level b betrachtet. Ziel dieser Versuchsreihe ist es zu zeigen, dass die Regelung nicht nur für translatorische Bewegungen und eine Rotation um die z-Achse, sondern auch für zusätzliche Rotationen um die x- und y-Achse eingesetzt werden kann. Die hierfür verwendete Tiefeninformation der einzelnen Keypoints beziehungsweise Modellpunkte erlaubt es, die zusätzliche Rotationsdynamik in der Schätzung und Regelung zu berücksichtigen.

12.3.1 Referenzverhalten unter Idealbedingungen

Abbildung 12.8 zeigt für Level b unter Standardbedingungen den medianen Verlauf der translatorischen sowie der rotatorischen Geschwindigkeit für Bauteil Typ 2. In Abbildung 12.8a ist die mediane rotatorische Geschwindigkeit dargestellt. Hier verlaufen Baseline, Filter und Filter mit LR über weite Strecken sehr ähnlich. Dies zeigt, dass die Erweiterung auf zusätzliche Rotationsfreiheitsgrade unter Idealbedingungen grundsätzlich stabil beherrscht wird und die reine Zuschaltung der Rotationsanteile nicht zu einer unmittelbaren Destabilisierung des Systems führt.

In Abbildung 12.8b ist die mediane translatorische Geschwindigkeit dargestellt. Hier zeigt die Baseline ohne Filter im Mittel einen etwas höheren Verlauf als die gefilterten Varianten. Dies ist konsistent mit den bereits in Level a beobachteten Ergebnissen, wonach die ungefilterte Regelung unter Idealbedingungen in der Feinannäherung teilweise direkter reagiert. Für die Einordnung der folgenden Okklusionsversuche ist daher wichtig, dass sich die Baseline unter ungestörten Bedingungen zunächst unauffällig verhält und die Unterschiede zwischen den Konfigurationen hier noch vergleichsweise gering ausfallen.



(a) Mediane rotatorische Geschwindigkeit (b) Mediane translatorische Geschwindigkeit

Abbildung 12.8: Level: b | Szen.: 1 | Bauteil: Typ 2 | Referenzverhalten unter Standardbedingungen

12.3.2 Verhalten unter Okklusion

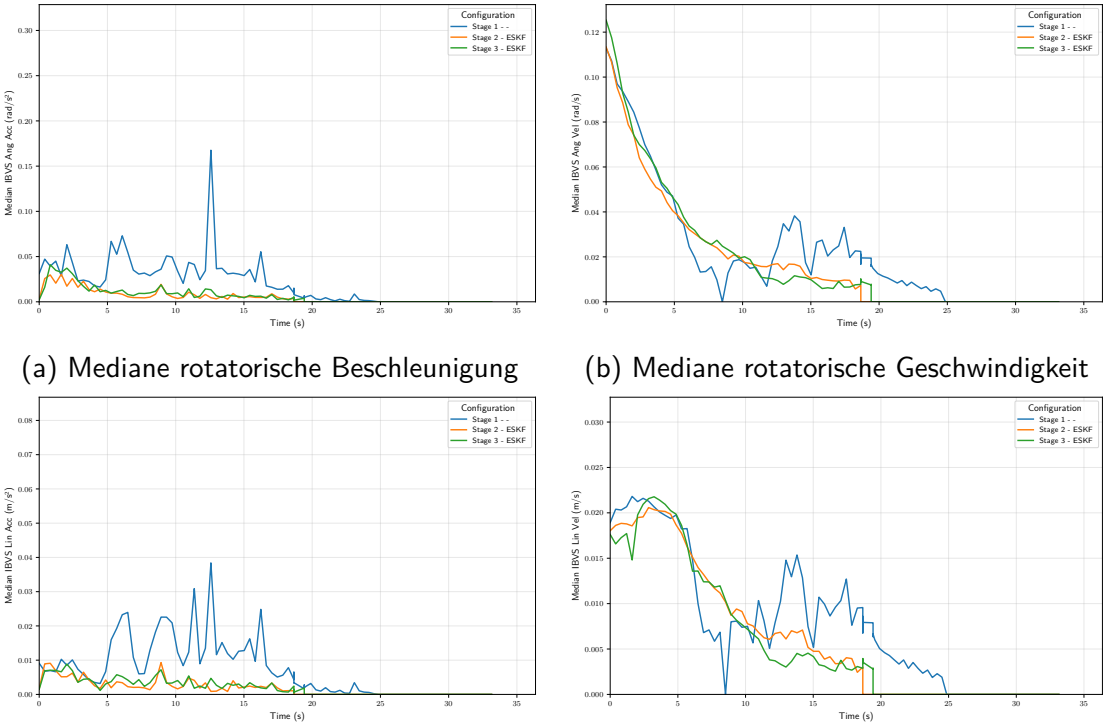
Die eigentliche Aussagekraft der Level-b-Versuche ergibt sich unter Okklusion. Abbildung 12.9 zeigt hierzu für Bauteil Typ 2 den medianen Verlauf der rotatorischen und translatorischen Geschwindigkeits- und Beschleunigungsgrößen.

In Abbildung 12.9b und Abbildung 12.9a sind die rotatorische Geschwindigkeit und Beschleunigung dargestellt. Die Baseline ohne Filter weist hier deutlich stärkere Sprünge und abruptere Änderungen auf. Demgegenüber verlaufen die gefilterten Kurven deutlich glatter. Dies zeigt, dass die modellbasierte Schätzung die zusätzlichen Rotationsanteile auch unter temporärem Merkmalsverlust stabilisiert und die Regelgröße im Winkelraum wirksam beruhigt.

Das gleiche Bild ergibt sich für die translatorischen Größen. In Abbildung 12.9d und Abbildung 12.9c zeigt die Baseline ohne Filter ein deutlich ruckligeres Verhalten mit ausgeprägten Beschleunigungs- und Abbremsphasen. Der ESKF führt hingegen zu einem wesentlich gleichmäßigeren Verlauf. Dadurch wird sichtbar, dass der Filter unter

Okklusion nicht nur einzelne Ausreißer reduziert, sondern die gesamte Trajektorienführung sowohl im translatorischen als auch im rotatorischen Anteil stabilisiert.

Auch in Level b zeigt die LR-Erweiterung keinen zusätzlichen Vorteil gegenüber dem reinen Filterbetrieb. Die Kurven von Stage 3 (ESKF + LR) bleiben zwar deutlich ruhiger als die Baseline, liegen jedoch meist oberhalb oder nahe bei den entsprechenden Stage-2-Verläufen. Damit bestätigt sich die bereits in den vorherigen Auswertungen beobachtete Tendenz, dass LR im vorliegenden Setup keinen konsistenten Zusatznutzen liefert.



(a) Mediane rotatorische Beschleunigung

(b) Mediane rotatorische Geschwindigkeit

(c) Mediane translatorische Beschleunigung

(d) Mediane translatorische Geschwindigkeit

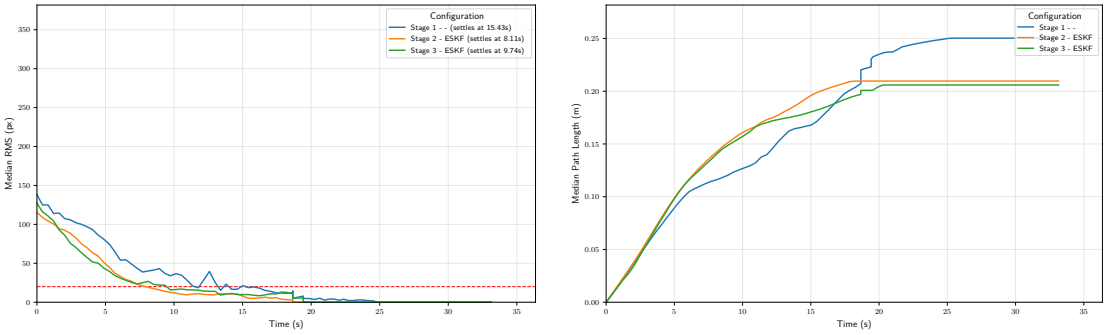
Abbildung 12.9: Level: b | Szen.: 2 | Bauteil: Typ 2 | Medianer Verlauf der Geschwindigkeits- und Beschleunigungsgrößen unter Okklusion

12.3.3 Fehlerverlauf und Pfadlänge

Abbildung 12.10 fasst die Auswirkungen der Okklusion auf den eigentlichen Regelungsfehler sowie auf die resultierende Pfadlänge zusammen. In Abbildung 12.10a ist der mediane IBVS-RMS-Fehler dargestellt. Hier zeigt sich ein deutlicher Vorteil des Filters: Während die Baseline die Konvergenzschwelle erst nach 15.43 s erreicht, liegt die Einschwingzeit bei Stage 2 mit ESKF bei 8.11 s und bei Stage 3 mit ESKF+LR bei 9.74 s. Der Filter reduziert damit nicht nur das sichtbare Zittern des Fehlerverlaufs, sondern beschleunigt die Konvergenz unter Okklusion deutlich. Gleichzeitig bestätigt auch diese Metrik, dass die zusätzliche LR-Erweiterung keinen Vorteil gegenüber dem reinen ESKF liefert.

Abbildung 12.10b zeigt ergänzend die mediane Pfadlänge über der Zeit. Hier ist erkennbar, dass die gefilterten Varianten eine kürzere und damit effizientere Trajektorie aufweisen als die Baseline. Die ungefilterte Regelung erzeugt durch die stärkeren

Schwankungen und Richtungsänderungen eine längere Pfadlänge, während der ESKF eine kompaktere und zielgerichtetere Bewegung ermöglicht. Auch in dieser Metrik zeigt Stage 2 das günstigste Verhalten.



(a) Medianer IBVS-RMS-Fehler

(b) Mediane Pfadlänge

Abbildung 12.10: Level: b | Szen.: 2 | Bauteil: Typ 2 | Fehlerverlauf und Pfadlänge unter Okklusion

12.4 Zusammenfassung der Ergebnisse

Die Ergebnisse zeigen insgesamt, dass die modellbasierte Filterung den entscheidenden Beitrag zur Stabilisierung des IBVS-Regelkreises leistet. Bereits in Level a wurde deutlich, dass der EKF unter Okklusion den nichtlinearen Alternativen ESKF und UKF unterlegen ist, während sich ESKF und UKF hinsichtlich der betrachteten Metriken sehr ähnlich verhalten. Vor dem Hintergrund des geringeren praktischen Aufwands wurde daher der ESKF als bevorzugte Filterlösung für die Folgeexperimente ausgewählt.

Die Bauteilauswahl zeigte zudem, dass Typ 2 den geeignetsten Kompromiss für die weiterführenden Analysen darstellt. Das Bauteil besitzt ausreichend Textur für eine stabile Merkmalsextraktion, weist zugleich jedoch reflektierende Eigenschaften auf und bildet damit einen praxisnahen und zugleich anspruchsvollen Versuchsfall. Typ 3 zeigte dagegen bereits unter Idealbedingungen einen ausgeprägten materialspezifischen Bias und wurde daher nicht als zentrales Vergleichsbauteil für die Vertiefung herangezogen.

Sowohl unter Okklusion als auch unter Lichtwechseln führt der ESKF zu glatteren Stellgrößen, einer stabileren Fehlerabnahme und einer insgesamt robusteren Zielannäherung als die ungefilterte Baseline. Dies zeigte sich zunächst in den aggregierten Level-a-Auswertungen und bestätigte sich anschließend in der Zusatzanalyse auf Einzelrun-Ebene. Der Filter reduziert die sichtbaren Ausreißer in Geschwindigkeit, Beschleunigung und Fehlerverlauf deutlich und erhöht zugleich die Reproduzierbarkeit zwischen den Durchläufen.

Für die LR-Erweiterung ergab sich dagegen in keinem der betrachteten Teile ein belastbarer Zusatznutzen. Weder unter Lichtwechsel noch unter Okklusion konnte Stage 3 (ESKF + LR) konsistent bessere Ergebnisse als Stage 2 (ESKF) erzielen. Vielmehr zeigten sich häufig stärkere run-spezifische Abweichungen, zusätzliche Ausschläge in den Stellgrößen und eine geringere Konsistenz im Fehlerverlauf. Die Ergebnisse legen daher nahe, dass LR im vorliegenden Setup zusätzliche, aber nicht hinreichend verlässliche Reassoziationen in den Filter einbringt. Statt die bereits stabile modellbasierte Vorher-

sage zu verbessern, können diese Updates photometrisch oder geometrisch inkonsistent sein und dadurch verrauschte Innovationsschritte erzeugen.

Ein konsistentes Gesamtbild ergibt sich schließlich auch in der Erweiterung auf Level b. Die Ergebnisse zeigen, dass die Regelung auch bei zusätzlicher Rotation um die x- und y-Achse stabil betrieben werden kann. Insbesondere unter Okklusion führte der ESKF zu glatteren Geschwindigkeits- und Beschleunigungsverläufen, einer kürzeren Pfadlänge sowie zu einer deutlich schnelleren Konvergenz des IBVS-Fehlers als die ungefilterte Baseline. Damit konnte gezeigt werden, dass die modellbasierte Schätzung nicht nur translatorische, sondern auch gekoppelte translatorisch-rotatorische Bewegungen wirksam stabilisiert.

Für das untersuchte System ergibt sich daher insgesamt, dass der reine ESKF ohne LR den besten Kompromiss aus Robustheit, Laufruhe und Reproduzierbarkeit bietet.

13 Fazit und Ausblick

13.1 Zusammenfassung

Ziel dieser Arbeit ist die Entwicklung und experimentelle Untersuchung eines robusteren IBVS-Systems für industrielle Bildbedingungen mit reflektierenden Oberflächen, temporären Okklusionen und Belichtungsänderungen. Hierzu wird ein Ansatz entwickelt, der reale Bildmerkmale mit modellbasierten Vorhersagen koppelt und diese über eine Filter- und Entscheidungslogik für die Regelung nutzbar macht.

Die Ergebnisse zeigen, dass die modellbasierte Filterung den entscheidenden Beitrag zur Stabilisierung des IBVS-Regelkreises leistet. In den Level-a-Versuchen wird deutlich, dass der EKF unter Okklusion den Alternativen ESKF und UKF unterlegen ist, während sich ESKF und UKF in den betrachteten Metriken sehr ähnlich verhalten. Vor dem Hintergrund des geringeren praktischen Aufwands wird daher der ESKF als bevorzugte Filterlösung für die weiteren Experimente ausgewählt.

Die Bauteilauswahl zeigt zudem, dass Typ 2 den geeignetsten Kompromiss für die vertiefenden Analysen darstellt. Das Bauteil besitzt ausreichend Textur für eine stabile Merkmalsextraktion und weist zugleich spiegelnde Eigenschaften auf. Damit bildet Typ 2 einen praxisnahen und zugleich anspruchsvollen Versuchsfall. Typ 3 zeigt unter Idealbedingungen einen ausgeprägten materialspezifischen Bias und wird daher nicht als zentrales Vergleichsbauteil für die Vertiefung herangezogen.

Sowohl unter Okklusion als auch unter Lichtwechseln führt der ESKF zu glatteren Stellgrößen, einer stabileren Fehlerabnahme und einer insgesamt robusteren Zielannäherung als die ungefilterte Baseline. Dies zeigt sich zunächst in den aggregierten Level-a-Auswertungen und bestätigt sich in der ergänzenden Detailanalyse auf Einzelrun-Ebene. Der Filter reduziert die sichtbaren Ausreißer in Geschwindigkeit, Beschleunigung und Fehlerverlauf deutlich und erhöht zugleich die Reproduzierbarkeit zwischen den Durchläufen.

Für die Local Rescue (LR)-Erweiterung ergiebt sich in den durchgeführten Experimenten kein belastbarer Zusatznutzen. Weder unter Lichtwechsel noch unter Okklusion kann die LR-erweiterte Variante konsistent bessere Ergebnisse als der reine ESKF erzielen. Zu erkennen sind häufig stärkere run-spezifische Abweichungen, zusätzliche Ausschläge in den Stellgrößen und eine geringere Konsistenz im Fehlerverlauf. Die Ergebnisse legen daher nahe, dass LR im vorliegenden Setup zusätzliche, aber nicht hinreichend verlässliche Reassoziationen in den Filter einbringt. Statt die bereits stabile modellbasierte Vorhersage zu verbessern, können diese Updates photometrisch oder geometrisch inkonsistent sein und dadurch verrauschte Innovationsschritte erzeugen.

Ein konsistentes Gesamtbild ergibt sich auch in der Erweiterung auf Level b. Die Ergebnisse zeigen, dass die Regelung nicht nur für translatorische Bewegungen und eine Rotation um die z-Achse, sondern auch bei zusätzlichen Rotationen um die x- und y-

Achse stabil betrieben werden kann. Insbesondere unter Okklusion führt der ESKF zu glatteren Geschwindigkeits- und Beschleunigungsverläufen, einer kürzeren Pfadlänge sowie zu einer schnelleren Konvergenz des IBVS-Fehlers als die ungefilterte Baseline.

Insgesamt kann damit gezeigt werden, dass die entwickelte Architektur eine robuste Grundlage für featurebasiertes IBVS unter schwierigen Bildbedingungen darstellt. Für das untersuchte System ergibt sich, dass der reine ESKF ohne LR den besten Kompromiss aus Robustheit, Laufruhe und Reproduzierbarkeit bietet.

13.2 Ausblick

Die vorliegende Arbeit hat gezeigt, dass eine robuste IBVS-Regelung auch unter schwierigen Sichtbedingungen durch die Kombination aus Referenzmerkmalen, modellbasierter Vorhersage und Filterung realisierbar ist. Aus den experimentellen Ergebnissen gehen Erweiterungen hervor, die das System sowohl hinsichtlich der Robustheit als auch der praktischen Einsetzbarkeit weiter verbessern könnten. Diese Ansätze werden im Folgenden skizziert.

Ein erster Ansatz besteht in der Implementierung einer automatisierten Kalibrierungsfahrt zur Parametrierung der Filter- und Entscheidungskomponenten. In der vorliegenden Arbeit werden Prozess- und Messrauschen sowie weitere Schwellenwerte weitgehend manuell gewählt. Dies ist für die experimentelle Untersuchung zweckmäßig, erschwert jedoch die Übertragbarkeit auf andere Objekte, Kamerakonfigurationen oder Bewegungsprofile. Daher schlagen wir vor, dass zukünftige Arbeiten eine definierte Kalibrierungsbewegung zu Beginn des Systems vorsehen, bei der Innovationsgrößen, Residuen und Merkmalsstabilitäten systematisch erfasst und zur automatischen Bestimmung geeigneter Filterparameter verwendet werden.

Ein weiterer relevanter Ausblick betrifft den Aufbau der Referenzdatenbank. Bislang erfolgt die Referenzaufnahme im Wesentlichen aus einer einzelnen Ausgangsperspektive. Zukünftig könnte die Datenbank bereits während der Initialisierung aus mehreren Bildern mit gezielt kleinen Blickwinkeländerungen aufgebaut werden. Auf diese Weise ließen sich diejenigen Keypoints identifizieren, die über benachbarte Ansichten hinweg besonders stabil, wiederholbar und zuverlässig matchbar sind.

Darüber hinaus deuten die experimentellen Ergebnisse darauf hin, dass die Filterung zwar unter Okklusion und bei Lichtwechsel einen klaren Robustheitsvorteil bietet, in der Endphase der Bewegung jedoch eine stärkere Orientierung an direkt beobachteten Bildmerkmalen vorteilhaft sein kann. Ursache hierfür ist, dass der Filter aus Echtzeitgründen nur auf einer begrenzten aktiven Merkmalsmenge arbeitet, während für die direkte Messung in Zielnähe deutlich mehr reale Keypoints zur Verfügung stehen können. Dadurch besitzt die ungefilterte Merkmalsauswertung unter günstigen Beobachtungsbedingungen das Potenzial für eine präzisere Feinjustierung. Wir schlagen daher vor, dass zukünftige Arbeiten eine kontinuierliche, zustandsabhängige Übergangsstrategie untersuchen. Anstatt den Filter abrupt zu deaktivieren, könnte mit abnehmendem Bildfehler und zunehmender Messzuverlässigkeit die Update-Frequenz schrittweise erhöht und die Regelung zunehmend stärker an realen Messdaten ausgerichtet werden. In der finalen Annäherungsphase wäre dann ein nahezu messdatengetriebener Betrieb denkbar, während bei Okklusion, Lichtstörungen oder unsicheren Beobachtungen weiterhin die robuste modellbasierte Schätzung dominiert. Auf diese Weise ließe sich die

Robustheit des Filters in den frühen Bewegungsphasen mit der höheren Präzision einer direkten Feinjustierung in Zielnähe verbinden.

Ergänzend könnte eine solche Übergangsstrategie mit Mechanismen zur Sicherstellung günstiger Sichtbedingungen kombiniert werden. Insbesondere Ansätze zur Berücksichtigung von Sichtfeldgrenzen und stabiler Merkmalsbeobachtung könnten dazu beitragen, auch während der Feinjustierung eine konsistente Merkmalsbasis aufrechtzuerhalten.

Insgesamt zeigen diese Ansätze, dass die entwickelte Architektur eine geeignete Grundlage für weiterführende Forschungsarbeiten bildet. Die Verbindung aus automatisierter Filterparametrierung, mehransichtiger Referenzbildung und einer kontinuierlichen, phasenabhängigen Anpassung der Merkmalsverwendung bietet ein hohes Potenzial, die Robustheit und Präzision des Systems weiter zu steigern.

A Anhang

A.1 Ablaufdiagramme Filter

Im Folgenden werden die Ablaufdiagramme zu den in Kapitel 3 vorgestellten Filtermethoden dargestellt.

A.1.1 Standard Kalman-Filter (KF)

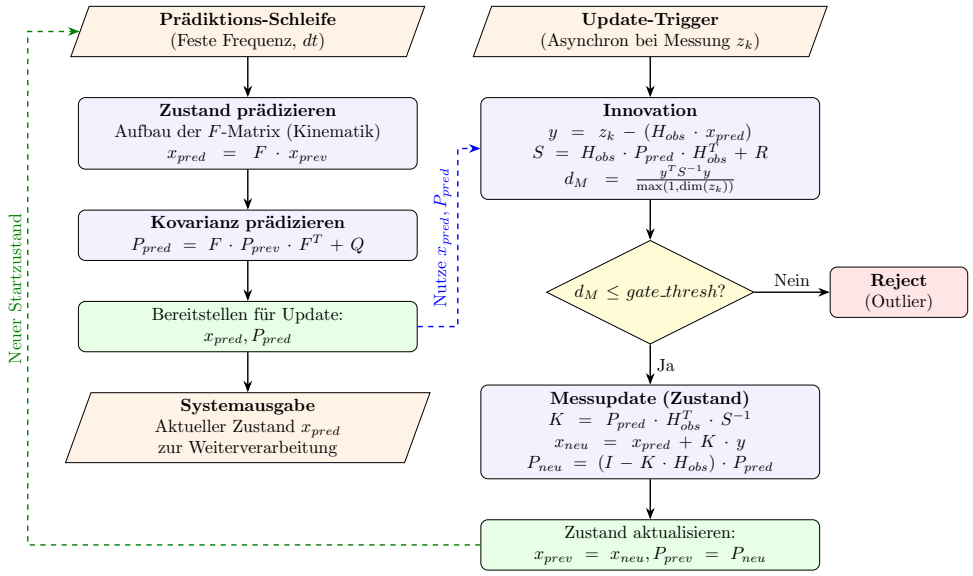


Abbildung A.1: Ablaufdiagramm des KF.

A.1.2 Extended Kalman-Filter (EKF)

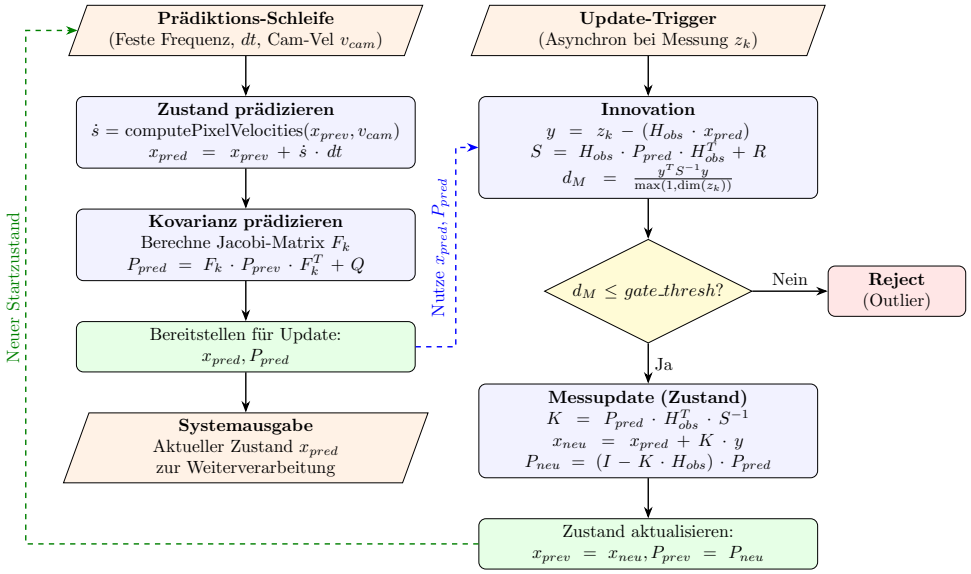


Abbildung A.2: Ablaufdiagramm des EKF.

A.1.3 Unscented Kalman-Filter (UKF)

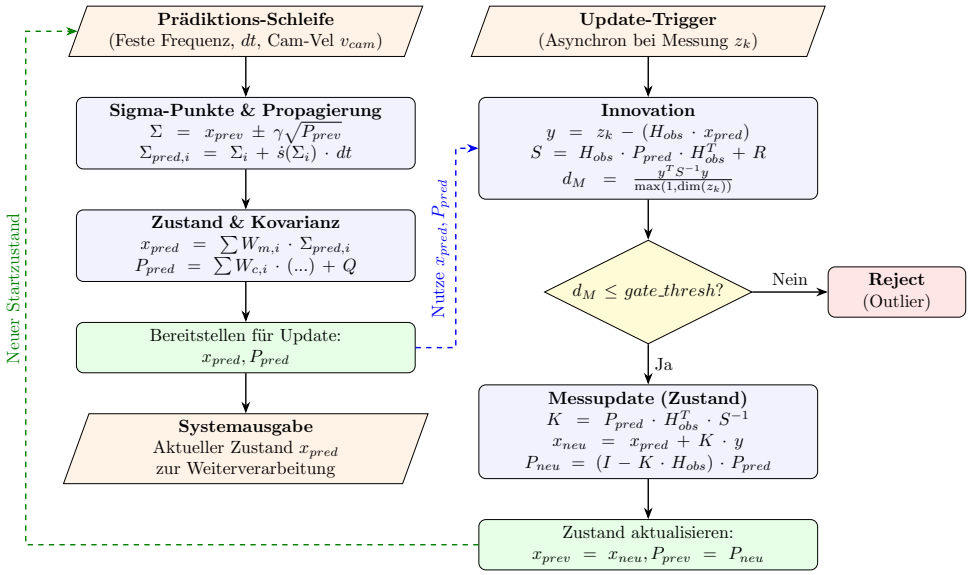


Abbildung A.3: Ablaufdiagramm des UKF.

A.1.4 Error State Kalman-Filter (ESKF)

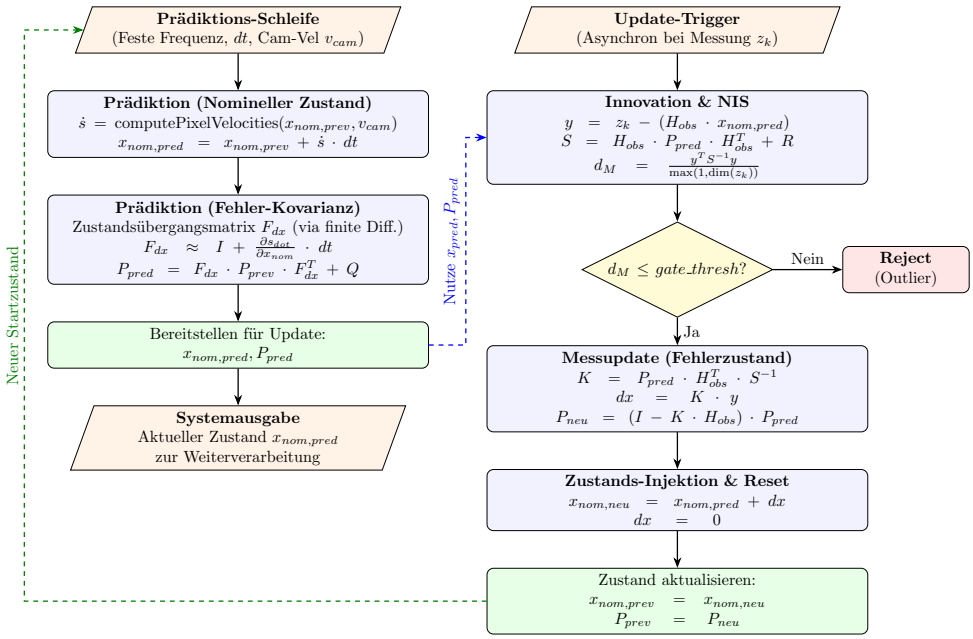


Abbildung A.4: Ablaufdiagramm des ESKF.

A.1.5 1€-Filter

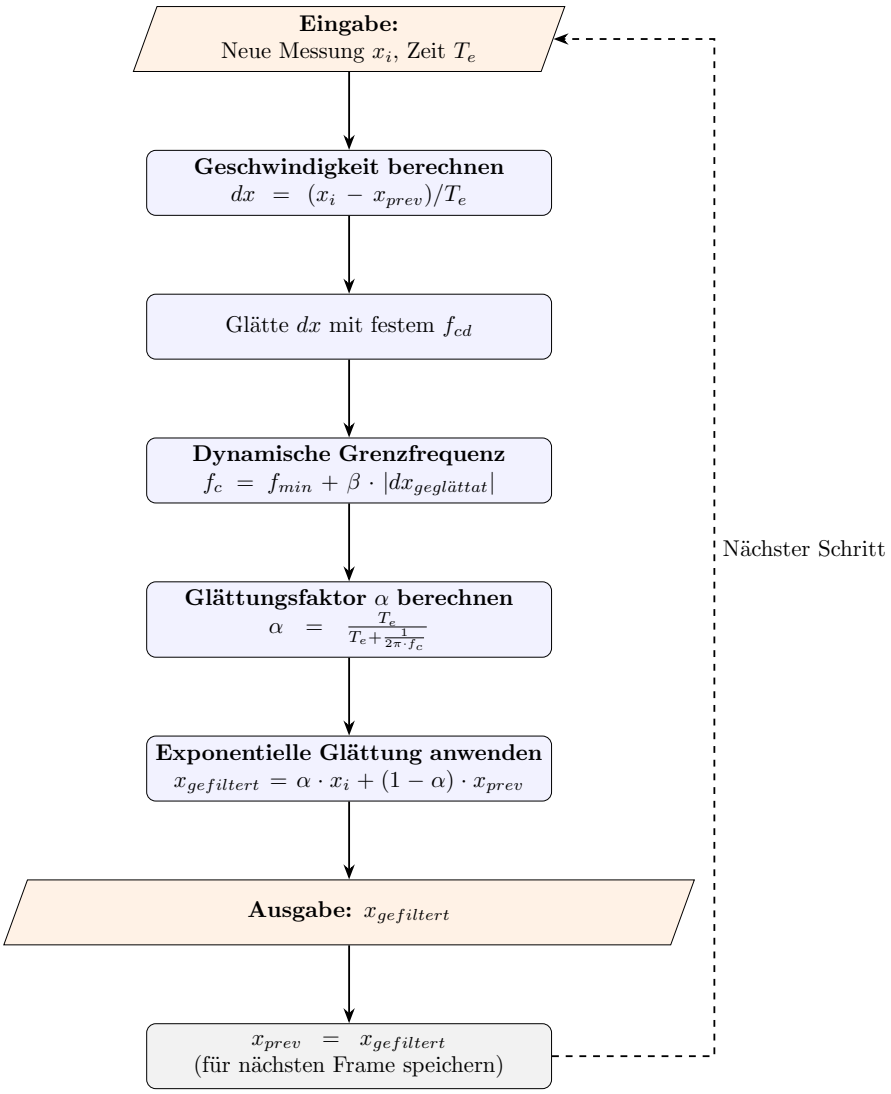


Abbildung A.5: Ablaufdiagramm des 1€-Filters.

Literaturverzeichnis

[1] François Chaumette and Seth Hutchinson. Visual servo control. i. basic approaches. *IEEE Robotics & Automation Magazine*, 13(4):82–90, 2006. doi: 10.1109/MRA.2006.250573.

[2] Peter Corke. *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, volume 118 of *Springer Tracts in Advanced Robotics*. Springer, 2nd edition, 2017.

[3] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2022.

[4] Guilherme Potje, Felipe Cadar, André Araujo, Renato Martins, and Erickson R. Nascimento. Xfeat: Accelerated features for lightweight image matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. doi: 10.1109/CVPR52733.2024.00259.

[5] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, Cambridge, MA, USA, 2005.

[6] François Chaumette and Seth Hutchinson. Visual servo control. ii. advanced approaches [tutorial]. *IEEE Robotics & Automation Magazine*, 14(1):109–118, 2007. doi: 10.1109/MRA.2007.339609.

[7] Greg Welch and Gary Bishop. An introduction to the kalman filter. 1995.

[8] Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[9] Greg Welch and Gary Bishop. An introduction to the kalman filter. Technical report, University of North Carolina at Chapel Hill, Chapel Hill, NC, USA, 2006.

[10] Joseph B Collins and Jeffrey K Uhlmann. Efficient gating in data association with multivariate gaussian distributed states. *IEEE Transactions on Aerospace and Electronic Systems*, 28(3):909–916, 1992.

[11] Hugh F Durrant-Whyte, BYS Rao, and Huosheng Hu. Toward a fully decentralized architecture for multi-sensor data fusion. In *Proceedings., IEEE International Conference on Robotics and Automation*, pages 1331–1336. IEEE, 1990.

[12] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory Algorithms and Software*. John Wiley & Sons, New York, NY, USA, 2001.

[13] Simon J. Julier and Jeffrey K. Uhlmann. New extension of the kalman filter to nonlinear systems. In *Signal Processing, Sensor Fusion, and Target Recognition VI*, volume 3068, pages 182–193. SPIE, 1997.

[14] Joan Solà. Quaternion kinematics for the error-state kalman filter. *arXiv preprint arXiv:1711.02508*, 2017.

[15] Christian Forster, Luca Carlone, Frank Dellaert, and Davide Scaramuzza. On-manifold preintegration for real-time visual–inertial odometry. *IEEE Transactions*

on Robotics, 33(1):1–21, 2017.

[16] G. Casiez, N. Roussel, and D. Vogel. 1€ filter: a simple speed-based low-pass filter for noisy input in interactive systems. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 2012.

[17] Gregory D. Hager, Seth Hutchinson, and Peter I. Corke. A tutorial on visual servo control. *IEEE Transactions on Robotics and Automation*, 12(5):651–670, 1996.

[18] Nicolás García-Aracil, Ezio Malis, Rafael Aracil-Santonja, and Carlos Pérez-Vidal. Continuous visual servoing despite the changes of visibility in image features. *IEEE Transactions on Robotics*, 21(6):1214–1220, 2005. doi: 10.1109/TRO.2005.851377.

[19] Charalampos P. Bechlioulis, Shahab Heshmati-alamdari, George C. Karras, and Kostas J. Kyriakopoulos. Robust image-based visual servoing with prescribed performance under field of view constraints. *IEEE Transactions on Robotics*, 35 (4):1063–1070, 2019. doi: 10.1109/TRO.2019.2893374.

[20] Kai Zhang, Fei Wang, et al. Occlusion-free image-based visual servoing using probabilistic control barrier certificates. *arXiv preprint arXiv:2309.03476*, 2023.

[21] Ezio Malis, François Chaumette, and Sylvie Boudet. 2-1/2-d visual servoing. *IEEE Transactions on Robotics and Automation*, 15(2):238–250, April 1999. doi: 10.1109/70.760345.

[22] Gabriel Silveira and Ezio Malis. Direct visual servoing: Vision-based estimation and control using only nonmetric information. *IEEE Transactions on Robotics*, 28 (4):974–980, 2012. doi: 10.1109/TRO.2012.2189498.

[23] Farrokh Janabi-Sharifi and Mohammed Marey. A kalman-filter-based method for pose estimation in visual servoing. *IEEE Transactions on Robotics*, 26(5):939–947, 2010. doi: 10.1109/TRO.2010.2061290.

[24] Ting-Yu Chang, Wei-Che Chang, Ming-Yang Cheng, and Shih-Sian Yang. Dynamic visual servoing with kalman filter-based depth and velocity estimator. *International Journal of Advanced Robotic Systems*, 18(3), 2021. doi: 10.1177/17298814211016674.

[25] Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superpoint: Self-supervised interest point detection and description. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 224–236, 2018.

[26] Paul-Edouard Sarlin, Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superglue: Learning feature matching with graph neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4938–4947, 2020.

[27] Jérôme Revaud, Philippe Weinzaepfel, César De Souza, Noé Pion, Gabriela Csurka, Yohann Cabon, and Martin Humenberger. R2d2: Repeatable and reliable detector and descriptor. In *Advances in Neural Information Processing Systems*, 2019.

[28] Quentin Bateux, Eric Marchand, Jürgen Leitner, François Chaumette, and Peter Corke. Visual servoing from deep neural networks, 2017.

[29] Open Robotics. Understanding ros 2 nodes. https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html, 2026. Zugriff am 23.03.2026.

[30] Open Robotics. Understanding ros 2 topics. https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html, 2026. Zugriff am 23.03.2026.

[31] RealSense. Stereo depth camera d435. https://realsenseai.com/products/stereo-depth-camera-d435/, 2026. Zugriff am 23.03.2026.

[32] RealSense. Intel realsense d400 series datasheet. https://www.realsenseai.com/wp-content/uploads/2023/10/Intel-RealSense-D400-Series-Datasheet-September-2023.pdf, 2023. Zugriff am 23.03.2026.

[33] Universal Robots. Technical specifications ur5e. https://www.universal-robots.com/manuals/EN/HTML/SW10_12_1/Content/prod-usr-man/complianceUR5e/H_g5_sections/appendix_g5/tech_spec_sheet.htm, 2026. Zugriff am 23.03.2026.

[34] Universal Robots. Universal robots ros 2 driver documentation. https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/, 2026. Zugriff am 23.03.2026.

[35] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.

[36] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. Orb: An efficient alternative to sift or surf. In *2011 IEEE International Conference on Computer Vision*, pages 2564–2571, 2011. doi: 10.1109/ICCV.2011.6126544.

[37] Andrew I. Comport, Éric Marchand, Muriel Pressigout, and François Chaumette. Real-time markerless tracking for augmented reality: The virtual visual servoing framework. *IEEE Transactions on Visualization and Computer Graphics*, 12(4):615–628, 2006. doi: 10.1109/TVCG.2006.78.

[38] Peter I. Corke and Michael C. Good. Dynamic effects in visual closed-loop systems. *IEEE Transactions on Robotics and Automation*, 12(5):671–683, 1996.

[39] E. A. Wan and R. Van Der Merwe. The unscented Kalman filter for nonlinear estimation. In *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium*, pages 153–158. IEEE, 2000.

[40] B. Williams, G. Klein, and I. Reid. Real-time SLAM relocalisation. In *Proceedings of the 11th IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2007.

[41] C. W. Wampler. Manipulator inverse kinematic solutions based on damped least-squares solutions. *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1):93–101, 1986.

[42] Richard Hartley and Andrew Zisserman. *Multiple view geometry in computer vision*. Cambridge university press, 2003.

[43] Margarita Chli and Andrew J Davison. Active matching. In *European conference on computer vision*, pages 72–85. Springer, 2008.

[44] Eric A. Wan and Rudolph van der Merwe. The unscented kalman filter for nonlinear estimation. In *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium*, pages 153–158, 2000. doi: 10.1109/ASSPCC.2000.882463.

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Karlsruhe, 31. März 2026

Paul Glaser, B. Eng

Tim Schäfer, B. Eng

Felix Wietschel, B. Eng